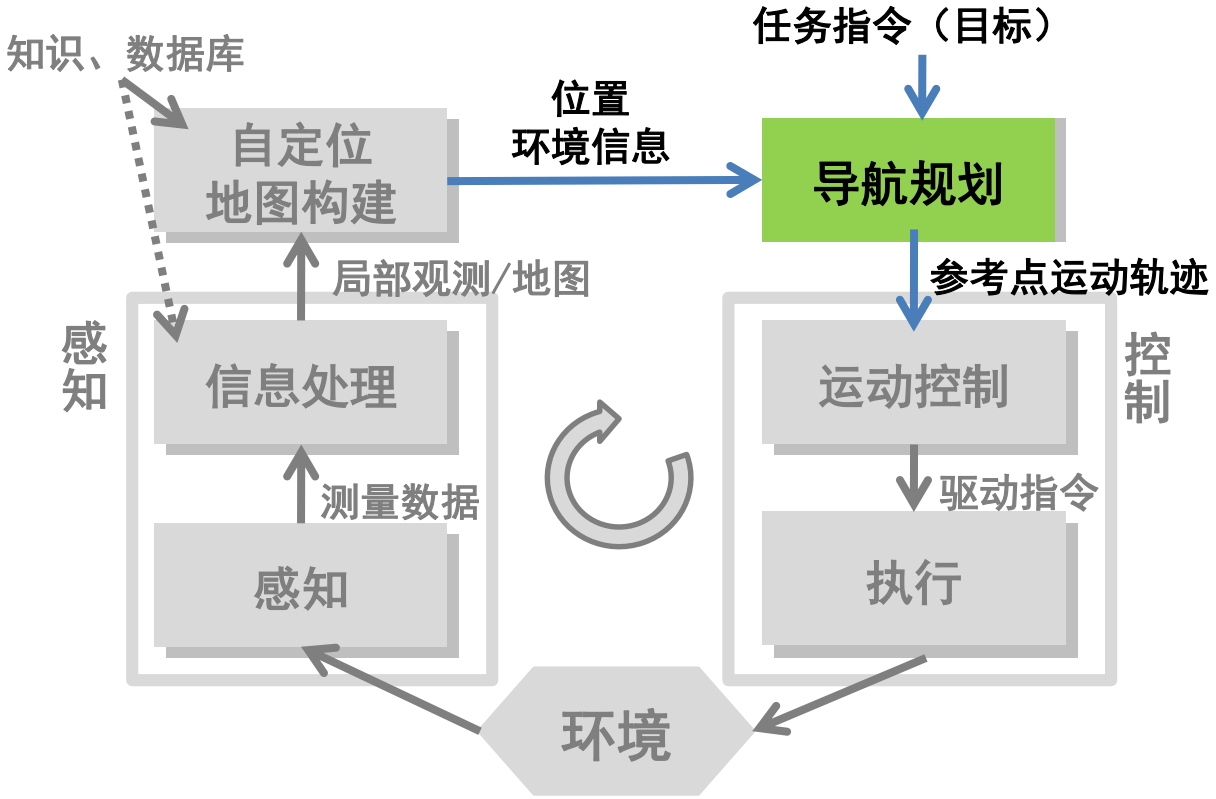
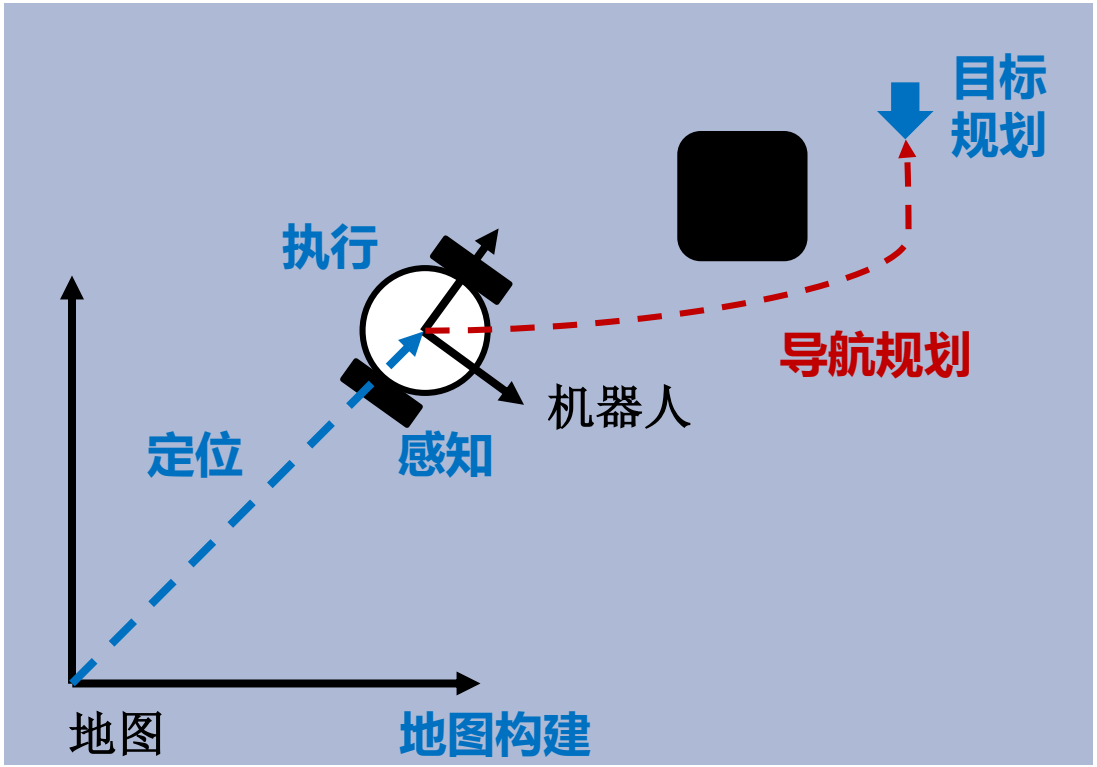
PART 1 导航规划

自主移动机器人一般架构



导航规划

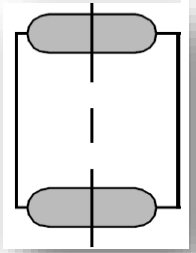


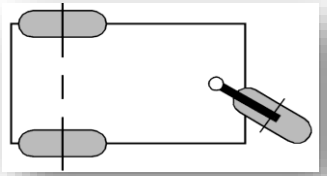
在给定环境的全局或局部知识以及一个或者一系列目标位置的条件下，使机器人能够根据知识和传感器感知信息高效可靠地到达目标位置

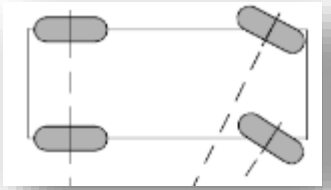
导航规划需要考虑的约束

- 环境几何约束
- 机器人执行约束
  - 最大速度
  - 最大加速度
  - 非完整性约束
  - 最小转弯半径 等

非完整性（NON-HOLOMIC）约束



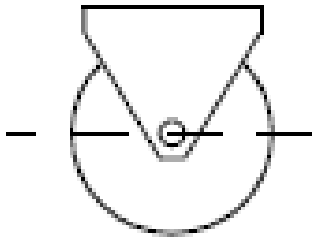




非完整性（NON-HOLOMIC）约束

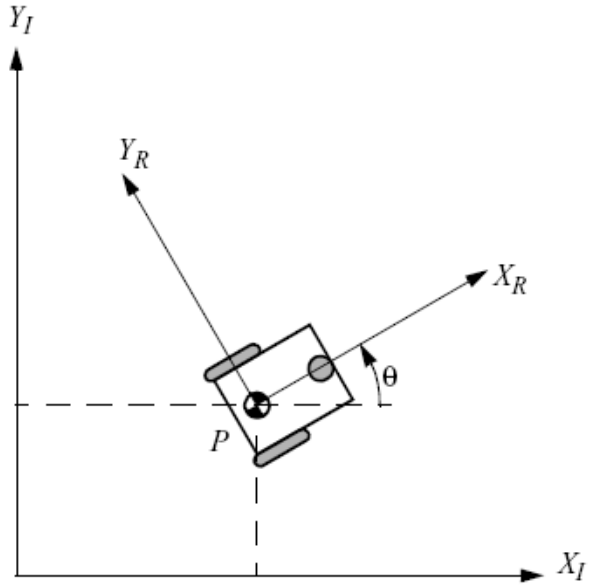
标准轮存在两个约束

- **滚动约束**，即轮子在相应方向发生运动时必须转动，即沿着轮平面的所有运动必须通过适当的旋转转量实现
- **无侧滑约束**，即轮子不能在垂直于轮子平面的方向发生滑动，即轮子垂直于轮平面的运动分量必须为零

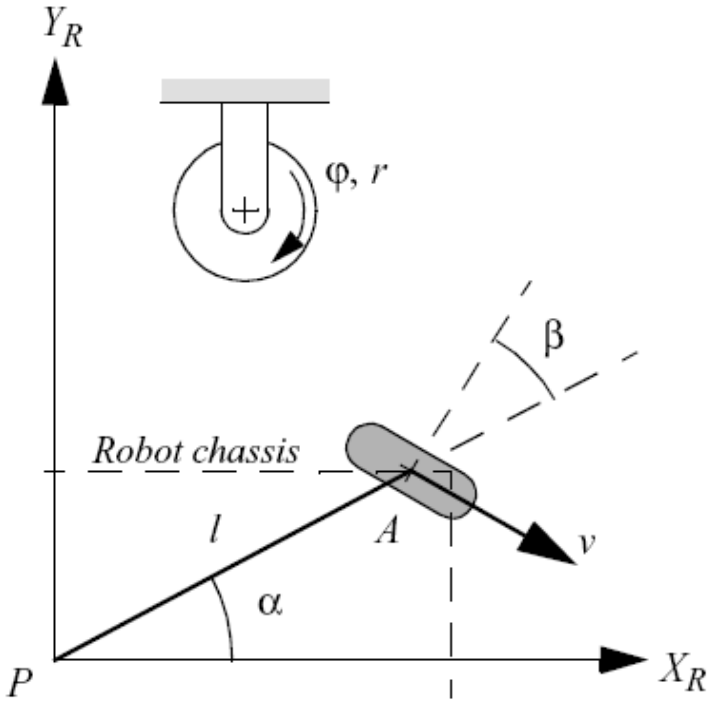


$$v_{\parallel} = r\dot{\varphi}$$

$$v_{\perp} = 0$$



$$\dot{\mathbf{X}}_R = R(q)\dot{\mathbf{X}}_I$$



机器人坐标系下，轮A的位置为 (l,α)

轮平面法向量相对于PA连线的角度为 β

非完整性（NON-HOLOMIC）约束

标准轮存在两个约束

- **滚动约束**

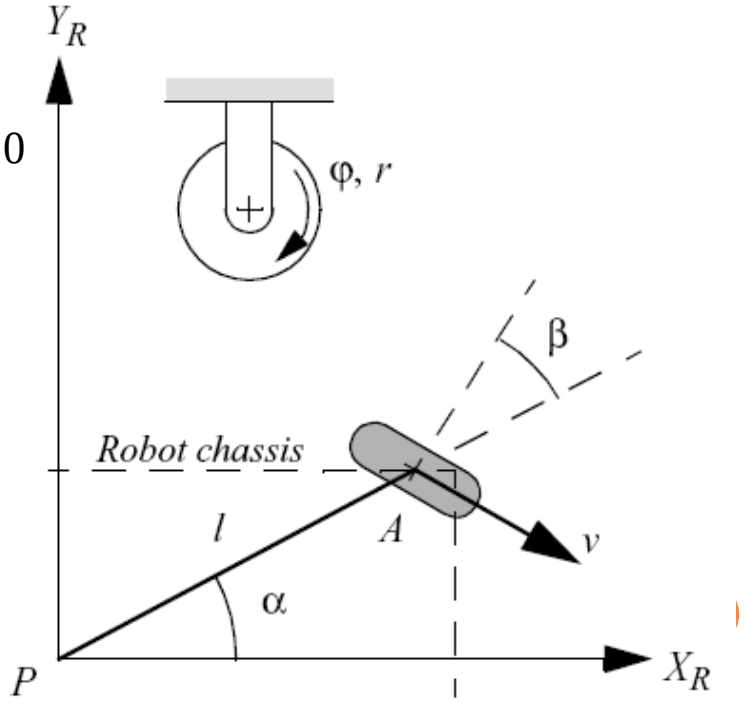
$$\begin{bmatrix} \sin(a+b) & -\cos(a+b) & (-l)\cos b \end{bmatrix} R(q)\dot{\mathbf{X}}_I - r\dot{j} = 0$$

完整约束：可表示为仅包含位置变量的函数

- **无侧滑约束**

$$\begin{bmatrix} \cos(a+b) & \sin(a+b) & l\sin b \end{bmatrix} R(q)\dot{\mathbf{X}}_I = 0$$

非完整约束：需要微分关系，且无法通过积分得到一个只包含位置变量的约束



最小转弯半径

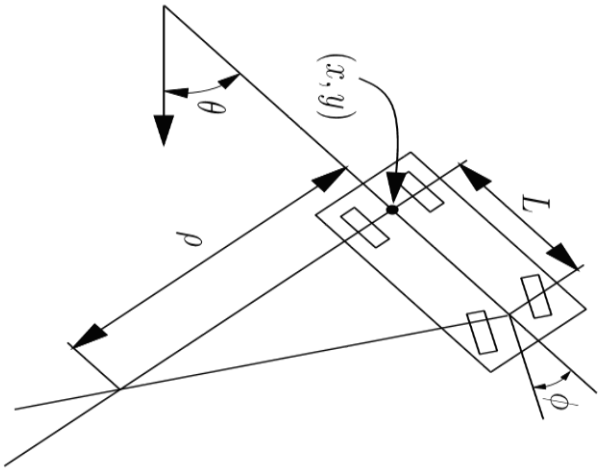
- 最小转弯半径

Ackman底盘存在最大转向角度约束

$$|\phi| \leq \phi_{max} < \frac{\pi}{2}$$

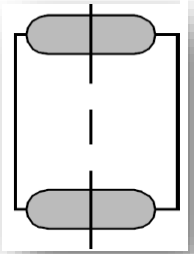
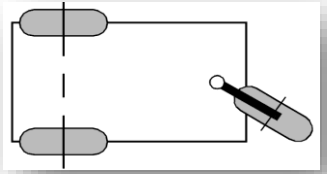
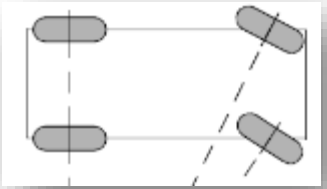
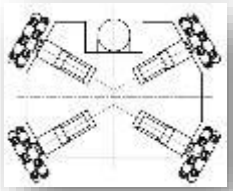
导致车辆存在最小转弯半径

$$\rho_{min} = L/\tan(\phi_{max})$$



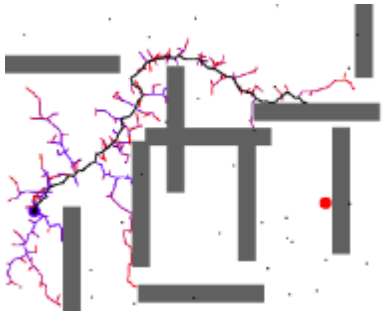
移动机器人形态丰富，运动学模型和约束各不相同

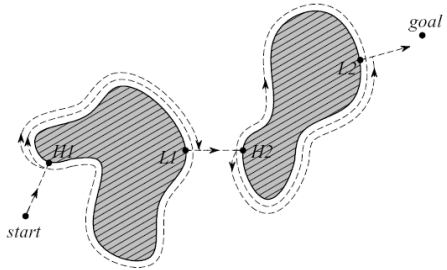


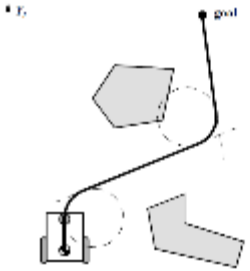






导航规划通常分解为三个问题

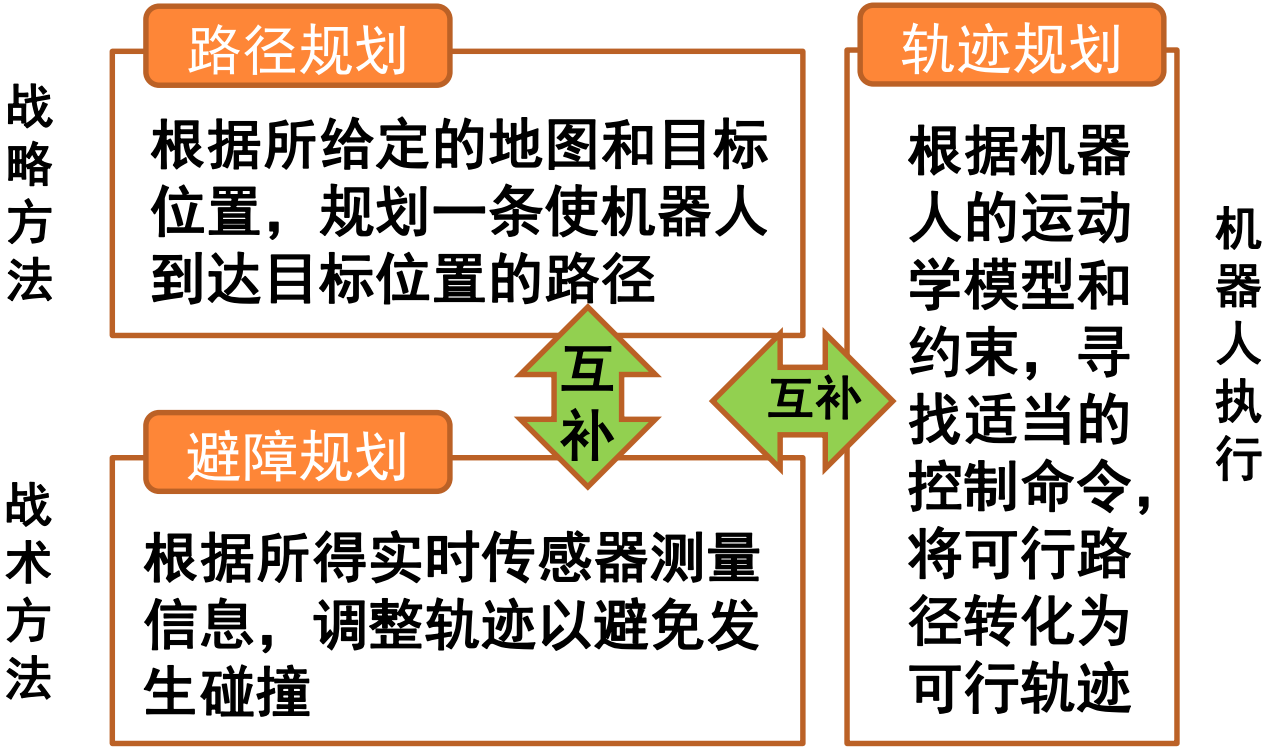
- **路径规划**：根据所给定的地图和目标位置，规划一条使机器人到达目标位置的路径（只考虑工作空间的几何约束，不考虑机器人的运动学模型和约束）
- **避障规划**：根据所得到的实时传感器测量信息，调整路径/轨迹以避免发生碰撞
- **轨迹生成**：根据机器人的运动学模型和约束，寻找适当的控制命令，将可行路径转化为可行轨迹。



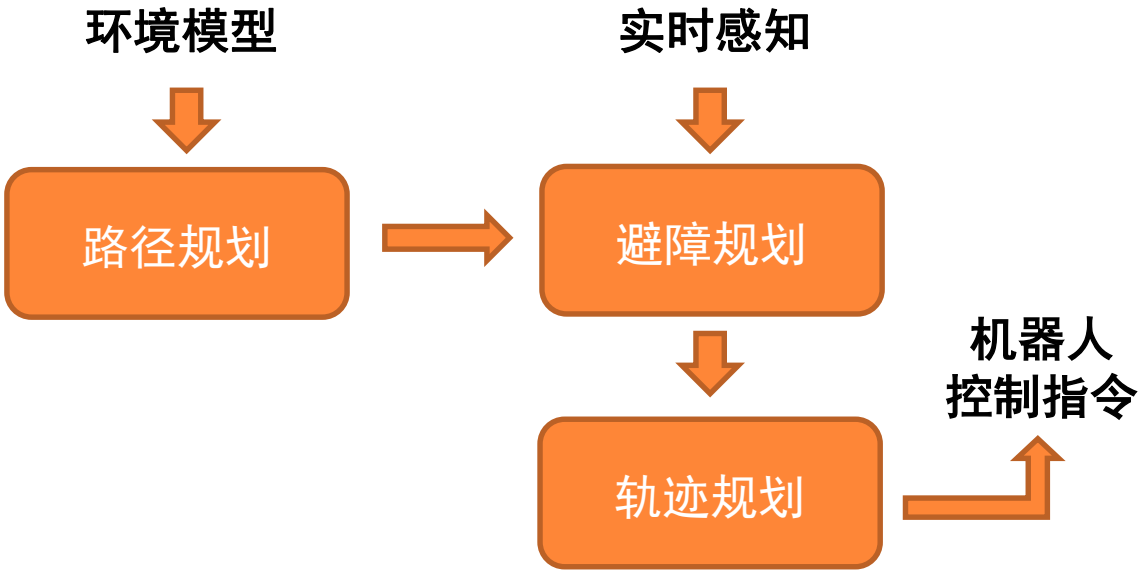




路径规划、避障规划、轨迹规划三者关系



路径规划、避障规划、轨迹规划三者关系

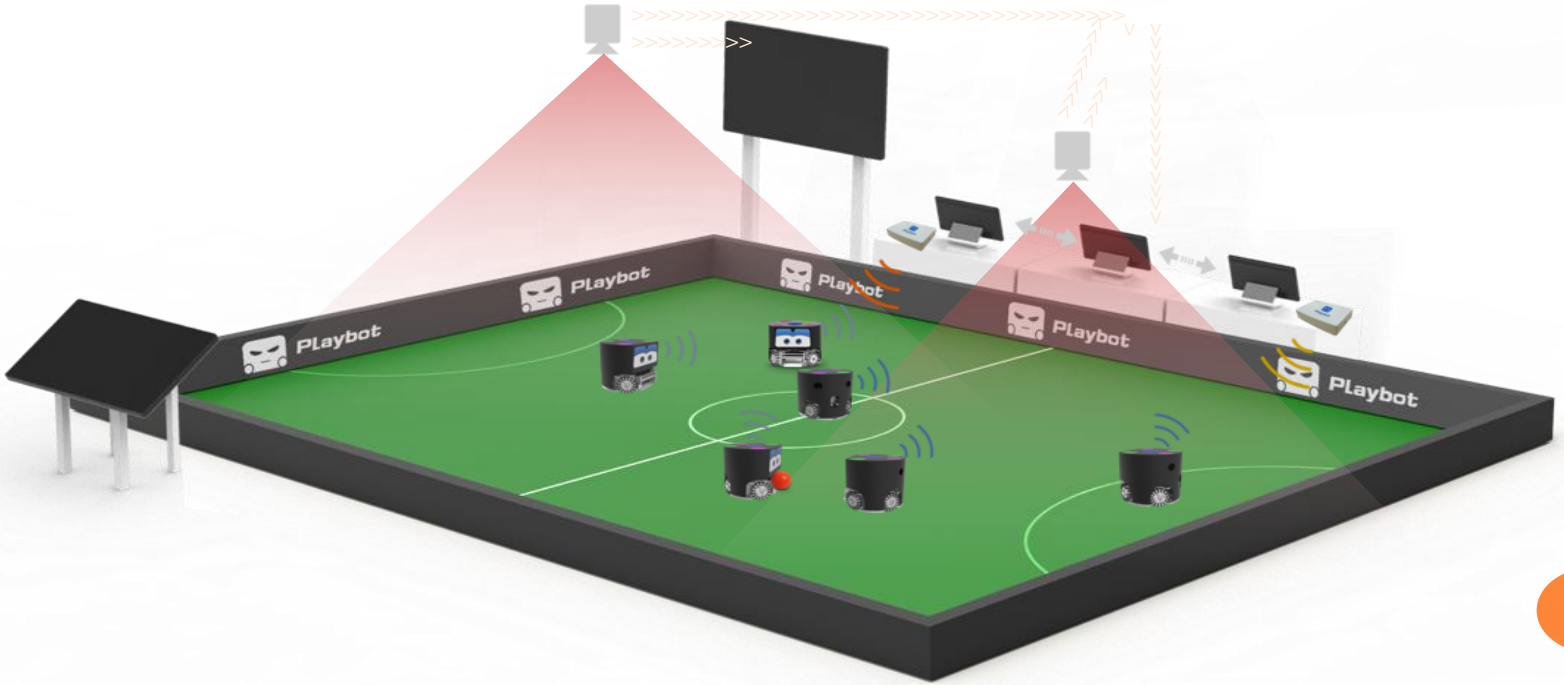


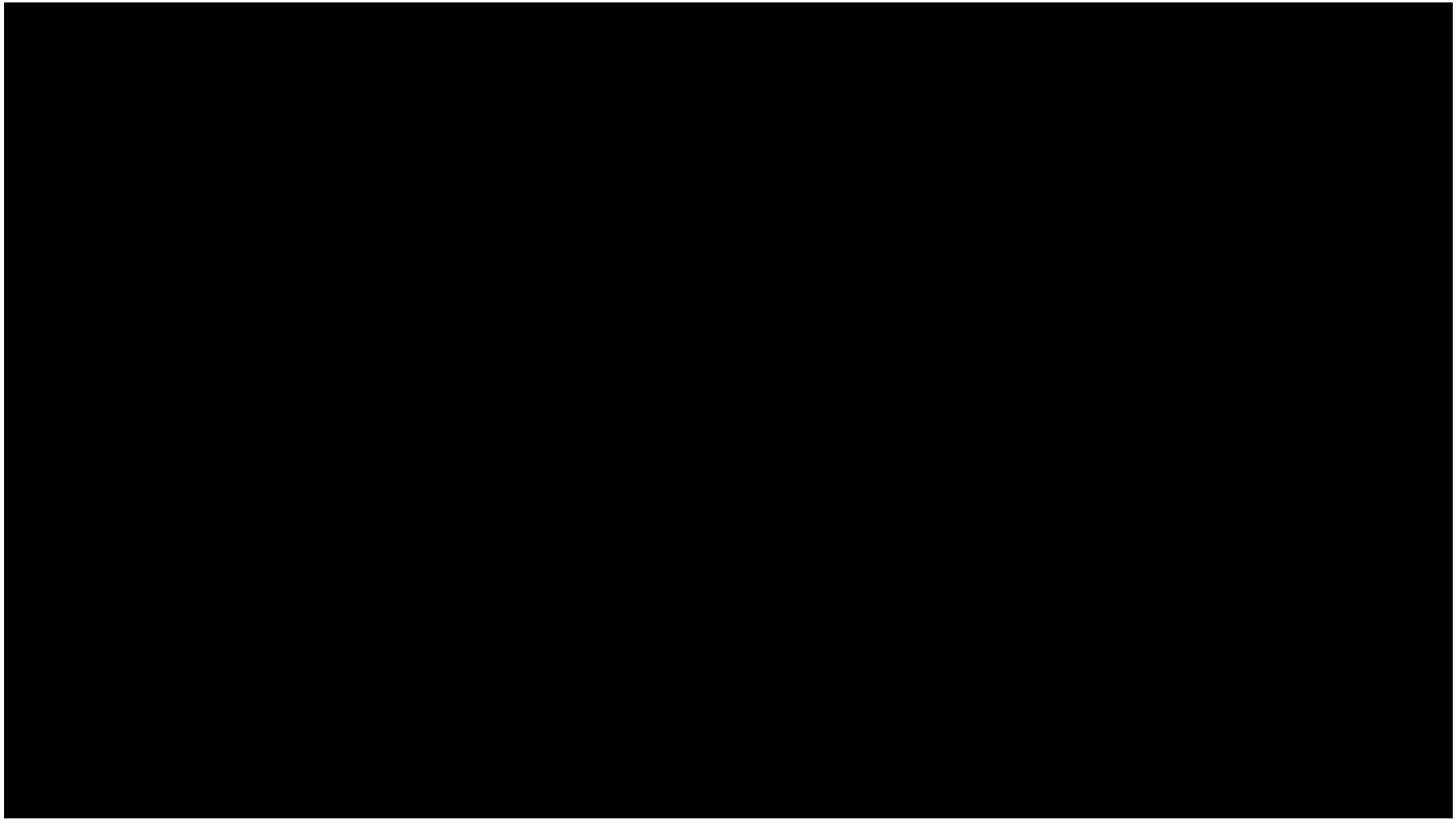
问题：实际应用中这三个模块都必须具备吗?

导航规划作业

- 要求在小型足球机器人仿真平台上实现动静态环境下的路径规划和轨迹规划，机器人采用差分驱动方式，需在给定点之间快速安全地来回行走。
- 测试环境1: 环境中有若干静态障碍物（用其他小车模拟），由系统随机生成；
- 测试环境2: 环境中有若干动态障碍物（用其他小车模拟），由系统随机生成并随机运动
- 测试评分：
  - 根据时间，机器人需在两点之间往返到点，到点评判标准为到达给定点中心10cm半径范围内，往返次数5次，根据所用时间评分，时间少得分高；
  - 根据安全性，计算碰撞次数，碰撞少得分高。
- 实践报告
- 具体要求和说明见“学在浙大”。
- 助教：贾慎涵、林隆中

小型足球机器人系统






D
5
E
9
F
1
3
2
B
4
A
8
0
6
7

第二讲 路径规划

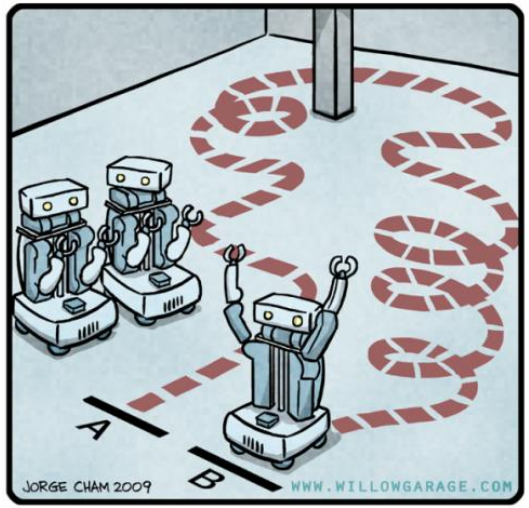
熊蓉

浙江大学 控制科学与工程学院

2.1 基本概念

路径规划

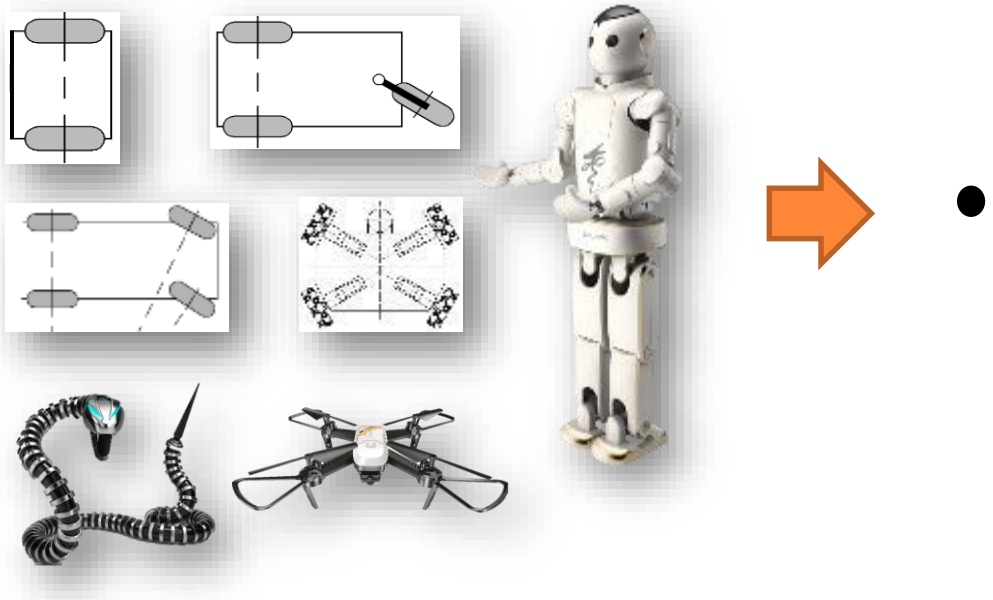
- 根据所给定的地图和目标位置，规划一条使机器人到达目标位置的路径



"HIS PATH-PLANNING MAY BE SUB-OPTIMAL, BUT IT'S GOT FLAIR."

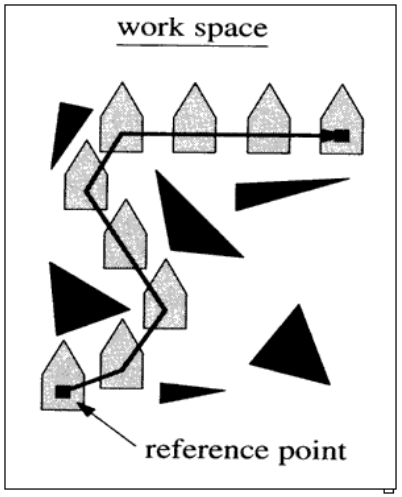
路径规划

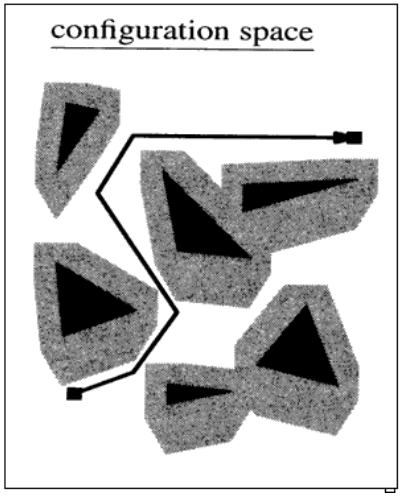
- 只考虑工作空间的几何约束，不考虑机器人的运动学模型和约束
- 可将移动机器人简化为工作空间中的一个点



工作空间与位形空间(C-SPACE)

工作空间





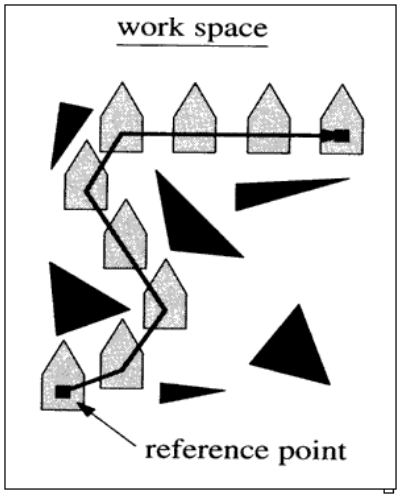
位形空间
(Configuration
Space)

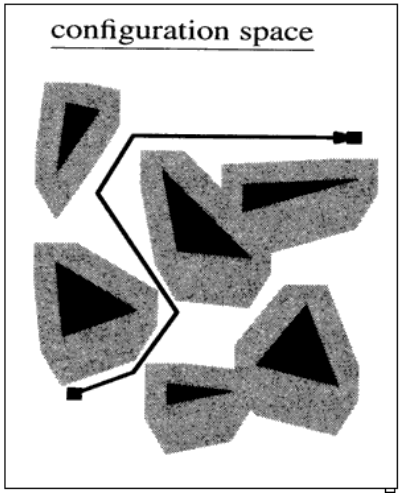
工作空间：移动机器人上的参考点能达到的空间集合，机器人采用位置和姿态描述，并需考虑体积

位形空间：机器人成为一个可移动点，不考虑姿态、体积和非完整运动学约束

工作空间与位形空间(C-SPACE)

工作空间





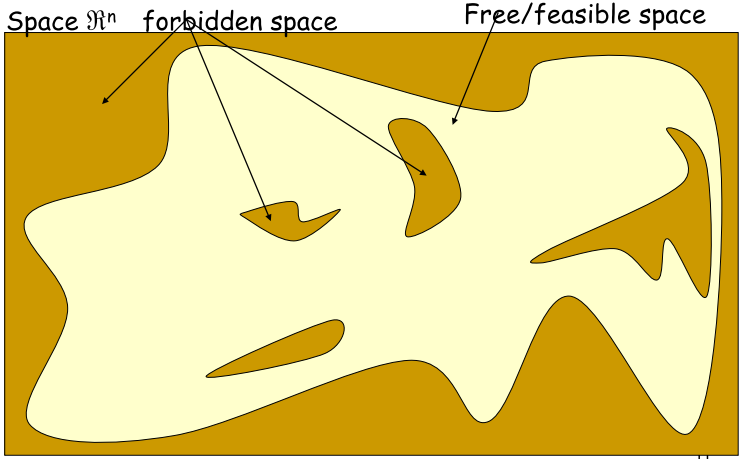
位形空间
(Configuration
Space)

障碍物按机器人半径进行膨胀 → 机器人成为一个点

忽略非完整约束对姿态的限制 → 机器人是完整的

位形空间

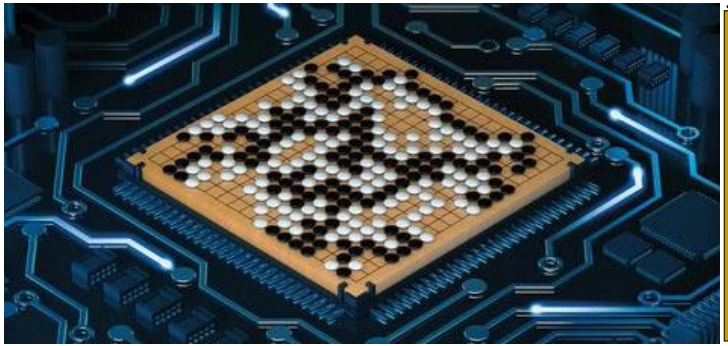
- 障碍物空间：不可行的位形集合
  - 在该空间中，机器人会与障碍物发生碰撞
- 自由空间：可行的位形集合
  - 在该空间中，机器人将无碰地安全移动

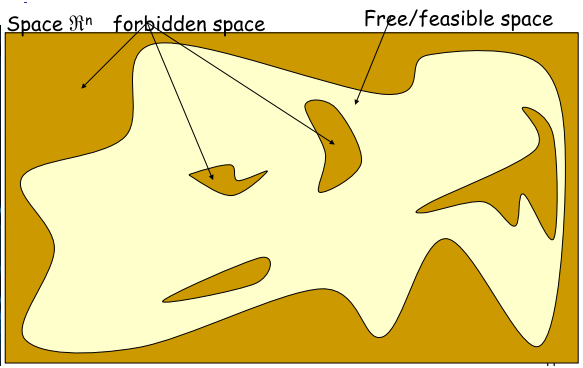


路径规划就是在自由位形空间中为机器人寻找一条路径，使其从初始位置运行到目标位置

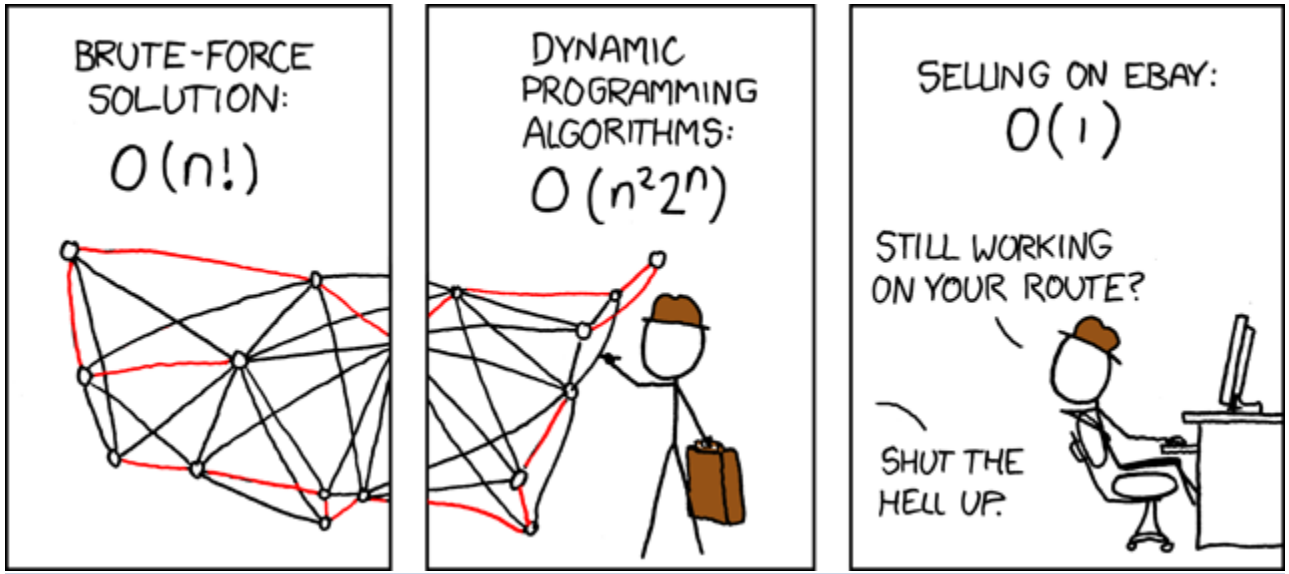
路径规划的完备性要求

- 完备性：当解存在时，能够在有限的时间内找到解
- 路径规划算法挑战：
  - 在连续空间内搜索，难以保证时间





一种常见的路径规划



拓扑连通图+最优路径搜索

路径规划的两个基本问题

拓扑连通图构建方法

- 基本思路：对空间作离散化
- 分辨率完备resolution completeness：
  解析性离散化，确保获得可行解
  - 行车图法：基于障碍物几何形状分解姿态空间
  - 单元分解法：区分空闲单元和被占单元
  - 势场法： 根据障碍物和目标对空间各点施加虚拟力
- 概率完备Probabilistic completeness:
  基于概率进行随机采样离散化，使获得解的概率趋近于1
  - PRM（Probabilistic RoadMaps）
  - RRT（Rapid-Exploring Radom Trees ）

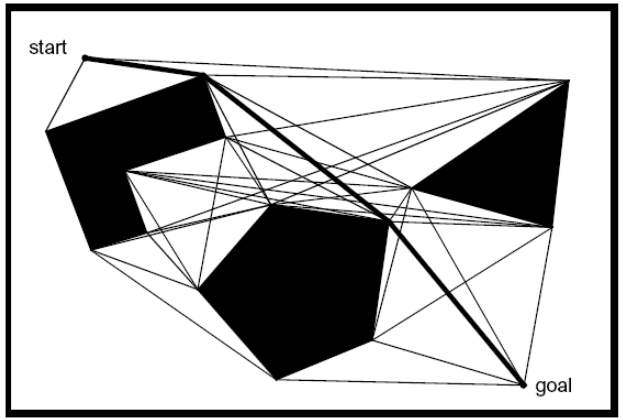
最优路径搜索方法

- 精确最优搜索法：深度优先法、宽度优先法
- 近似最优搜索法
  - 启发式搜索法：A*，D*
  - 准启发式搜索算法：退火、进化和蚁群优化等

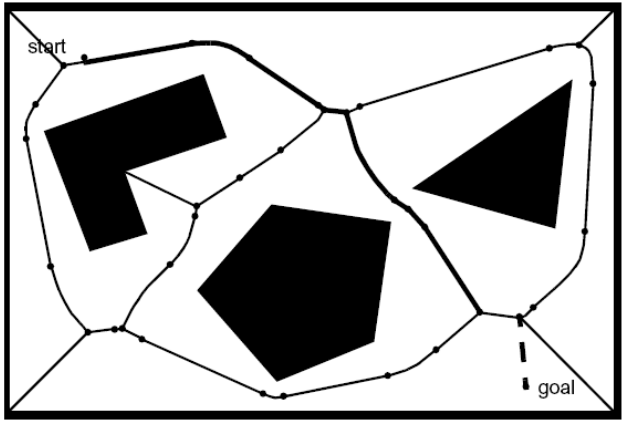
2.2 分辨率完备的连通图构建

1. 行车图法

- 基本思想：基于障碍物几何形状分解位形空间，将自由空间的连通性用一维曲线的网格表示，在加入起始点和目标点后，在该一维无向连通图中寻找一条无碰路径
- 构建行车图的典型方法：



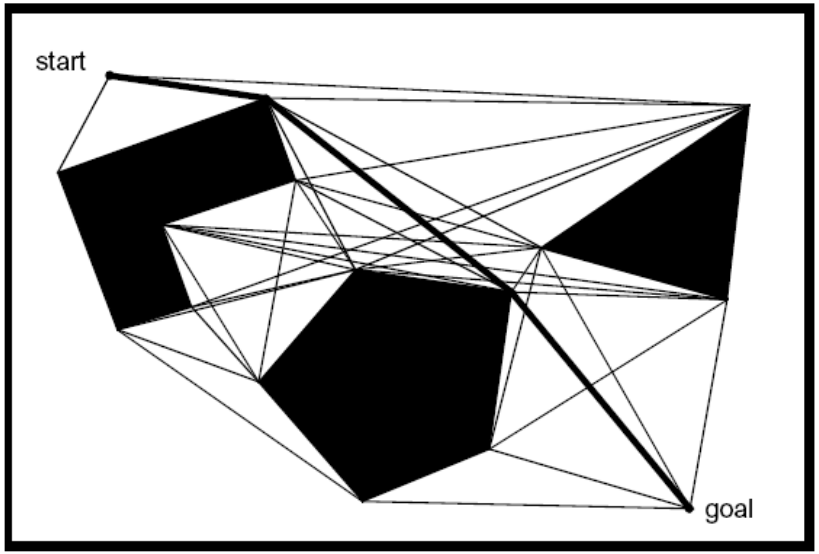
可视图（Visibility graph）



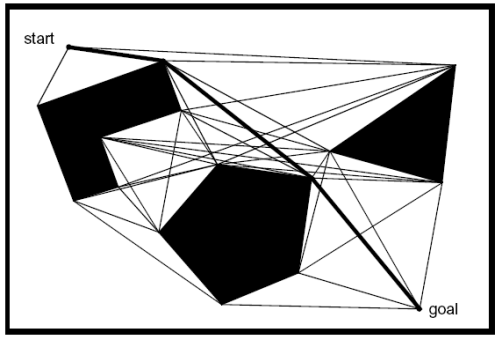
Voronoi diagram

1.1 可视图法

- 可视图由所有连接可见顶点对的边组成
  - 可见指顶点之间无障碍物
  - 初始位置和目标位置也作为顶点



1.1 可视图法



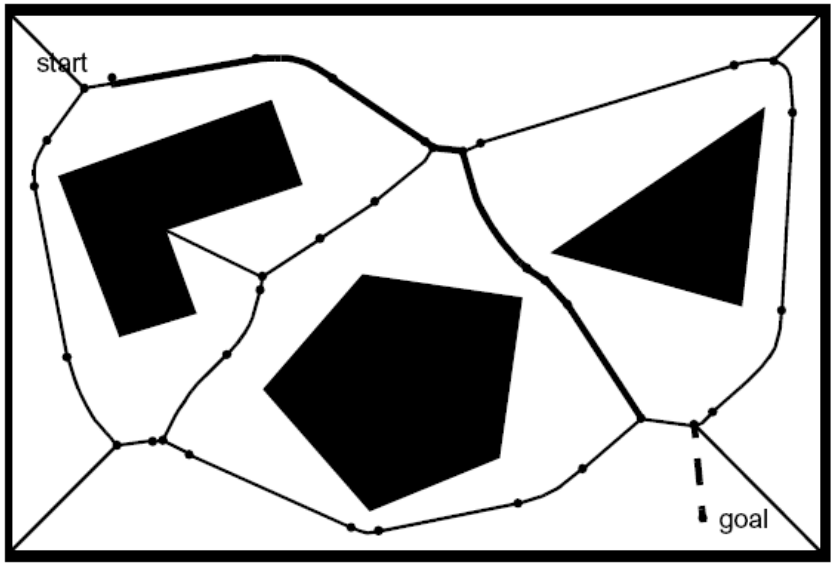
- **优点：**
  - 非常简单，特别是当环境地图用多边形描述物体时
  - 可得到在路径长度上最优的解
- **缺点：**
  - 所得路径过于靠近障碍物，不够安全。

常用的解决方法：

- 以远大于机器人半径的尺寸膨胀障碍物，但容易造成可行路径的消失
- 在路径规划后修改所得路径，使其与障碍物保持一定的距离

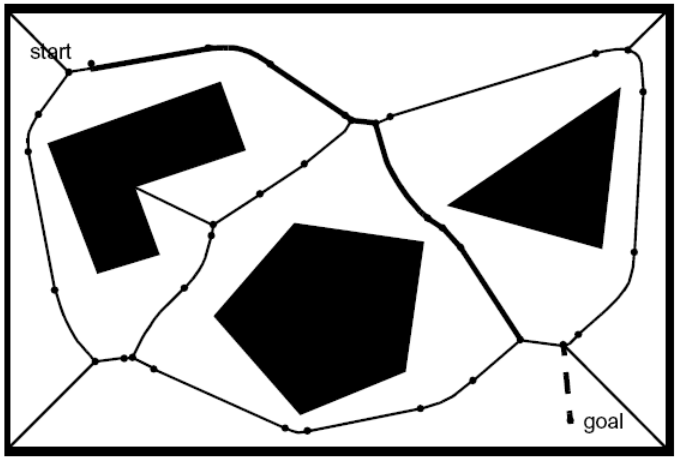
1.2 Voronoi diagram

- 基本思想：取障碍物之间的中间点，以最大化机器人和障碍物之间的距离



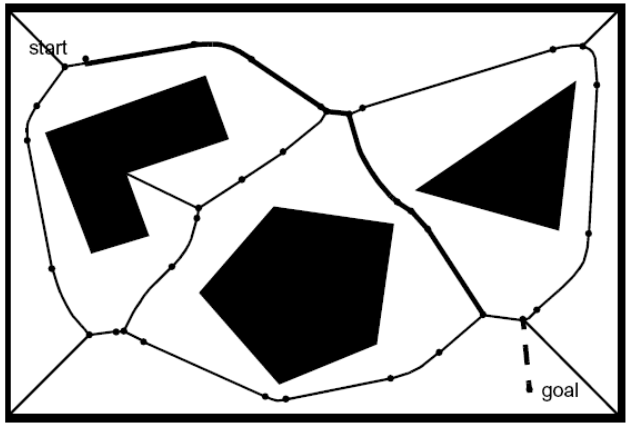
1.2 Voronoi diagram

- 构建方法：
  - 对于自由空间中的每一点，计算它到最近障碍物的距离；
  - 在垂直于二维空间平面的轴上用高度表示该点到障碍物的距离，类似于画直方图；
  - 当某个点到两个或多个障碍物距离相等时，其距离点处出现尖峰，Voronoi diagram就由连接这些尖峰点的边组成。



1.2 Voronoi diagram

- **优点：**安全性高
- **缺点：**计算复杂、路径长度较可视图法长、不适用于短距离定位传感器

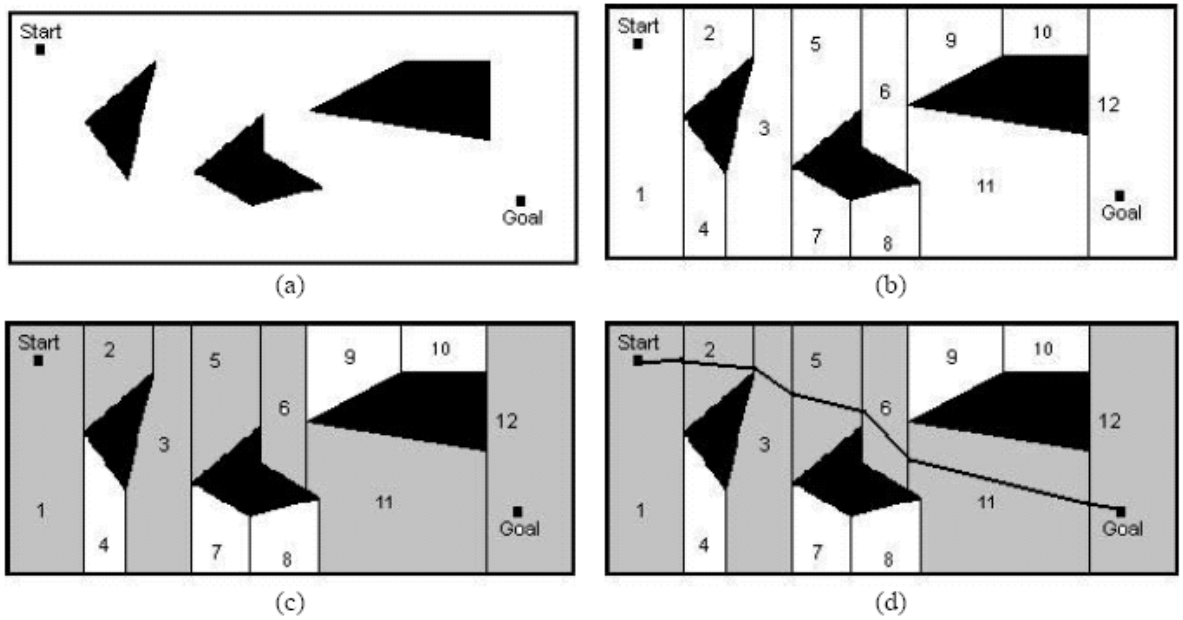


2. 单元分解法

- 基本思想
  - 首先，将位形空间中的自由空间分为若干的小区域，每一个区域作为一个单元，以单元为顶点、以单元之间的相邻关系为边构成一张连通图；
  - 其次，在连通图中寻找包含初始姿态和目标姿态的单元，搜索连接初始单元和目标单元的路径；
  - 最后，根据所得路径的单元序列生成单元内部的路径
- 主要方法
  - 精确单元分解
  - 近似单元分解

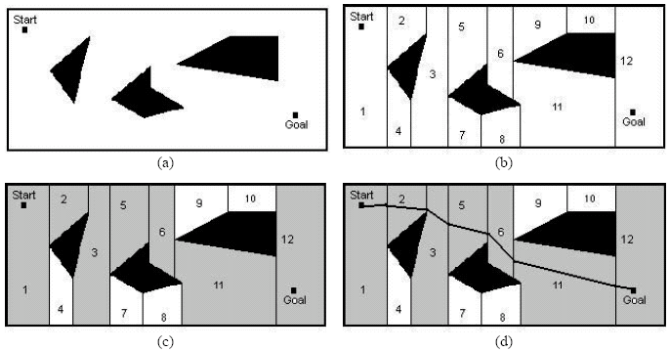
2.1 精确单元分解

- 单元边界严格基于环境几何形状分解，所得单元完全空闲



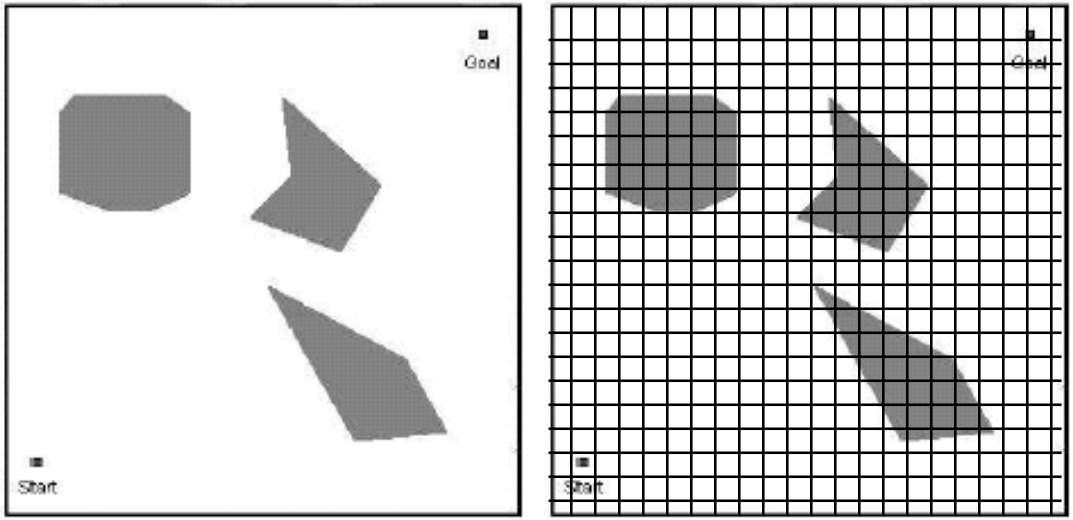
2.1 精确单元分解

- 优点：
  - 机器人不需要考虑它在每个空闲单元中的具体位置，只需要考虑如何从一个单元移动到相邻的空闲单元
  - 单元数与环境大小无关
- 缺点：计算效率极大地依赖于环境中物体的复杂度



2.2 近似单元分解

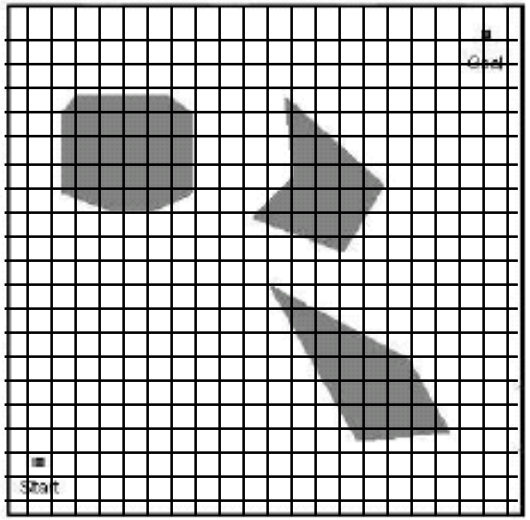
- 栅格表示法，将环境分解成若干个大小相同的栅格



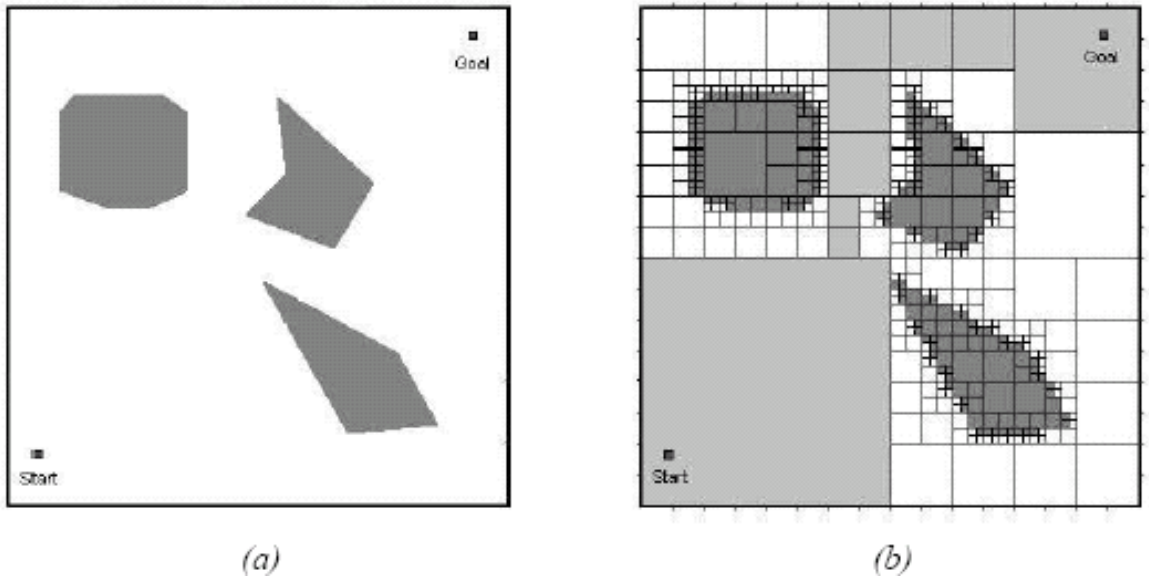
并不是每个单元都是完全被占或者完全空闲的，因此分解后的单元集合是对实际地图的一种近似

2.2 近似单元分解

- 优点
  - 非常简单，与环境的疏密和物体形状的复杂度无关
- 缺点：
  - 对存储空间有要求



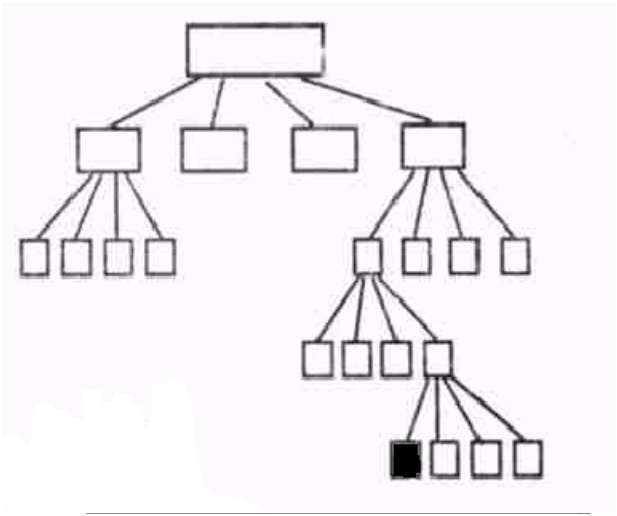
可变大小的近似单元分解



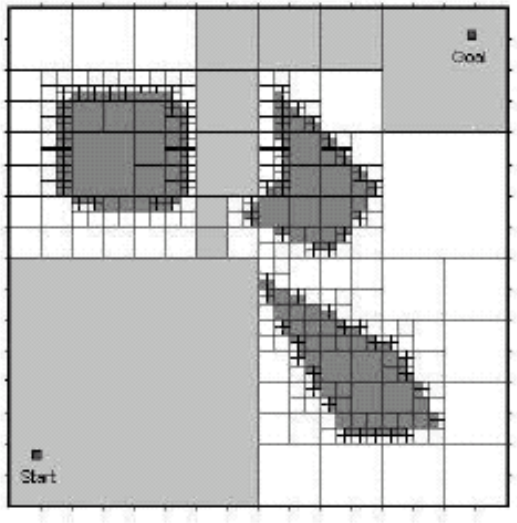
(a) (b)

四叉树表示法：递归地把环境分为4个大小相等的子区域。
直到每个区域中所包含的基本元素全为0或全为1。

可变大小的近似单元分解



(a)

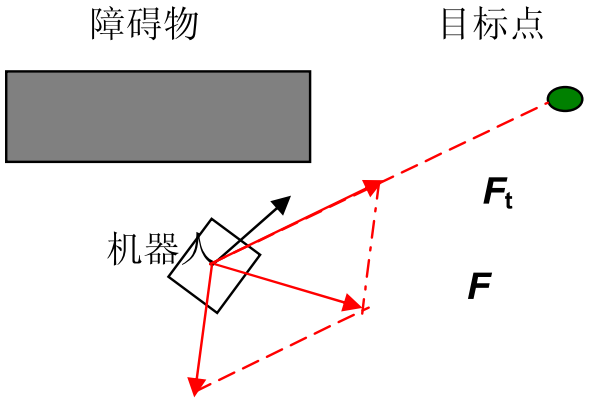


(b)

四叉树表示法：递归地把环境分为4个大小相等的子区域。
直到每个区域中所包含的基本元素全为0或全为1。

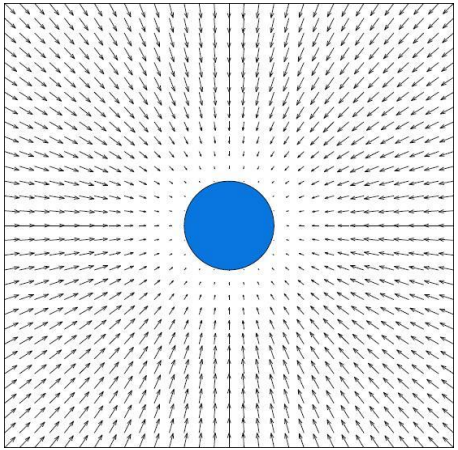
3. 人工势场法

- **基本思想：**
  - 目标点对机器人产生吸引力，障碍物对机器人产生排斥力
  - 所有力的合成构成机器人的控制律



3. 人工势场法

- **步骤1：构建人工势场(Artificial Potential Field)**
  - **目标点：吸引势场**



$$U_{att}(\mathbf{x}) = \begin{cases} K_a \,|\mathbf{x}-\mathbf{x}_d|^2 & |\mathbf{x}-\mathbf{x}_d| \le d_a \\ K_a(2d_a\,|\mathbf{x}-\mathbf{x}_d| - {d_a}^2) & |\mathbf{x}-\mathbf{x}_d| > d_a \end{cases}$$

K_a为系数

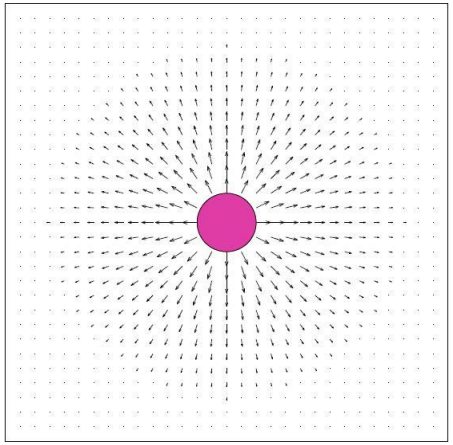
$\mathbf{x}$为被评估点

$\mathbf{x}_d$为目标点

d_a为距离阈值

人工势场法

- 步骤1：构建人工势场(Artificial Potential Field)
  - 目标点：吸引势场
  - 障碍物：推斥势场



$$U_{rep}(\mathbf{x}) = \begin{cases} \frac{1}{2} K_r \left(\frac{1}{\rho} - \frac{1}{\rho_0} \right)^2 & \rho \le \rho_0 \\ 0 & \rho > \rho_0 \end{cases}$$

r 被评估点和障碍物点之间的距离

r_0 预定义距离阈值

3. 人工势场法

- **步骤2：根据人工势场计算力**
  - **对势场求偏导数**

$$F_{att}(\mathbf{x}) = -\nabla U_{att}(\mathbf{x}) = \begin{cases} -2K_a(\mathbf{x}-\mathbf{x}_d) & |\mathbf{x}-\mathbf{x}_d| \le d_a \\ -2K_a d_a \dfrac{\mathbf{x}-\mathbf{x}_d}{|\mathbf{x}-\mathbf{x}_d|} & |\mathbf{x}-\mathbf{x}_d| > d_a \end{cases}$$

$$F_{rep}(\mathbf{x}) = -\nabla U_{rep}(\mathbf{x}) = \begin{cases} K_r\left(\dfrac{1}{\rho}-\dfrac{1}{\rho_0}\right)\dfrac{1}{\rho^2}\dfrac{\partial\rho}{\partial\mathbf{x}} & \rho \le \rho_0 \\ 0 & \rho > \rho_0 \end{cases}$$

$$\frac{\partial\rho}{\partial\mathbf{x}} = \left(\frac{\partial\rho}{\partial x} \quad \frac{\partial\rho}{\partial y}\right)^T = \frac{\mathbf{x}-\mathbf{x}_0}{\rho}$$

$\mathbf{x}_0$为最近障碍物的坐标向量

3. 人工势场法

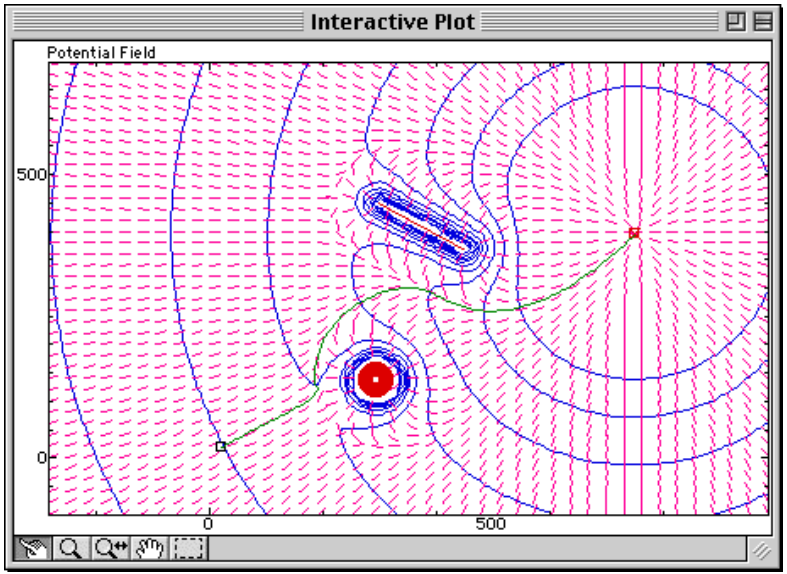
- **步骤3：计算合力，并进而由力计算得到控制律**

$$
\begin{aligned}
F(\mathbf{x}) &= -\nabla U(\mathbf{x}) \\
&= -\nabla U_{att}(\mathbf{x}) - \nabla U_{rep}(\mathbf{x}) \\
&= F_{att}(\mathbf{x}) + F_{rep}(\mathbf{x})
\end{aligned}
$$

力的方向就是机器人运动方向，大小可以对应加速度控制

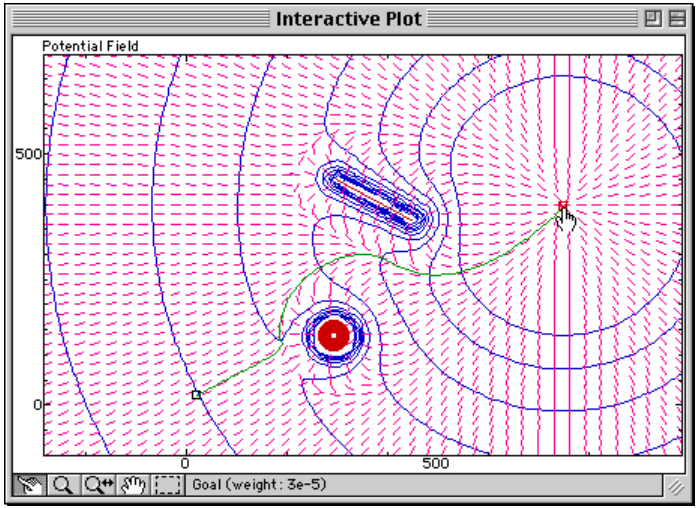
3. 人工势场法

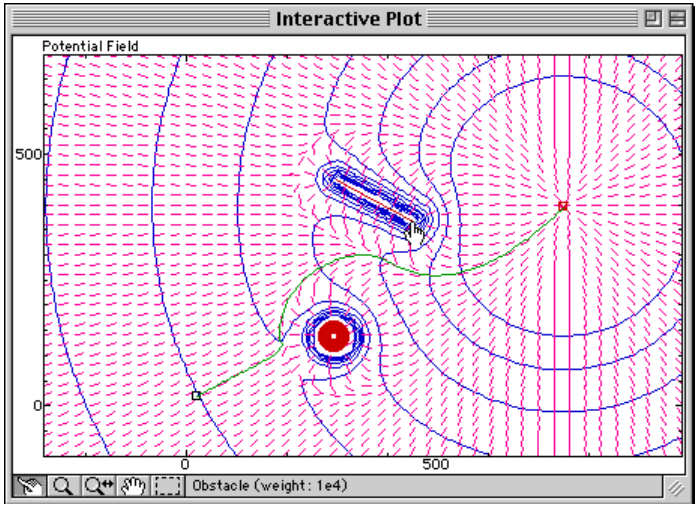
- 机器人是受人工势场影响的一个点，沿着势场方向就可以避开障碍物达到目标点



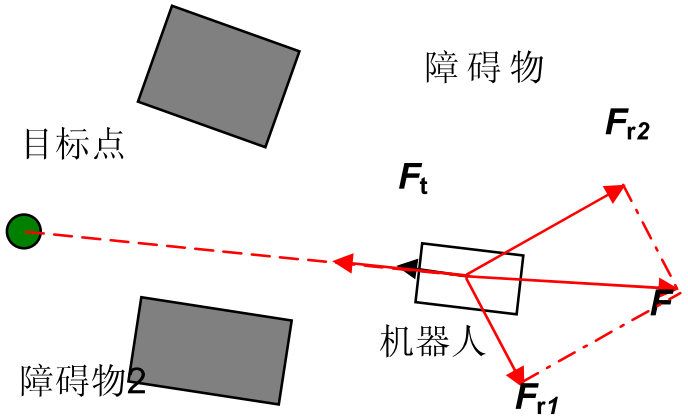
3. 人工势场法

- 不仅是一种路径规划方法，所构建的势场也构成了机器人的控制律，能够较好地适应目标的变化和环境中的动态障碍物，可以作为实时避障算法





3. 人工势场法

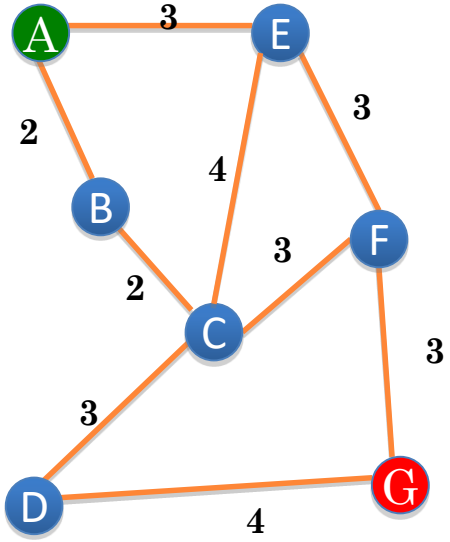


- 缺点：存在局部最小，容易产生振荡和死锁

2.3 路径搜索算法

最优路径搜索算法

- 在构建形成的连通图中搜索最优路径



最优路径搜索算法

- 精确算法：生成精确的最优解
  - 深度优先法、广度优先法
  - 遍历获得所有路径后选择最优解
  - 优化：优先级定义的广度优先法、Dijstra算法
  - 存在问题：耗时，难以满足机器人在线快速规划要求
- 近似算法
  - 启发式搜索算法：A*, D*, Focused D*等
  - 准启发式搜索算法：退火、进化和蚁群优化等

广度优先法与深度优先法比较

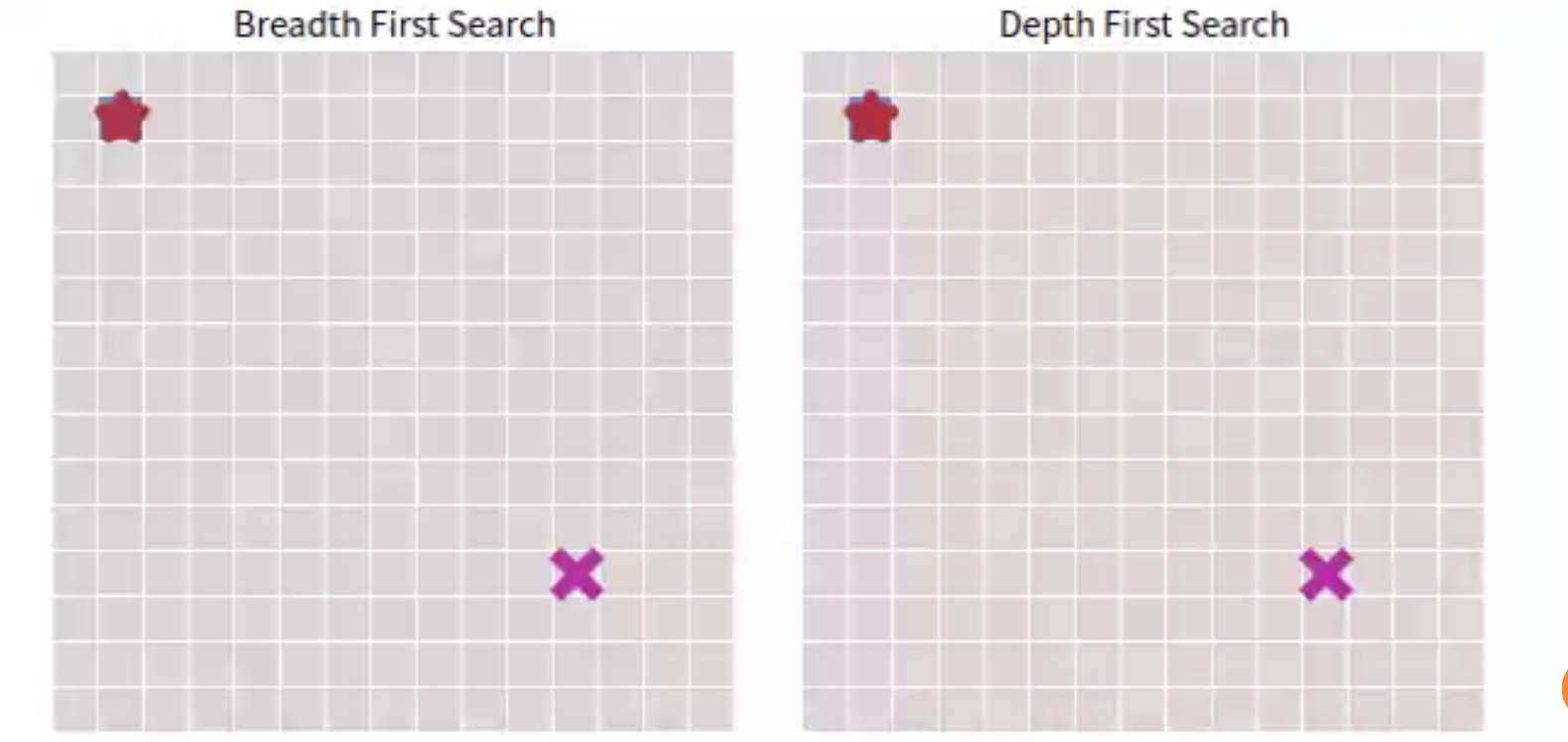
广度优先法

Courtesy: Amit Patel's Introduction to A*, Stanford

深度优先法

Courtesy: Amit Patel's Introduction to A*, Stanford

广度优先法与深度优先法比较

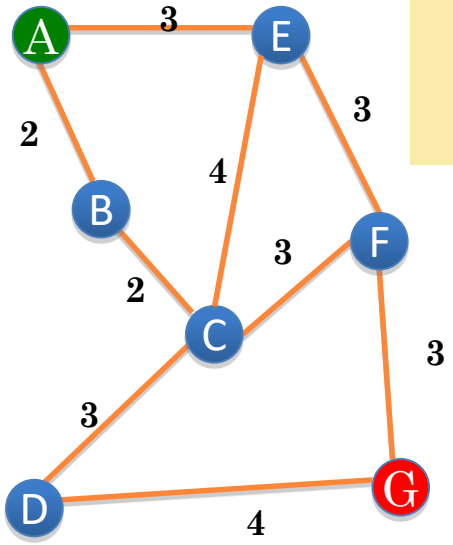


DIJKSTRA算法

- 采用优先级定义的广度优先搜索思想
- 在有向图中以起始点为中心按路径长度递增方式层层向外扩展，直到扩展到终止点

  - 设置两个集合S和T， S存放已经找到最短路径的顶点，T是尚未确定路径的顶点集合，同时也描述了起始点经过集合S中顶点到该点的最短路径及长度。
  - 每次更新时，从T中找出路径最短的点加入到集合S，T中顶点最短路径及长度则根据加入点作为中间点后起始点到该点距离是否减小来决定是否更新

DIJKSTRA算法



- 设置两个集合S和T， S存放已经找到最短路径的顶点，T是尚未确定路径的顶点集合，同时也描述了起始点经过集合S中顶点到该点的最短路径及长度。
- 每次更新时，从T中找出路径最短的点加入到集合S，T中顶点最短路径及长度则根据加入点作为中间点后起始点到该点距离是否减小来决定是否更新

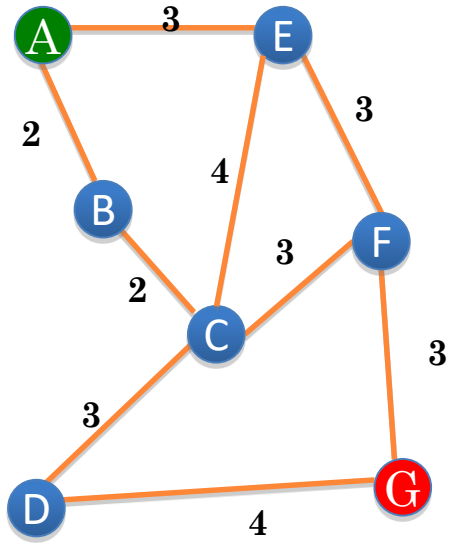
S={A→A=0}

T={A→B=2, A→C=∞, A→D=∞, A→E=3, A→F=∞, A→G=∞}

S={A→A=0, A→B=2}

T={A→C=4(路径为ABC), A→D=∞, A→E=3, A→F=∞, A→G=∞}

DIJKSTRA算法



问题规模大时搜索耗时

S={A→A=0, A→B=2, A→E=3}

T={A→C=4(路径为ABC), A→D=∞, A→F=6(路径为AEF), A→G=∞ }

S={A→A=0, A→B=2, A→E=3
A→C=4(路径为ABC)}

T={A→D=7(路径为ABCD), A→F=6(路径为AEF), A→G=∞}

……

启发式搜索算法：A*

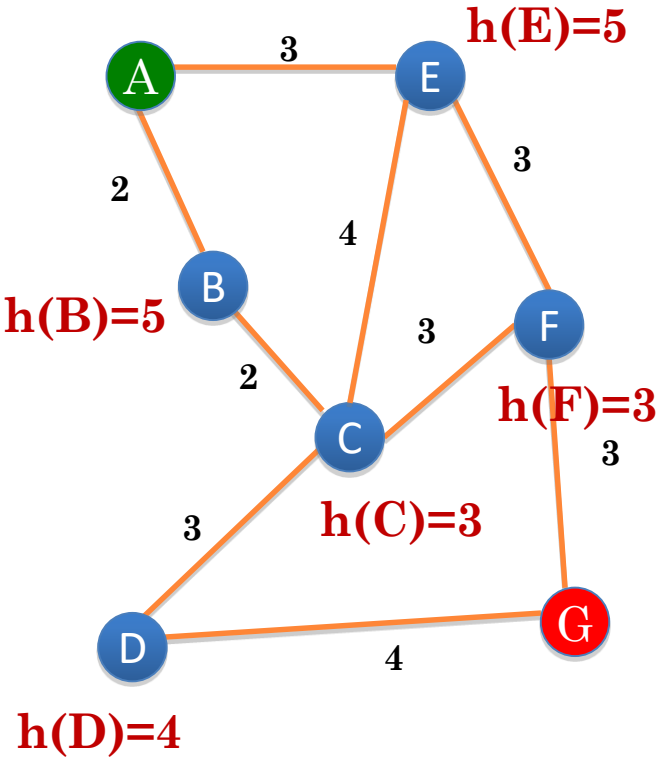
- 基于优先级定义的广度优先搜索
- 根据启发式评估函数在连通图中寻找最优路径
  - 当选择下一个探索结点时，通过启发式评估函数进行评估，选择路径代价最小的结点作为下一步探索结点而跳转其上

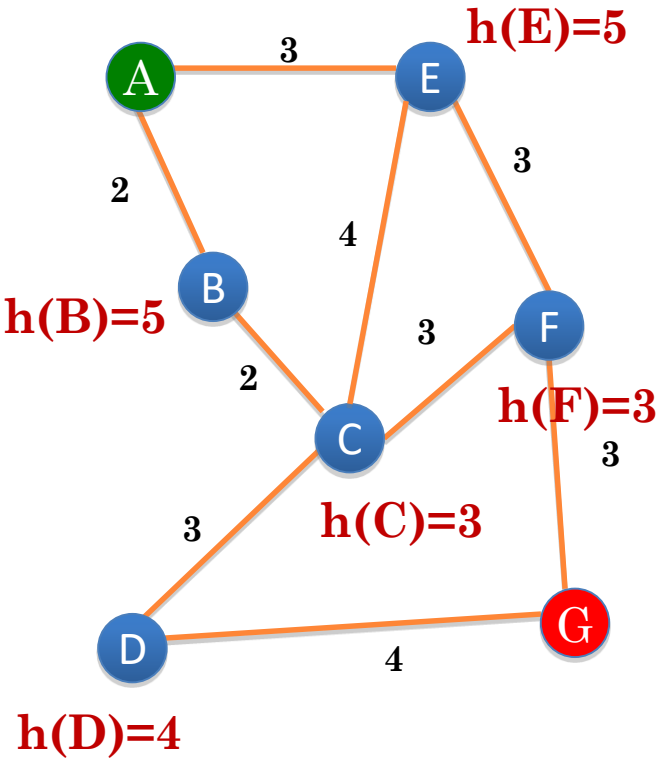
评估函数 $f(n) = g(n) + h(n)$

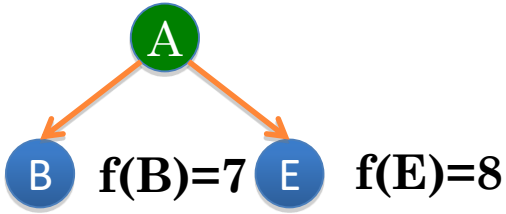
n 表示节点

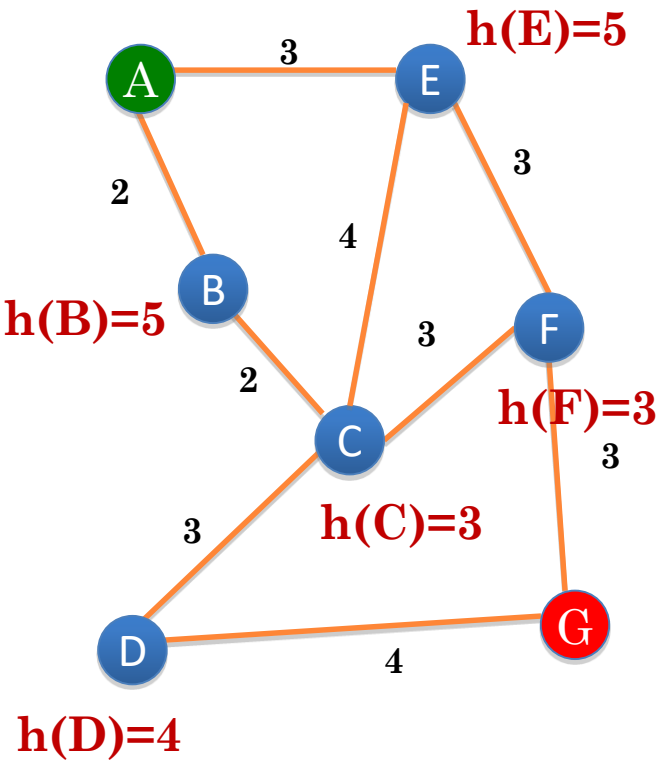
g(n) 表示从起始点到节点 的实际代价

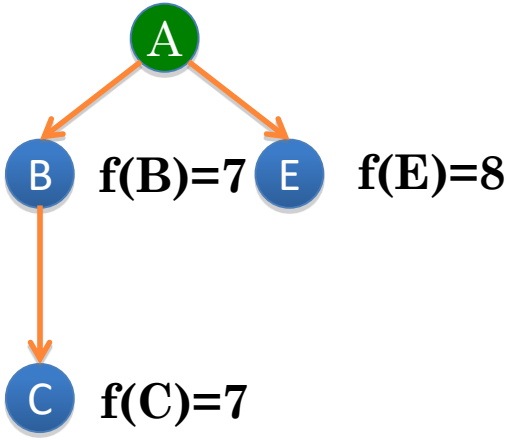
h(n) 为从节点 到目标点的最佳路径的估计代价

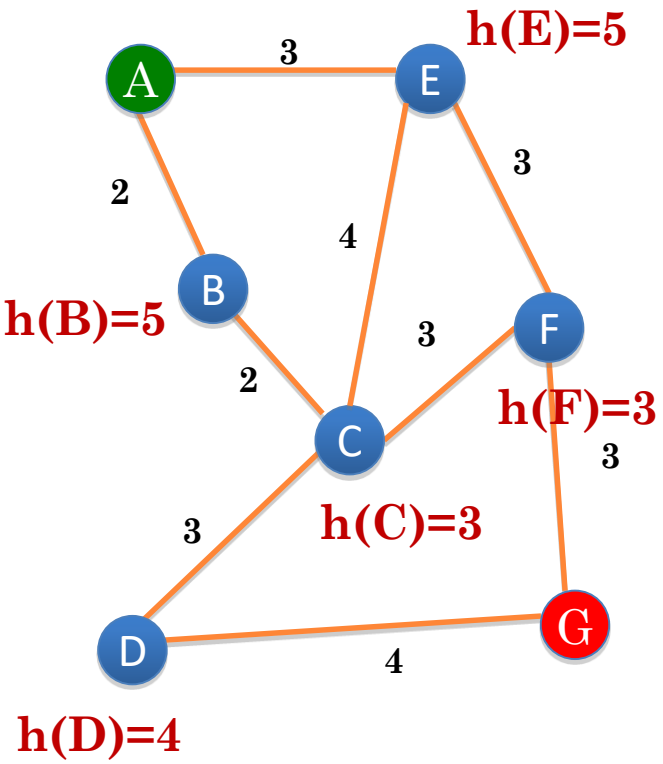

h(E)=5
A
3
E
3
2
4
B
h(B)=5
3
F
2
h(F)=3
C
3
h(C)=3
3
D
4
G
h(D)=4

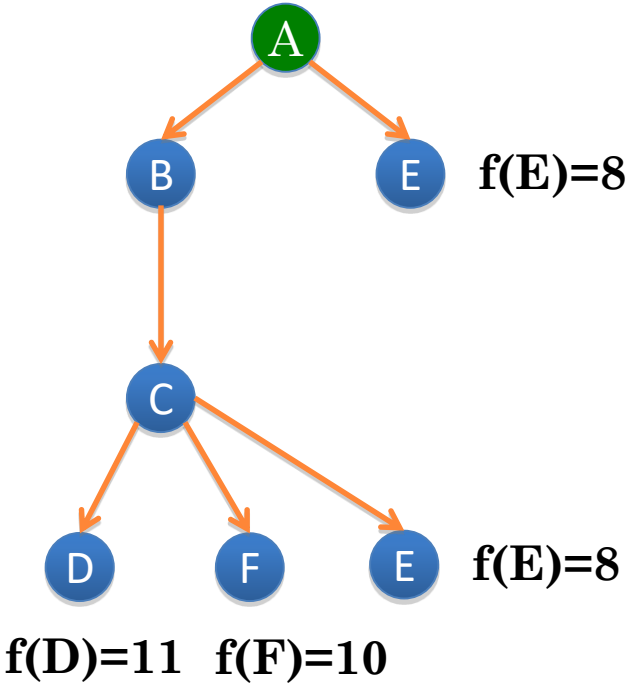

h(E)=5
A
3
E
2
3
4
B
h(B)=5
3
F
2
h(F)=3
C
3
3
h(C)=3
D
4
G
h(D)=4

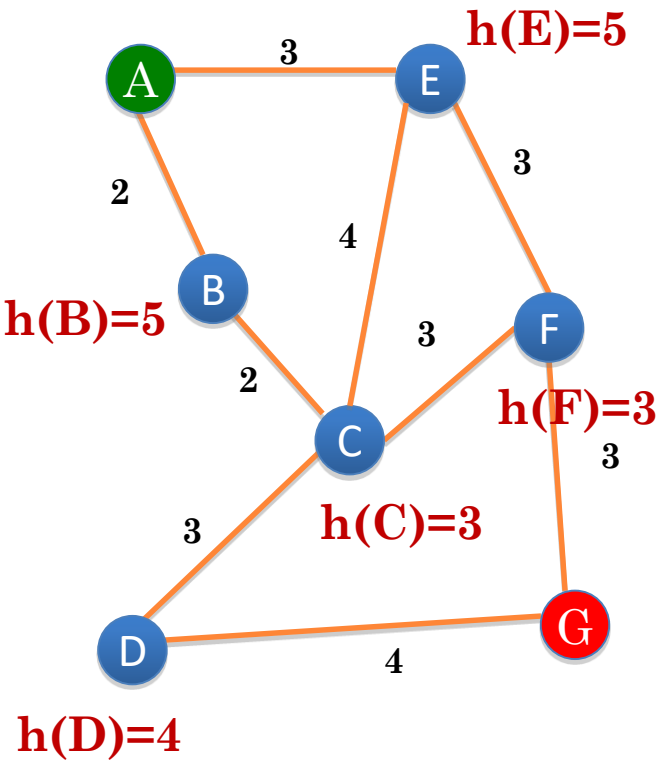

A
B
f(B)=7
E
f(E)=8

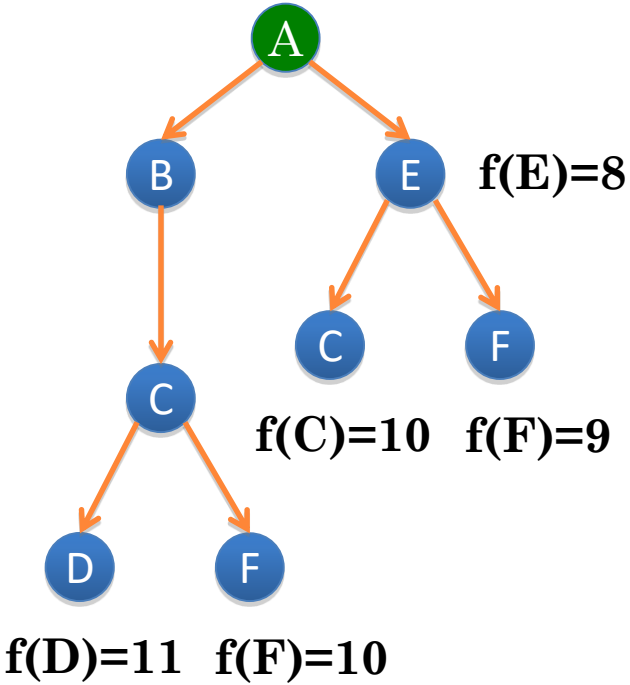

A
E
B
C
F
D
G
3
2
4
3
3
2
3
3
3
4
h(E)=5
h(B)=5
h(F)=3
h(C)=3
h(D)=4

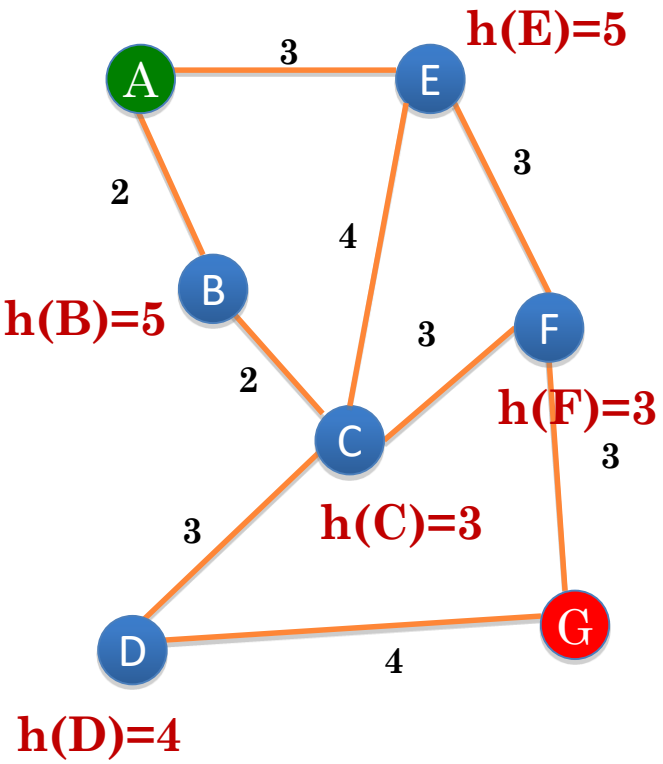

A
B
E
C
f(B)=7
f(E)=8
f(C)=7

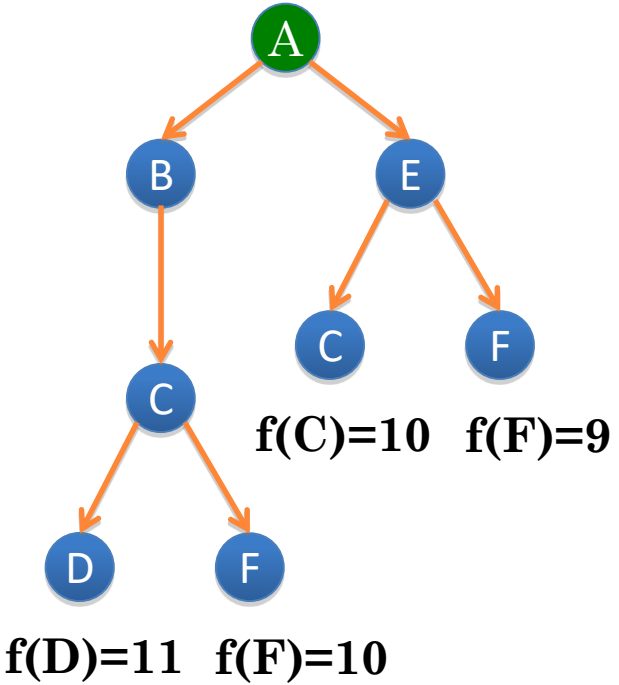

h(E)=5
A
3
E
2
3
4
B
h(B)=5
3
F
2
h(F)=3
C
3
3
h(C)=3
D
4
G
h(D)=4

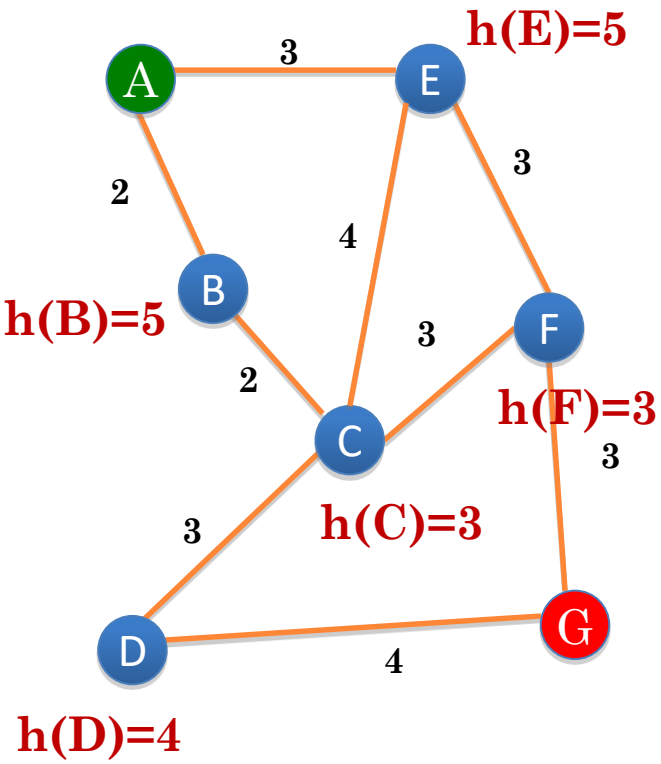

A
B
E
f(E)=8
C
D
F
E
f(E)=8
f(D)=11
f(F)=10

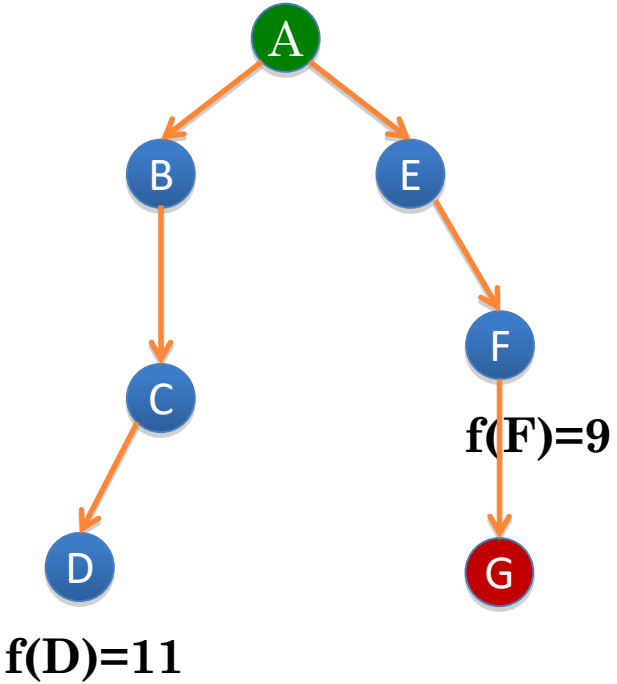

h(E)=5
A
3
E
2
3
4
B
h(B)=5
3
F
2
h(F)=3
C
3
h(C)=3
3
D
4
G
h(D)=4

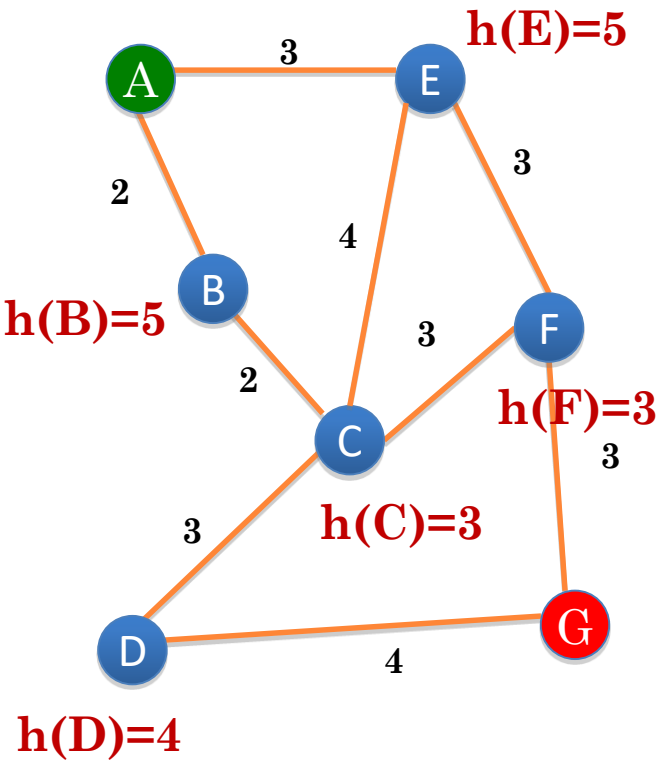

A
B
E
f(E)=8
C
F
f(C)=10
f(F)=9
D
F
f(D)=11
f(F)=10

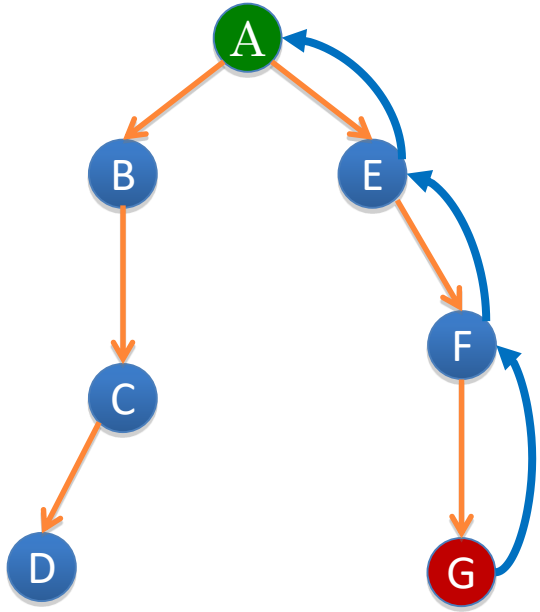

A
E
B
C
F
D
G
3
2
4
3
3
2
3
3
3
4
h(E)=5
h(B)=5
h(F)=3
h(C)=3
h(D)=4

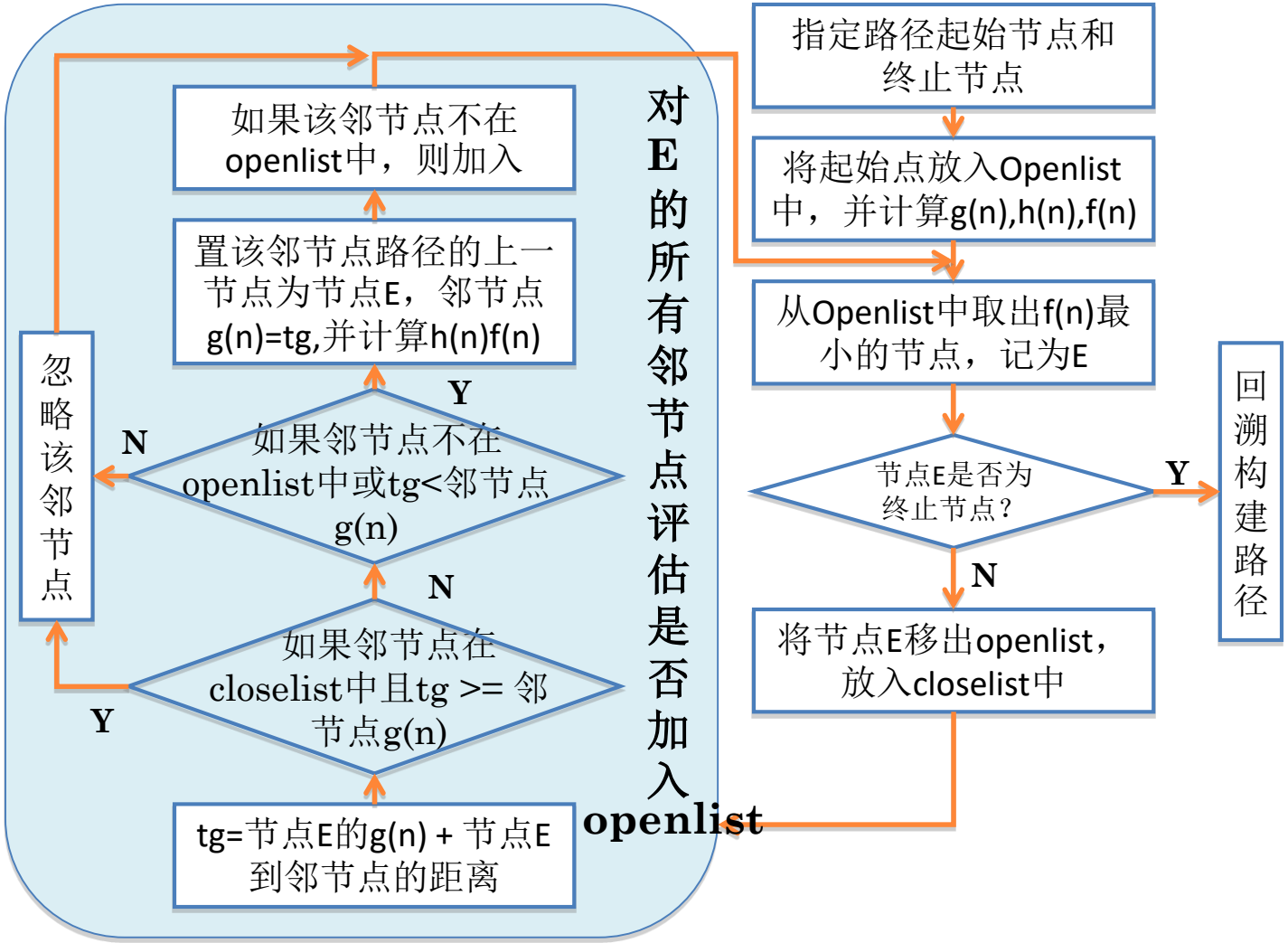

A
B
E
C
F
C
F
D
F
f(C)=10
f(F)=9
f(D)=11
f(F)=10

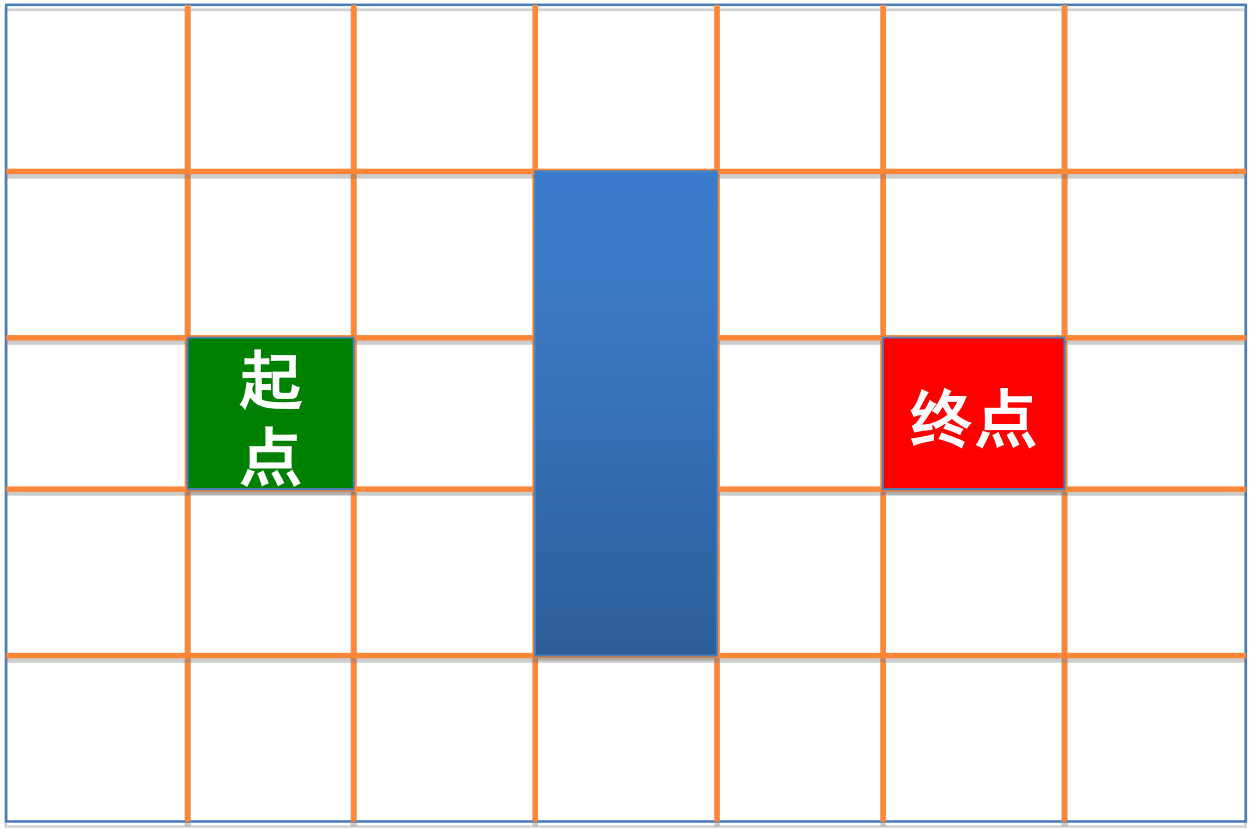

A
E
B
F
C
D
G
3
2
4
3
3
2
3
3
3
4
h(E)=5
h(B)=5
h(F)=3
h(C)=3
h(D)=4

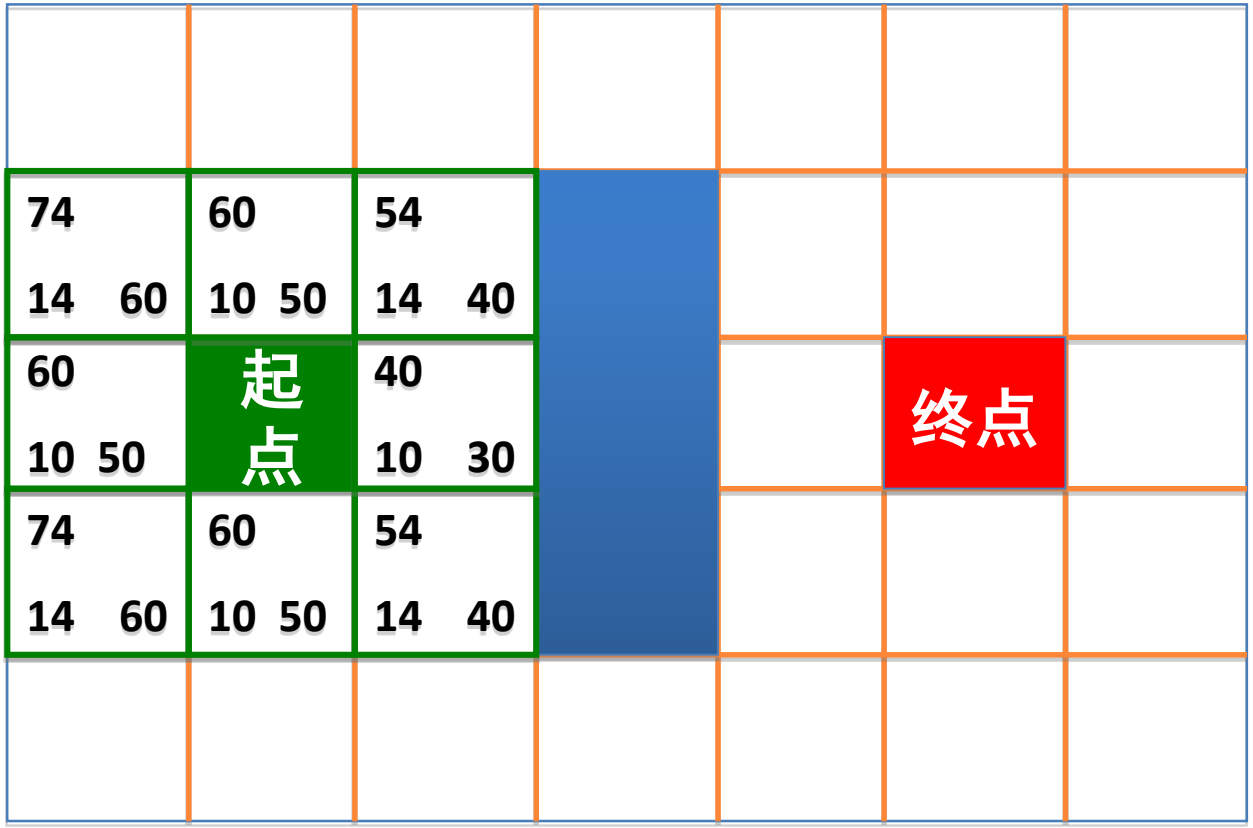

A
B
E
C
F
D
G
f(F)=9
f(D)=11

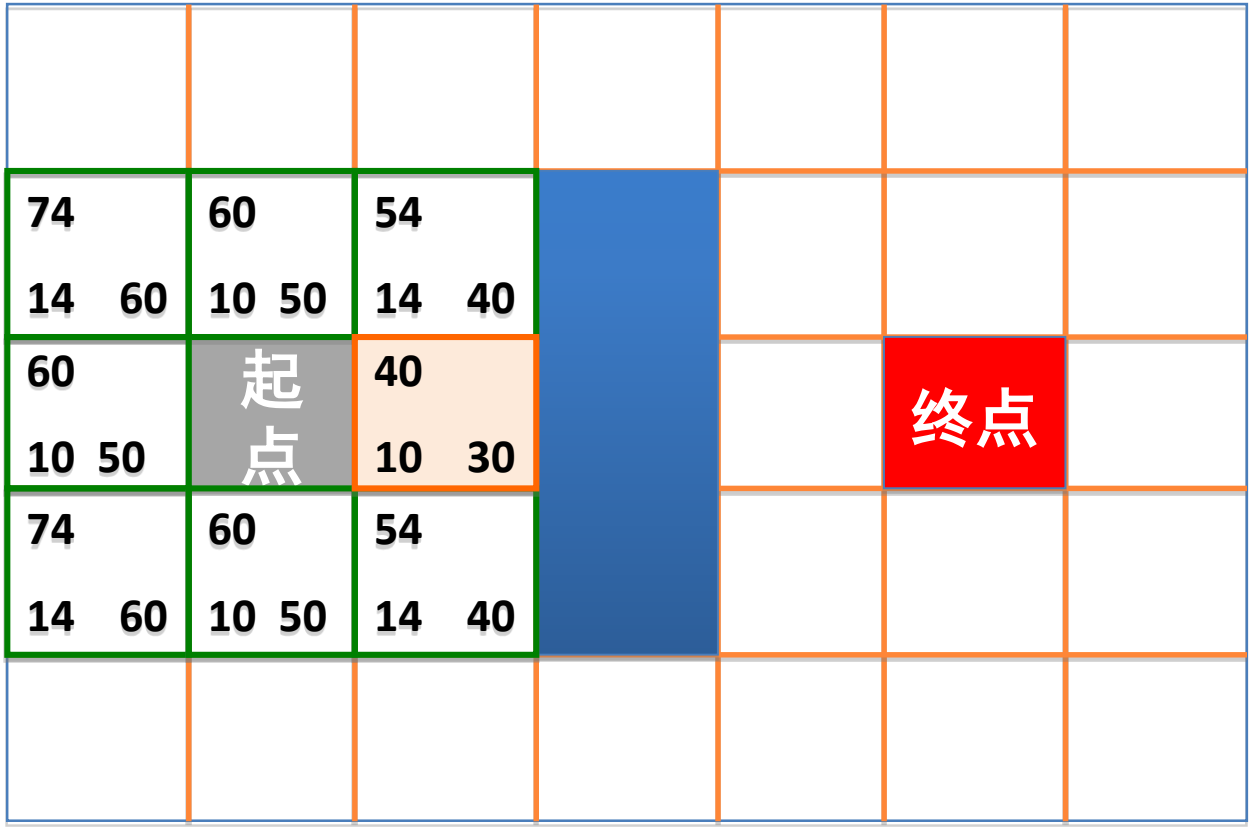

A
E
B
F
C
D
G
3
2
4
3
3
2
3
3
3
4
h(E)=5
h(B)=5
h(F)=3
h(C)=3
h(D)=4

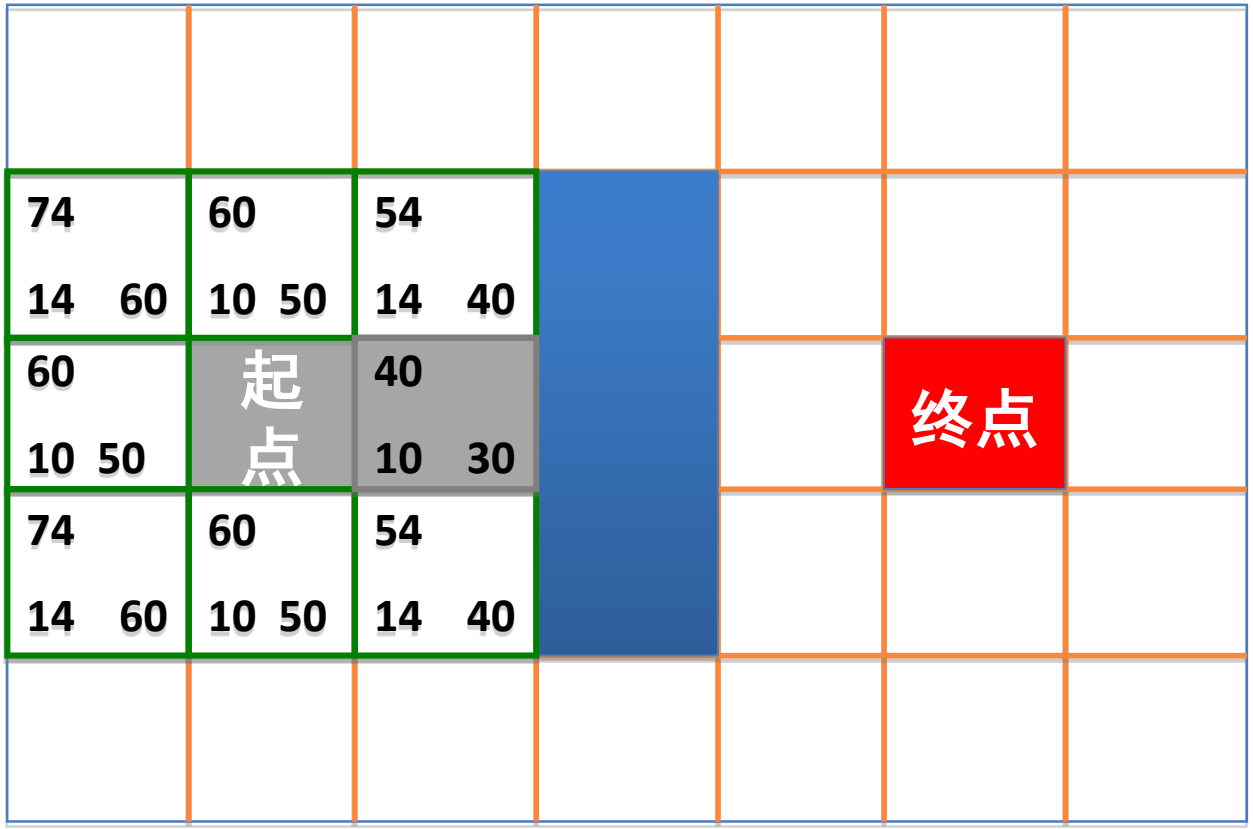

A
B
E
C
F
D
G
f(D)=11


指定路径起始节点和终止节点
将起始点放入Openlist中，并计算g(n),h(n),f(n)
从Openlist中取出f(n)最小的节点，记为E
节点E是否为终止节点？
Y
回溯构建路径
N
将节点E移出openlist，放入closelist中
对E的所有邻节点评估是否加入openlist
如果该邻节点不在openlist中，则加入
置该邻节点路径的上一节点为节点E，邻节点g(n)=tg,并计算h(n)f(n)
Y
如果邻节点不在openlist中或tg<邻节点g(n)
N
忽略该邻节点
N
如果邻节点在closelist中且tg >= 邻节点g(n)
Y
tg=节点E的g(n) + 节点E到邻节点的距离

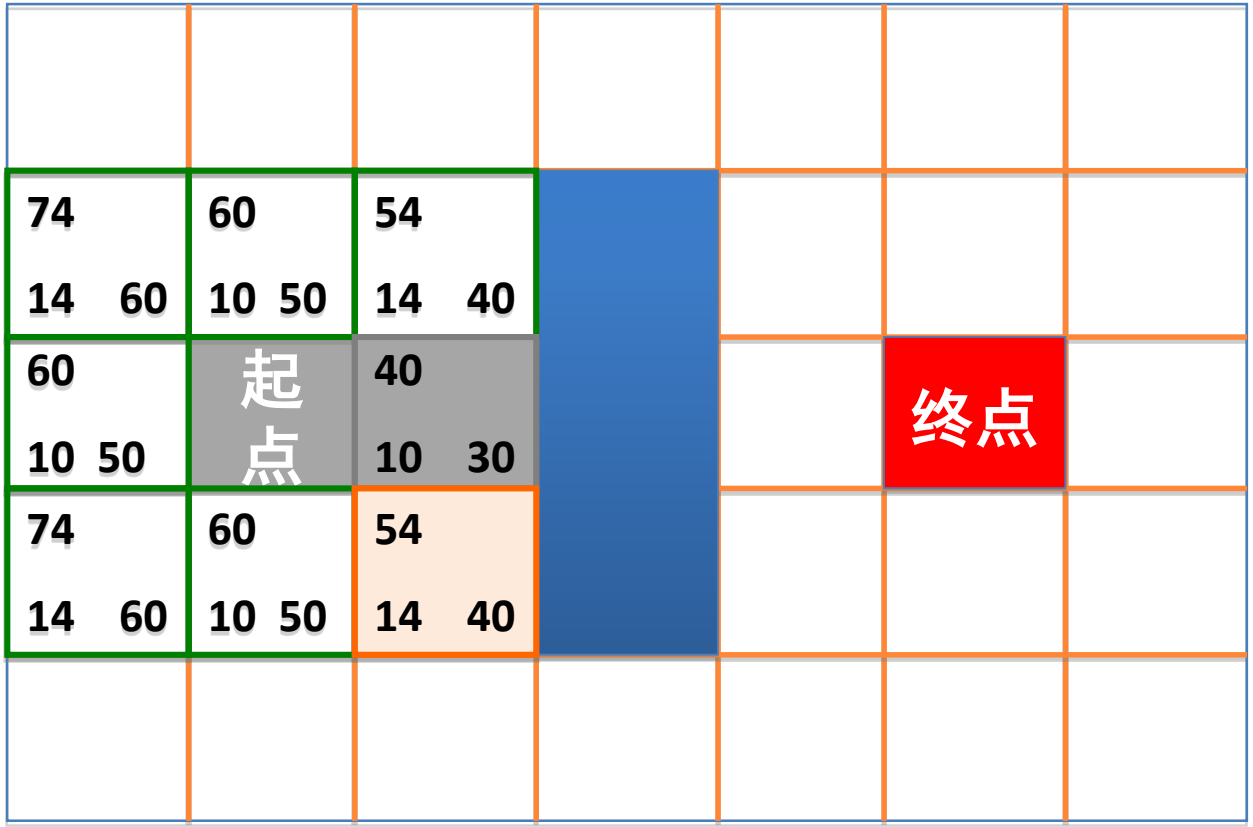

起点
终点

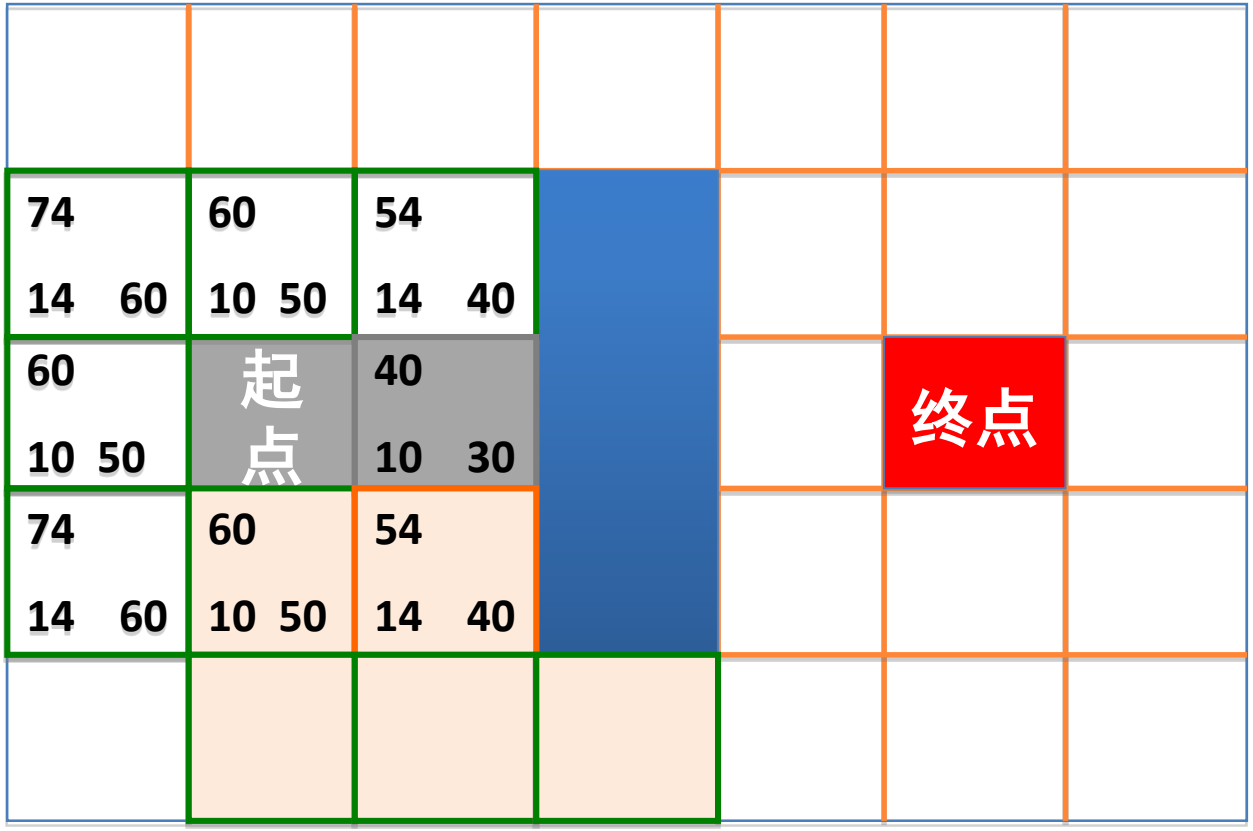

74
14 60
60
10 50
54
14 40
60
10 50
起点
40
10 30
74
14 60
60
10 50
54
14 40
终点

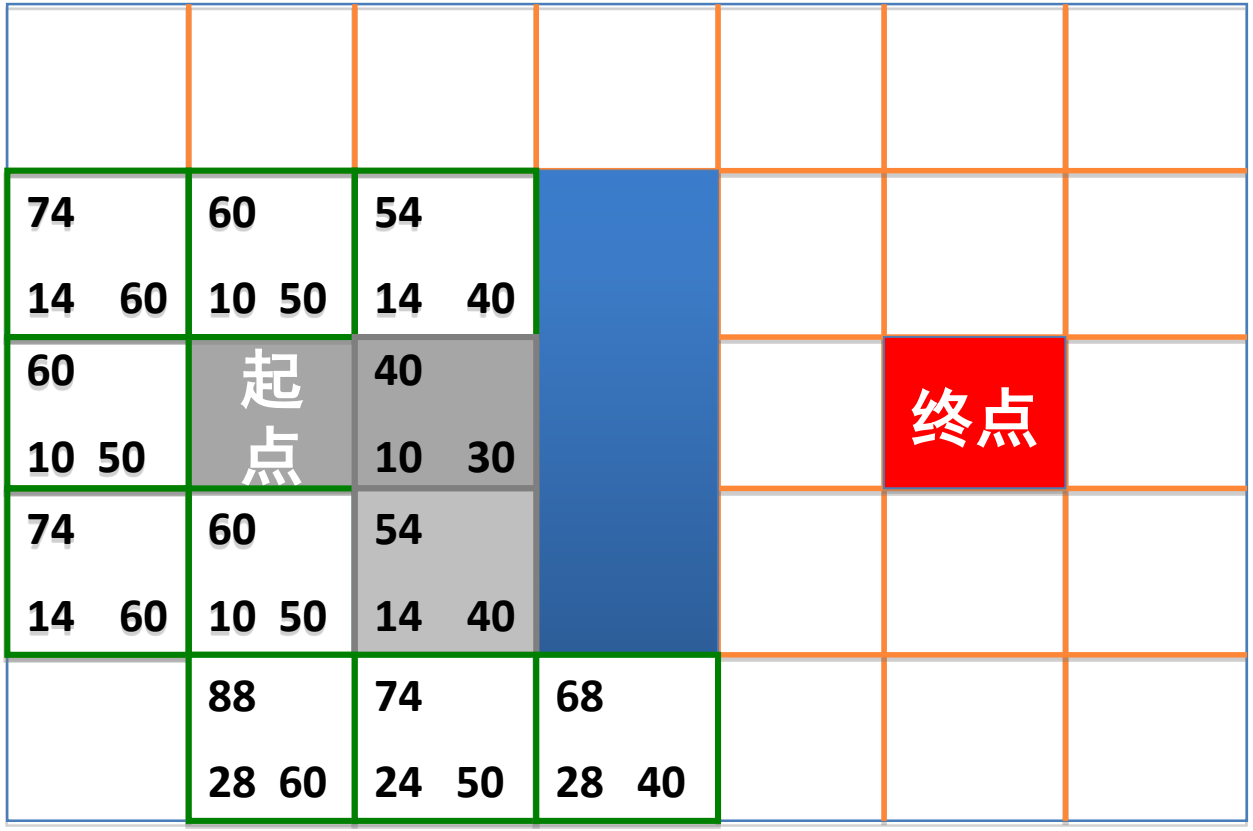

74
14 60
60
10 50
54
14 40
60
10 50
起点
40
10 30
终点
74
14 60
60
10 50
54
14 40

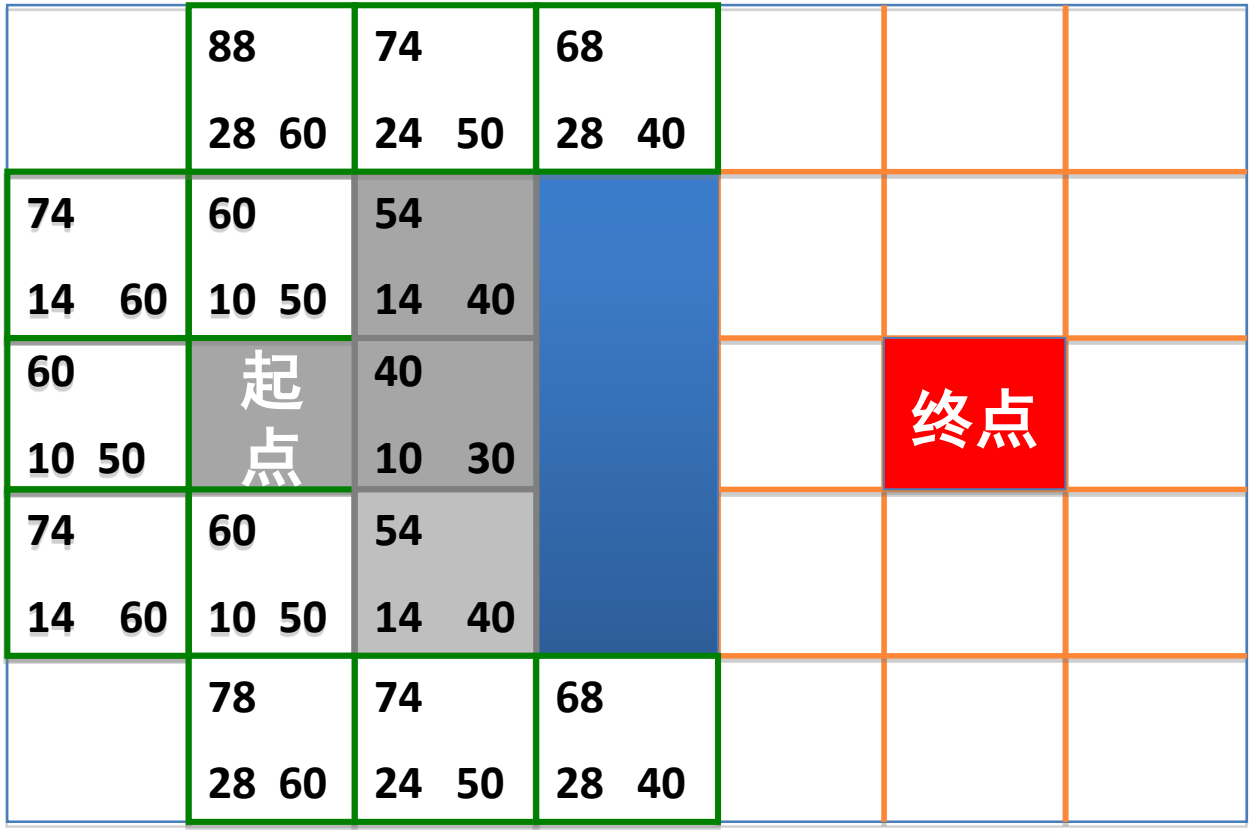


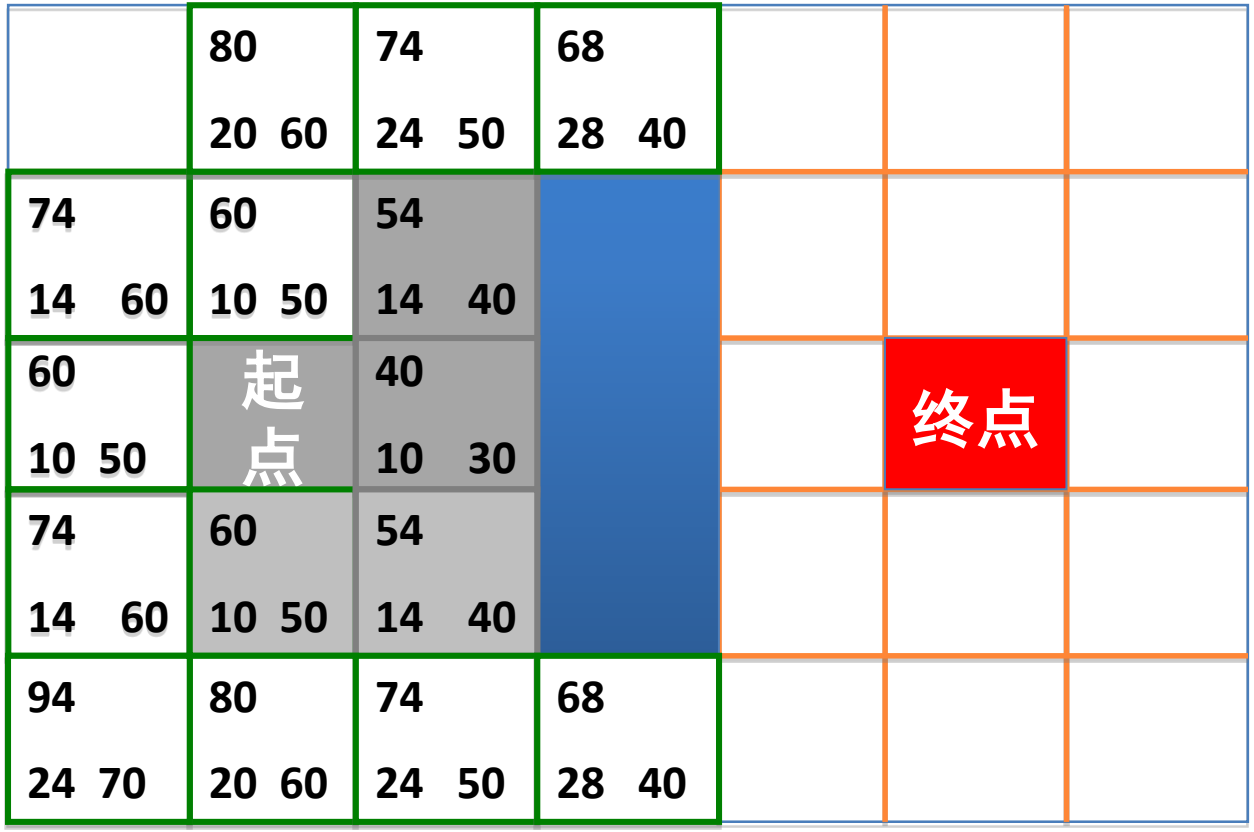
该节点被放入closelist，各邻节点已在openlist中，
tg = 10+ 10 或14 ，大于各邻节点g(n)，无操作

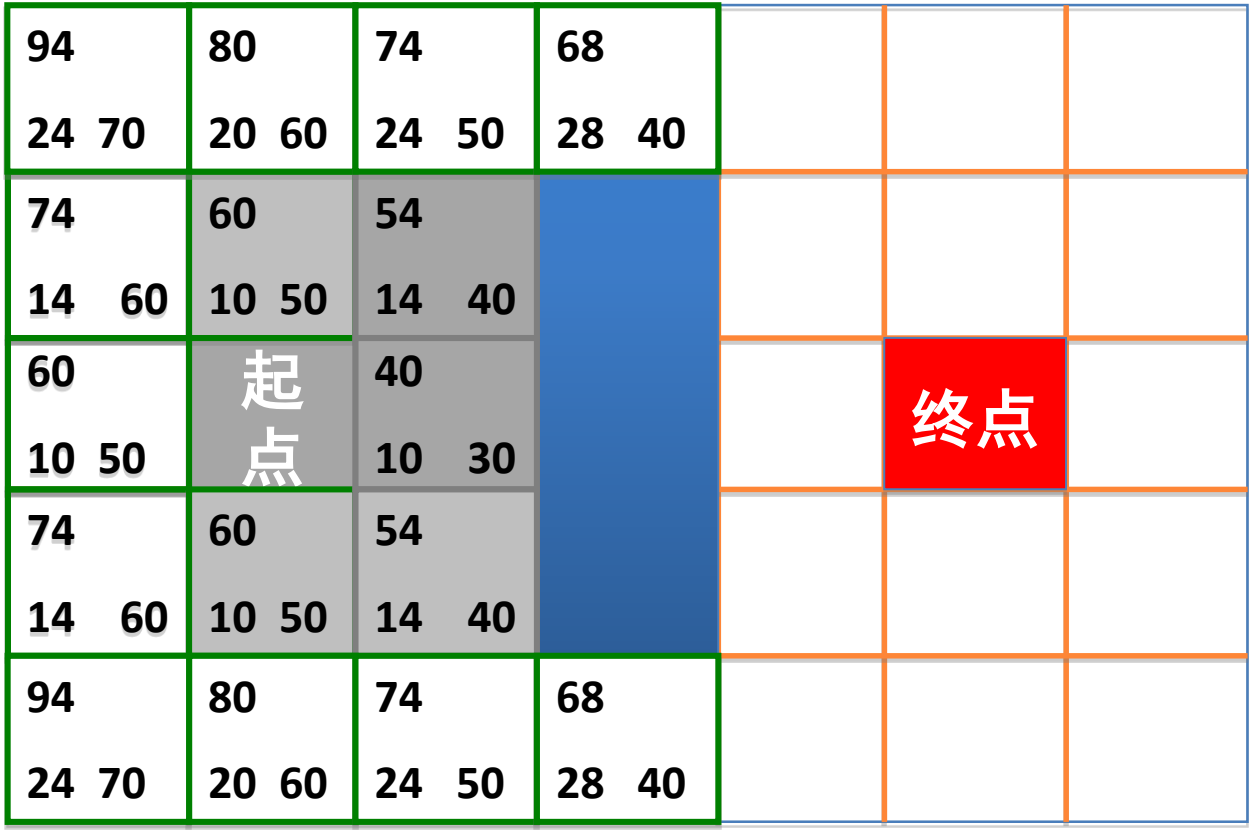

74
14 60
60
10 50
54
14 40
60
10 50
起点
40
10 30
终点
74
14 60
60
10 50
54
14 40

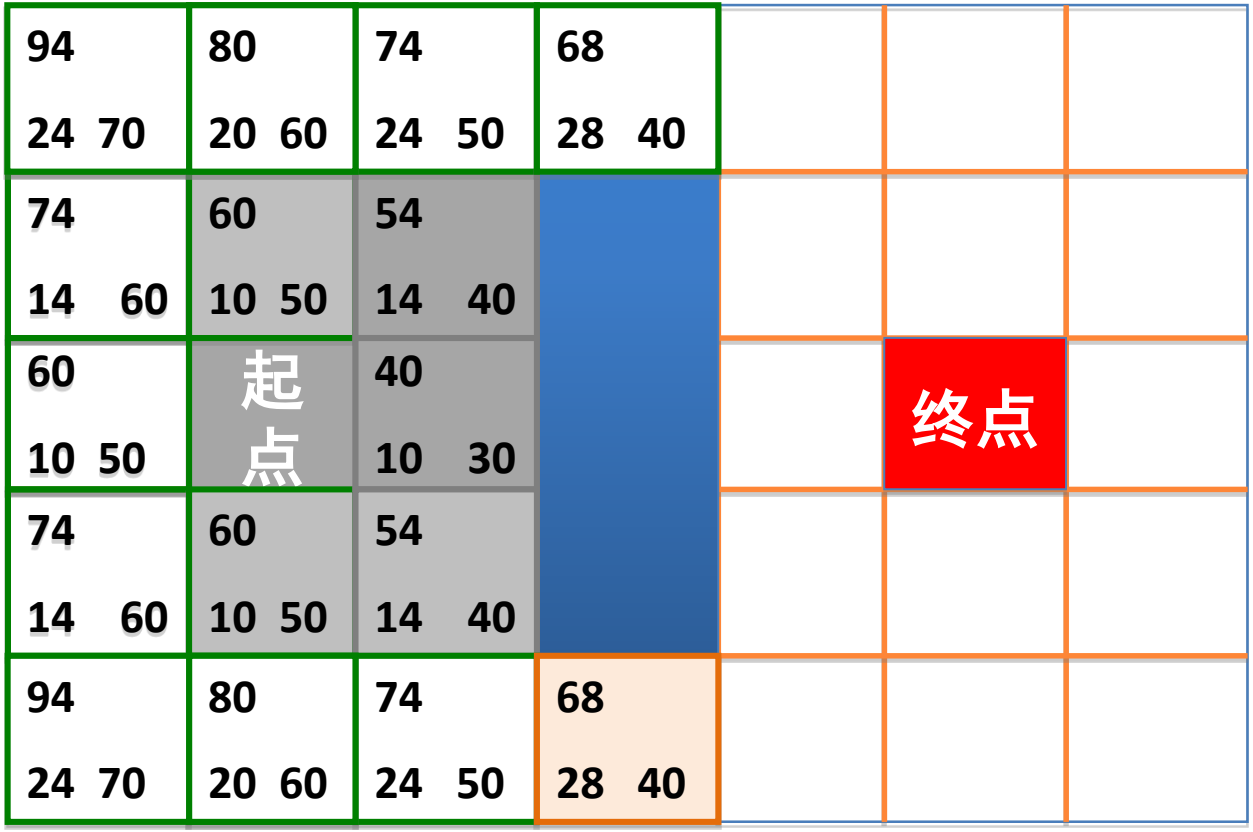

74
14 60
60
10 50
54
14 40
60
10 50
起点
40
10 30
终点
74
14 60
60
10 50
54
14 40

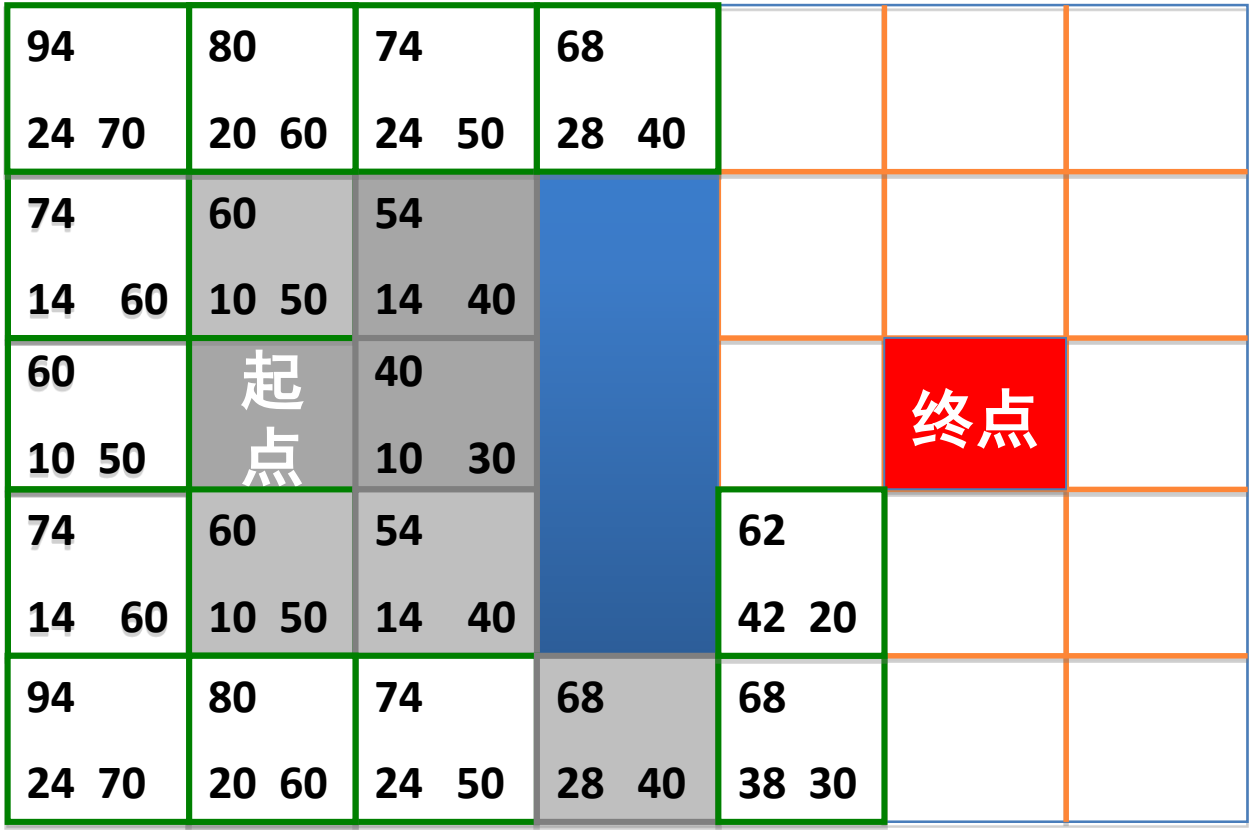

74
14 60
60
10 50
54
14 40
60
10 50
起点
40
10 30
终点
74
14 60
60
10 50
54
14 40
88
28 60
74
24 50
68
28 40


88
28 60
74
24 50
68
28 40
74
14 60
60
10 50
54
14 40
60
10 50
起点
40
10 30
终点
74
14 60
60
10 50
54
14 40
78
28 60
74
24 50
68
28 40

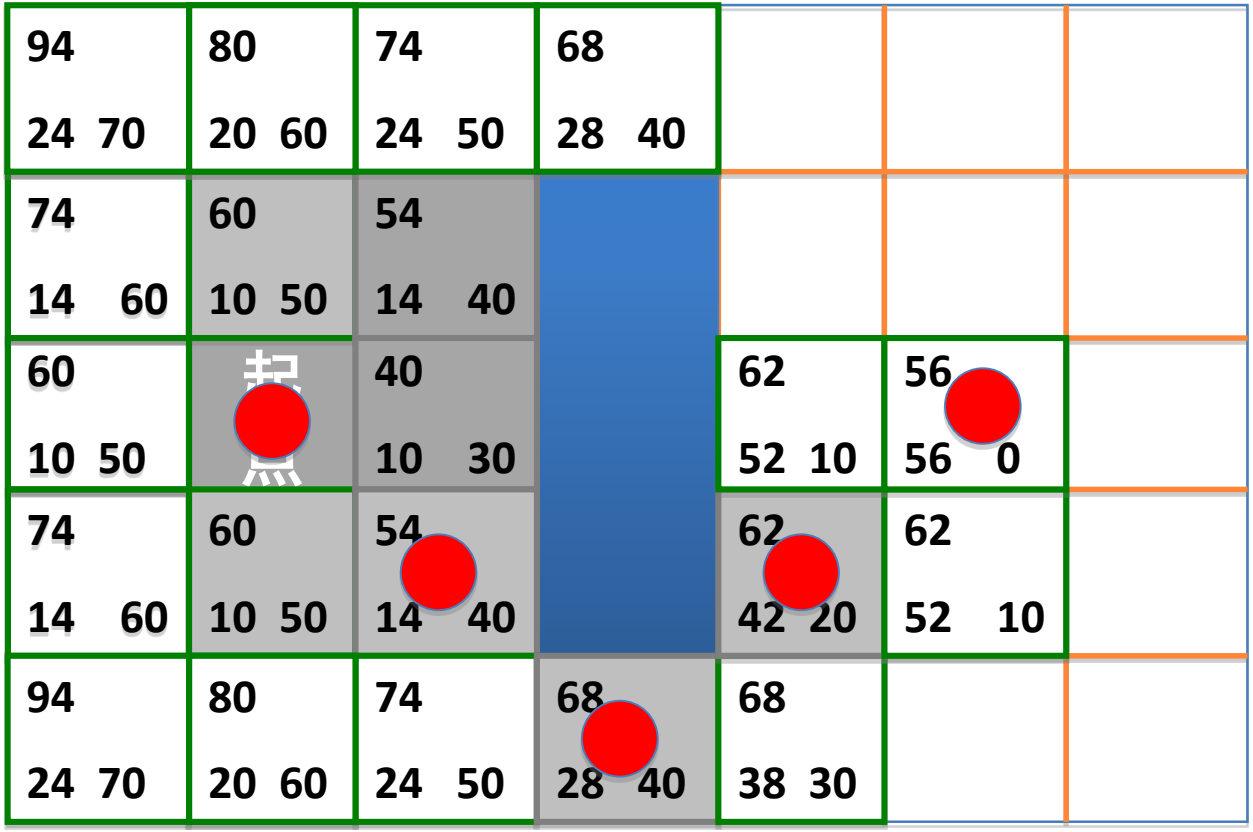

80
20 60
74
24 50
68
28 40
74
14 60
60
10 50
54
14 40
60
10 50
起点
40
10 30
终点
74
14 60
60
10 50
54
14 40
94
24 70
80
20 60
74
24 50
68
28 40


94
24 70
80
20 60
74
24 50
68
28 40
74
14 60
60
10 50
54
14 40
60
10 50
起点
40
10 30
终点
74
14 60
60
10 50
54
14 40
94
24 70
80
20 60
74
24 50
68
28 40


94
24 70
80
20 60
74
24 50
68
28 40
74
14 60
60
10 50
54
14 40
60
10 50
起点
40
10 30
终点
74
14 60
60
10 50
54
14 40
94
24 70
80
20 60
74
24 50
68
28 40


94
24 70
80
20 60
74
24 50
68
28 40
74
14 60
60
10 50
54
14 40
60
10 50
起点
40
10 30
终点
74
14 60
60
10 50
54
14 40
62
42 20
94
24 70
80
20 60
74
24 50
68
28 40
68
38 30

94 24 70	80 20 60	74 24 50	68 28 40			
74 14 60	60 10 50	54 14 40				
60 10 50	起点	40 10 30		62 52 10	56 56 0	
74 14 60	60 10 50	54 14 40		62 42 20	62 52 10	
94 24 70	80 20 60	74 24 50	68 28 40	68 38 30		


94
24 70
80
20 60
74
24 50
68
28 40
74
14 60
60
10 50
54
14 40
60
10 50
起
40
10 30
62
52 10
56
56 0
74
14 60
60
10 50
54
14 40
62
42 20
62
52 10
94
24 70
80
20 60
74
24 50
68
28 40
68
38 30

路径成本的计算

- 欧式距离（2-norm距离）
- 曼哈顿距离（Manhattan distance, Block distance, 1-norm距离，L1 distance）

欧式距离

- 2-norm距离

$$\|\mathbf{x}-\mathbf{y}\|_2=\sqrt{\sum_i (x_i-y_i)^2}$$

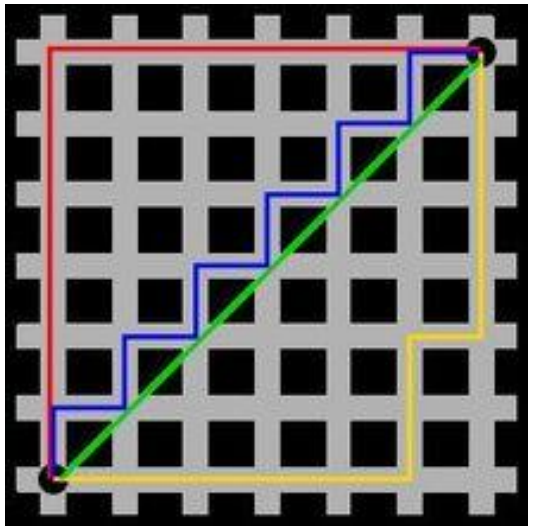
$$\mathbf{x}=(x_1,x_2,\square\ \ ,x_n)$$

$$\mathbf{y}=(y_1,y_2,\square\ \ ,y_n)$$

曼哈顿距离

- **Manhattan distance**

$$\|\mathbf{x}-\mathbf{y}\|_1 = \sum_i |x_i - y_i|$$

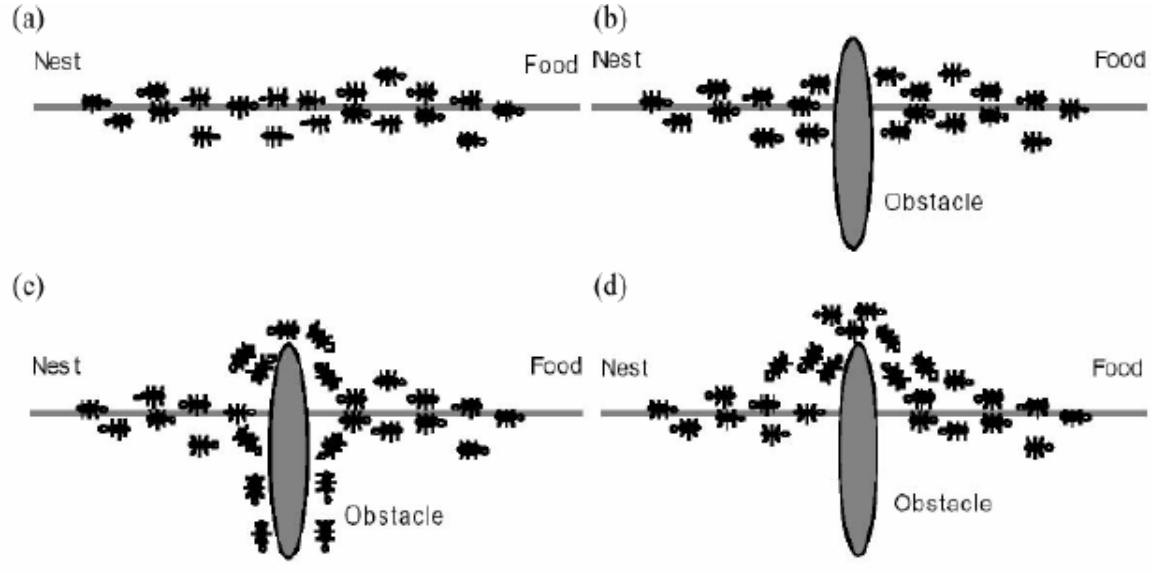


用以标明两个点上在标准坐标系上的绝对轴距总和

绿线为欧氏距离
红线为曼哈顿距离
黄蓝线为等价曼哈顿距离

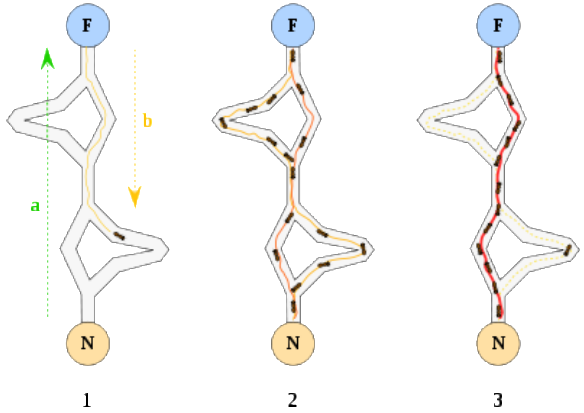
准启发式搜索算法： 蚁群算法

- 1992年由意大利科学家M.Dorigo提出
- 来源于模拟蚂蚁觅食过程中寻找路径的行为



蚁群算法

- 基本思想：
  - 用一个蚂蚁的行走路径表示待优化问题的一个可行解
  - 用整个蚂蚁群体的所有路径作为待优化问题的解空间
  - 用蚂蚁群体收敛选择的路径作为问题的优化解



蚁群算法

- 实现方式：
  - 每个蚂蚁在经过的路径上释放信息素，称为正反馈

$$\tau_{ij}(t) \leftarrow \tau_{ij}(t) + \Delta\tau$$

    - $\Delta\tau$ 可以是经过该路段留下的即时信息素，也可以是到达目标点后对整段路径留下的信息素
    - 信息素与路径长度成反比
    - 为避免早熟收敛，信息素采用挥发机制

$$\tau \leftarrow (1-\rho)\tau, \rho \in [0，\ 1]$$

蚁群算法

- 实现方式：
  - 信息素计算总表达式

$$\tau_{ij}(t+n) = (1-\rho)^n \tau_{ij}(t) + \Delta\tau_{ij}(t+n)$$

$$\Delta\tau_{ij}(t+n) = \sum_{k=1}^{m} \Delta\tau_{ij}^{k}(t+n)$$

m 为蚂蚁总数

k 为蚂蚁编号

蚁群算法

- 实现方式：
  - 每个蚂蚁根据后续可选边的评估概率地决策

$$p_{ij}^{k}(t)=\begin{cases}\dfrac{a_{ij}(t)}{\sum_{l\in N_i^k}a_{il}(t)} & if\ j\in N_i^k\\ 0 & if\ not\end{cases}$$

k表示蚂蚁编号，N_i^k为第k个蚂蚁当点节点i的相邻节点集合，$a_{ij}(t)$ 为节点i的路径评估值

蚁群算法

$$a_{ij} = \frac{\tau_{ij}^{\alpha}(t)\eta_{ij}^{\beta}(t)}{\sum_{l \in N_i^k} \tau_{il}^{\alpha}(t)\eta_{il}^{\beta}(t)}, \qquad \forall \mathbf{j} \in \boldsymbol{N_i^k}$$

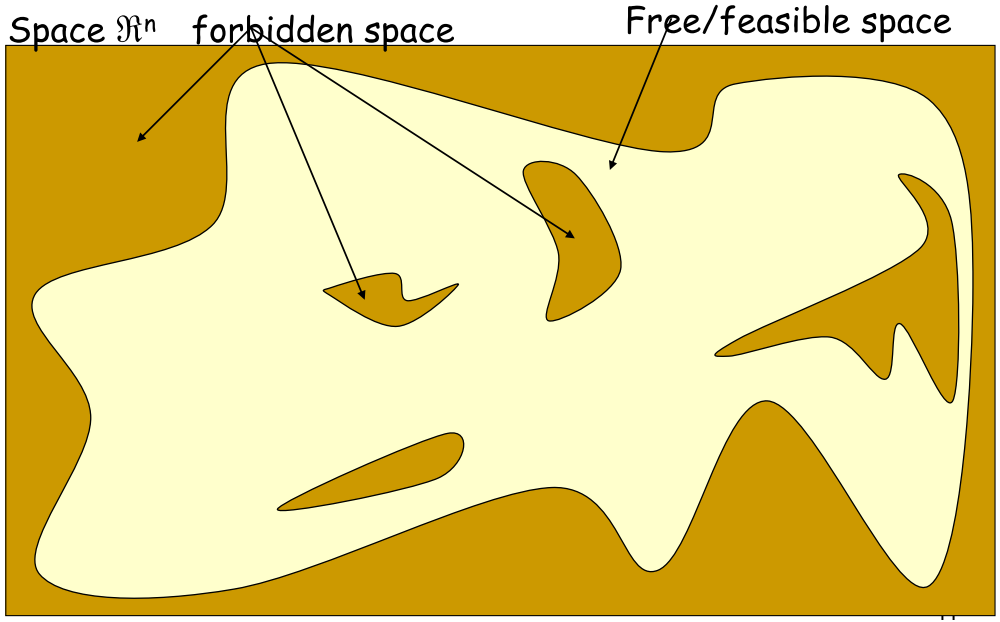
- α为信息启发式因子，反应路径上信息素对蚂蚁选择路径的影响程度，值越大蚂蚁之间的协作性越强
- $\eta_{ij}(t)$为目标启发式函数，表示从节点i到节点j的期望程度，通常取i和j之间路径长度的反比
- β为期望因子，表示启发式的重要性
- 需要在启发式和信息素两者之间做权衡

蚁群算法

- 通过局部探测加记忆机制来搜索可行解
- 优点：
  - 问题的图表示随着最优化过程同步变化，因此可以适应问题的动态性
  - 具有稀疏分布计算架构，可实现并行计算
- 缺点：
  - 计算量大，求解需要时间长
  - 参数需要依靠经验反复调试，不同环境适配不同参数

2.4 概率完备的连通图构建

1. PRM(Probabilistic Roadmap)

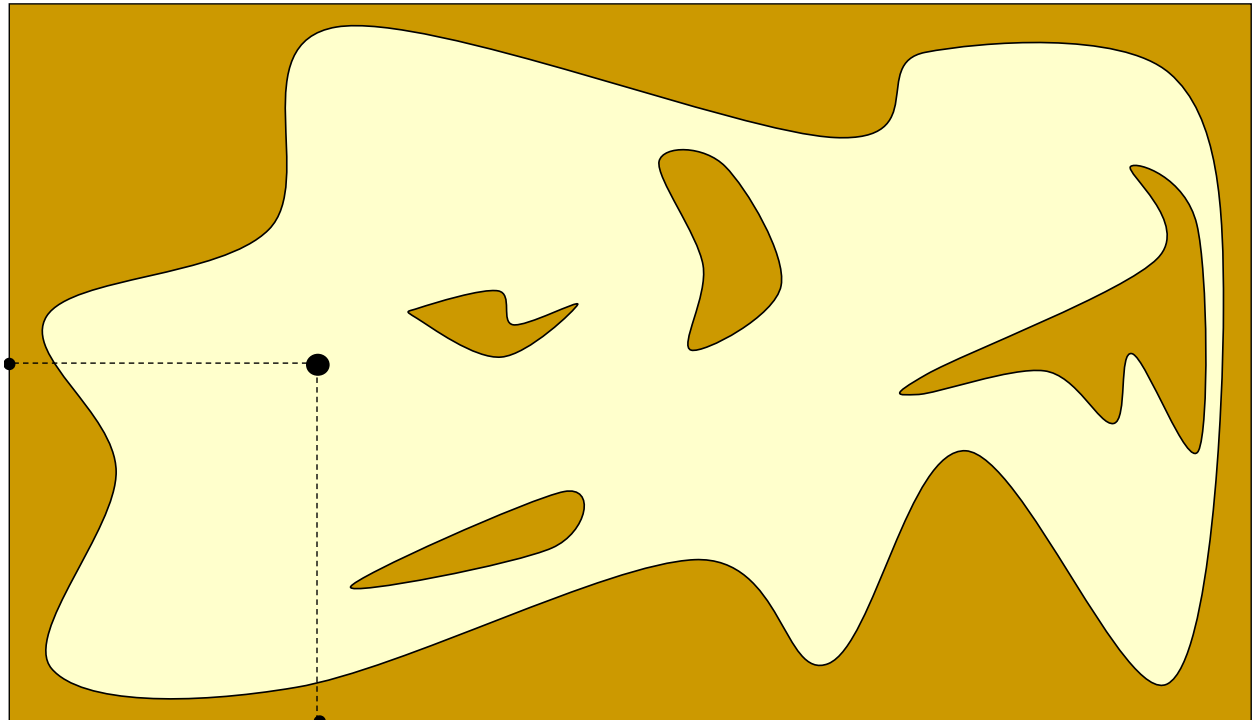


基本思想：

通过随机采样和碰撞检测找到自由位形空间中的路径点和无碰路径，构建连通图

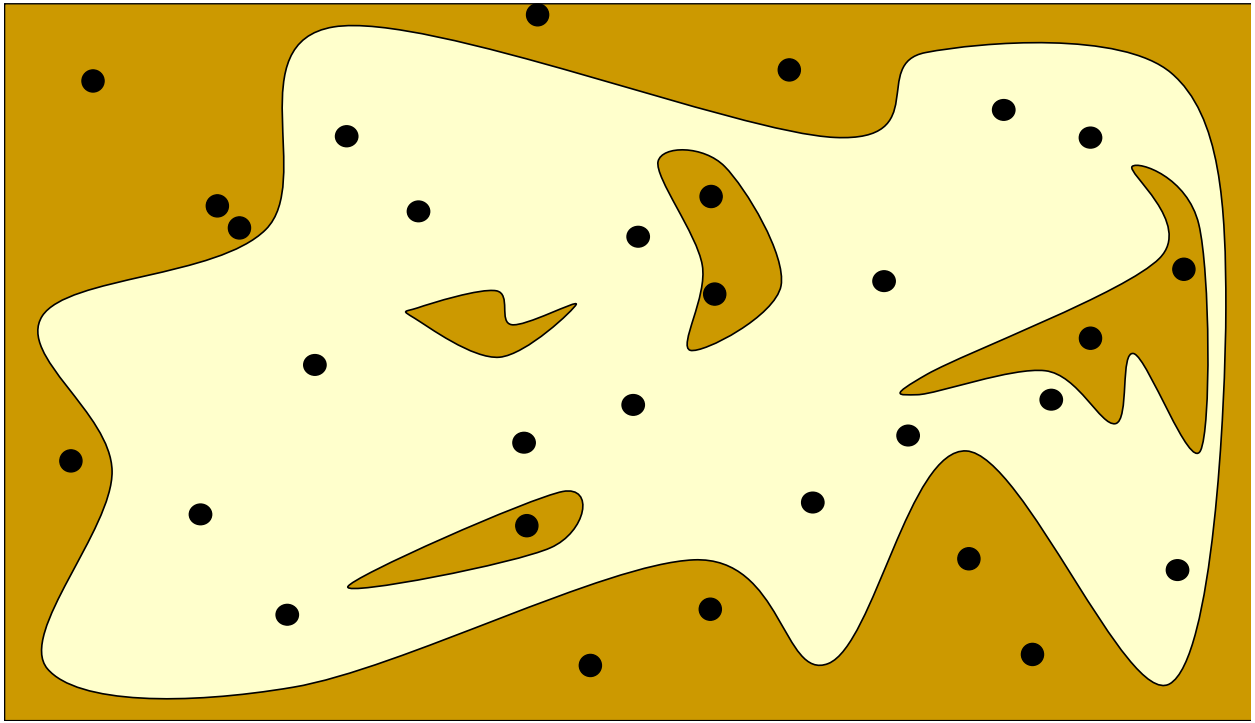
1. PRM(Probabilistic Roadmap)

在位形空间坐标系中随机取点



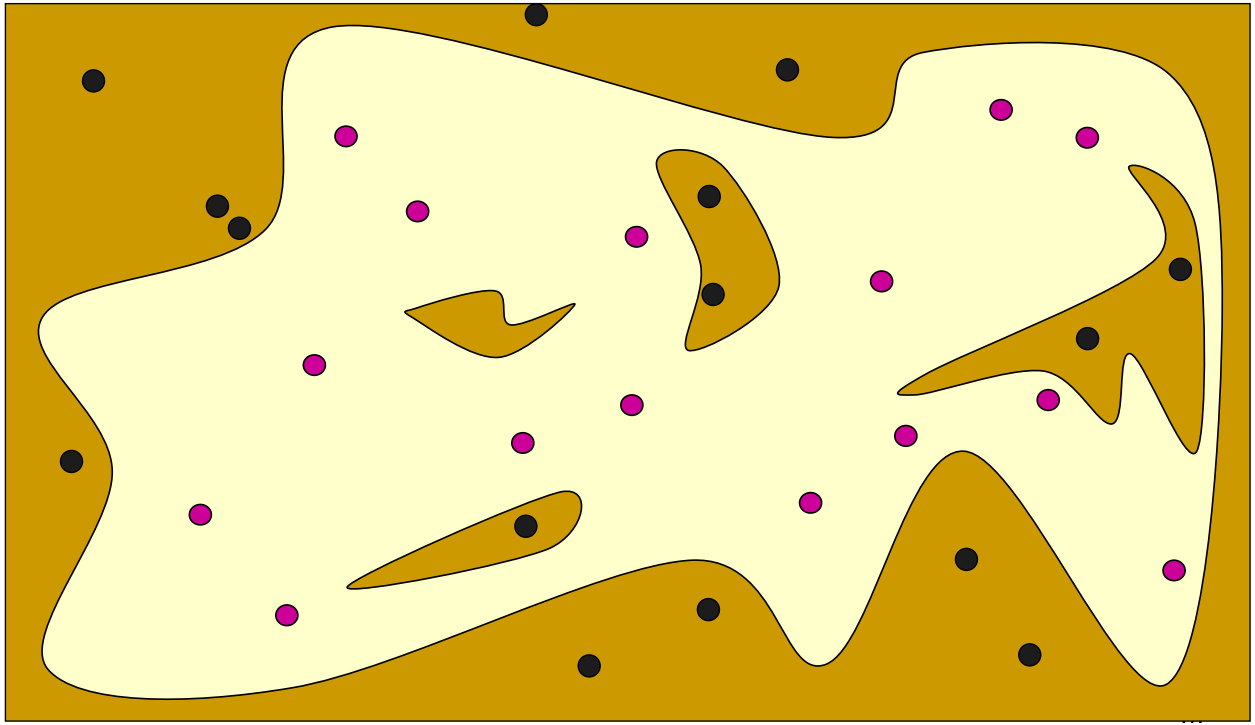
1. PRM(Probabilistic Roadmap)

在位形空间坐标系中随机取点



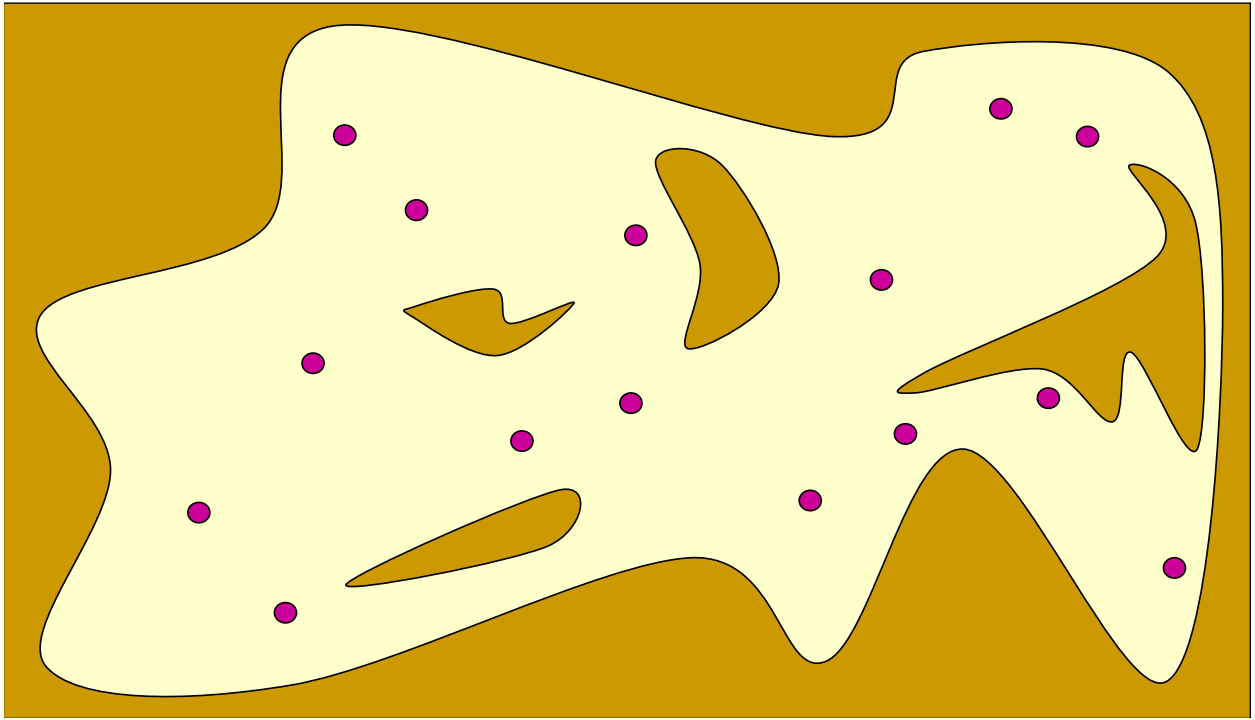
1. PRM(Probabilistic Roadmap)

对采样的姿态进行碰撞检测



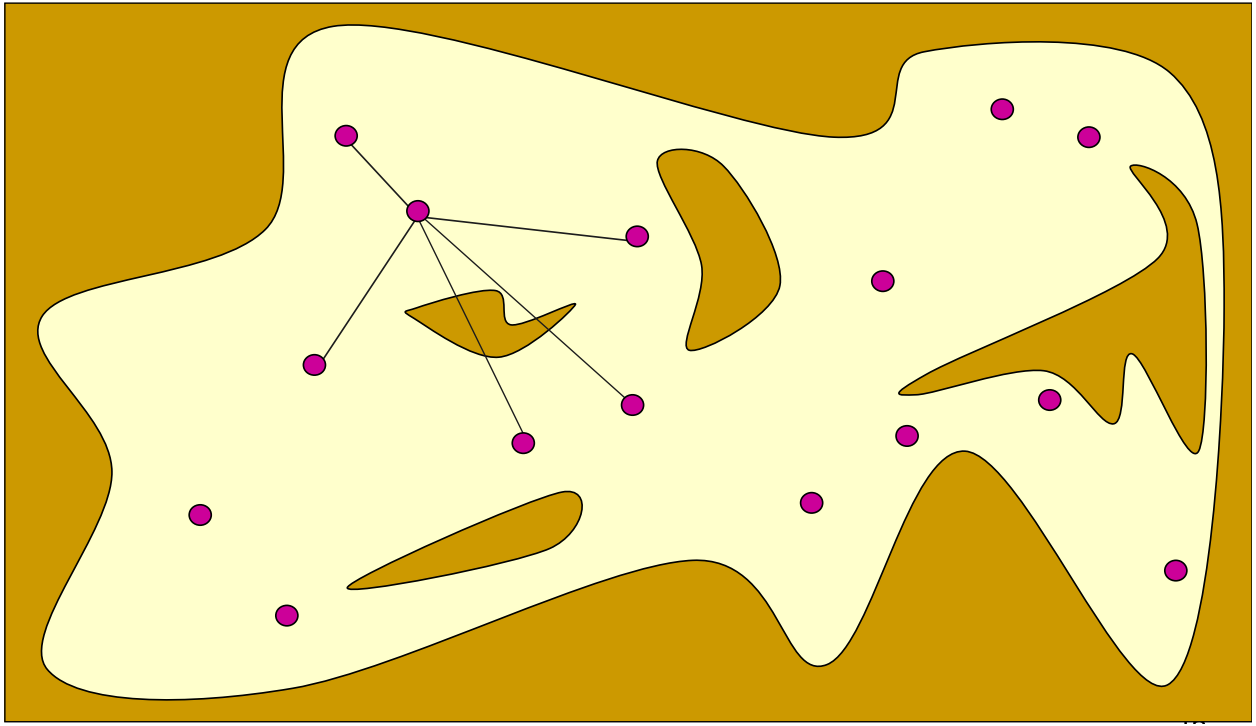
1. PRM(Probabilistic Roadmap)

无碰撞姿态成为图节点



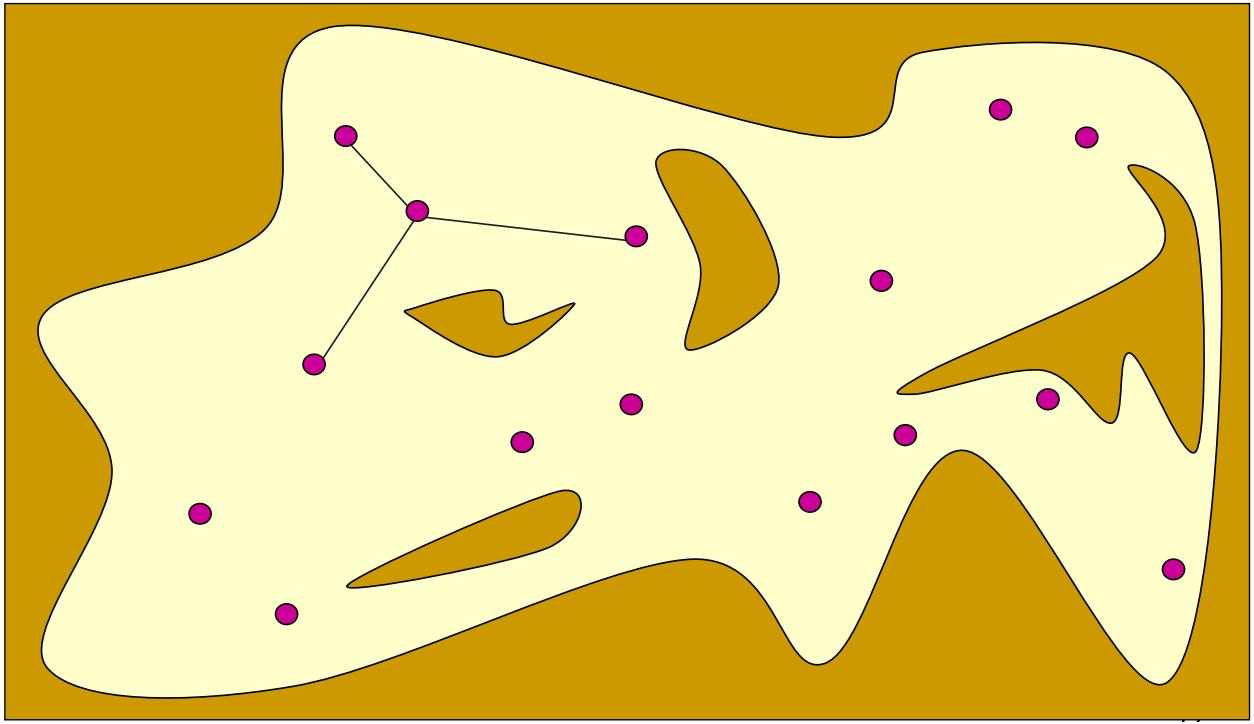
1. PRM(Probabilistic Roadmap)

每个图节点和其最近相邻的k个节点直线连接



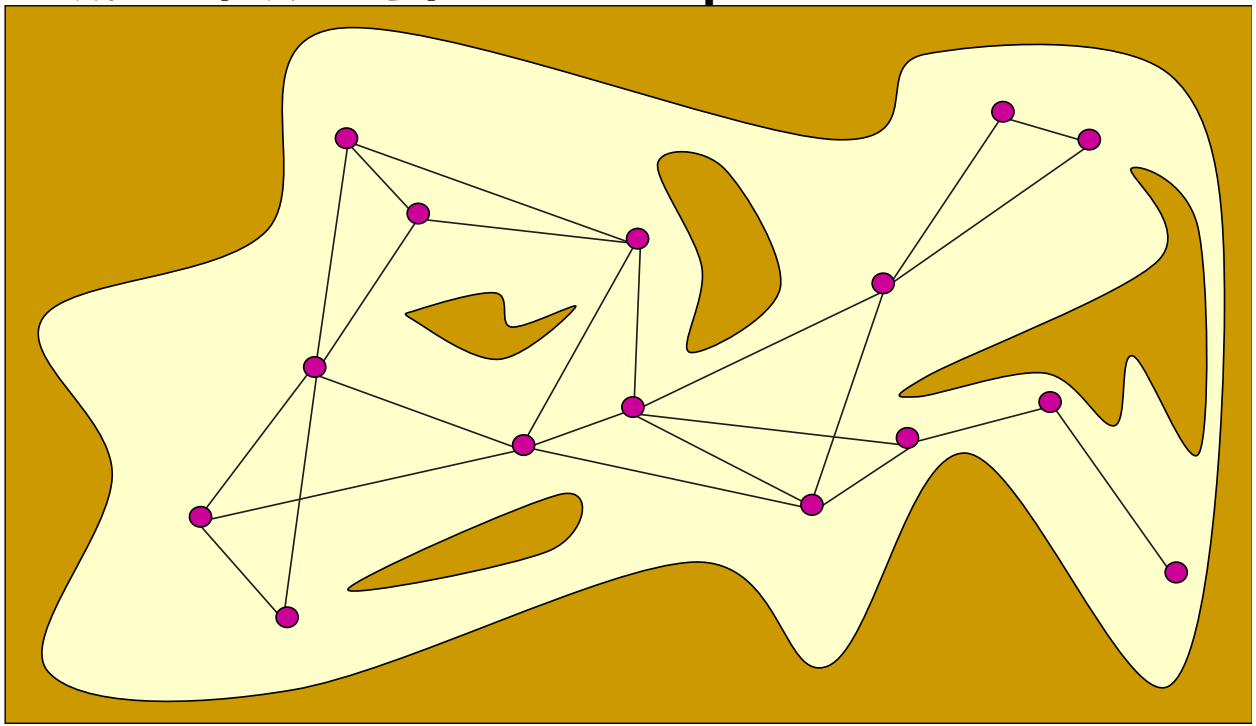
1. PRM(Probabilistic Roadmap)

保留无碰路径为图的边



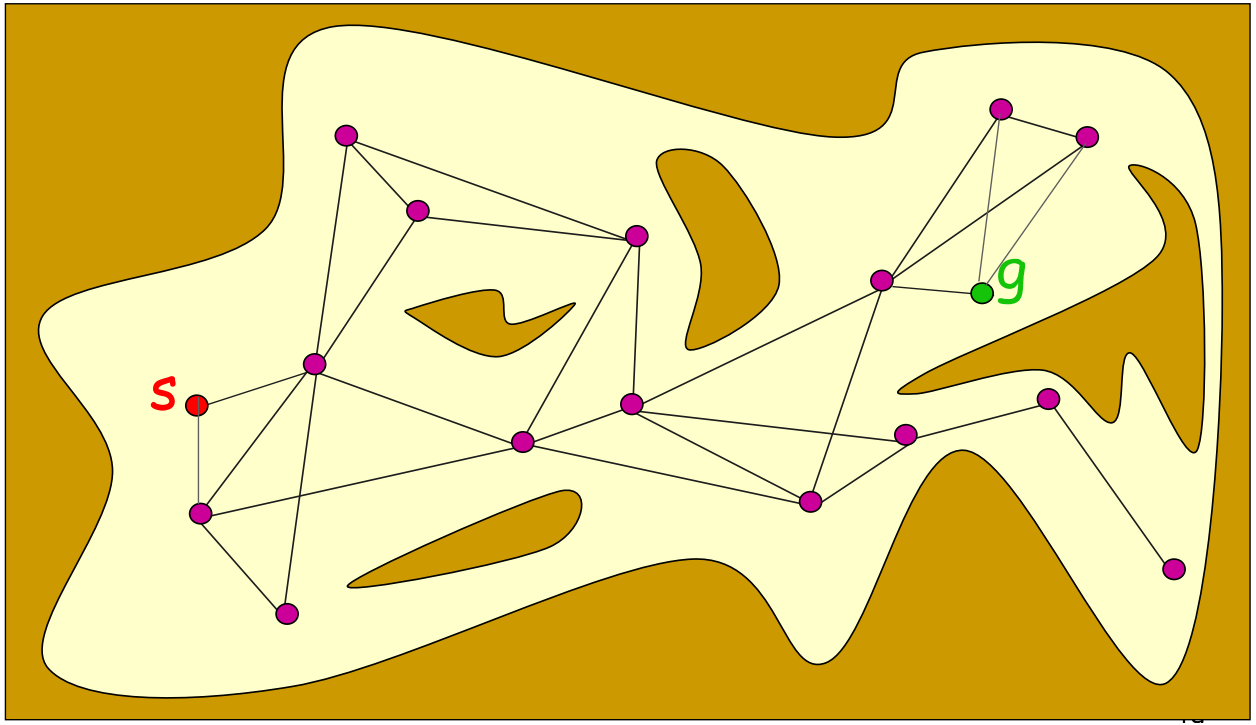
PRM(Probabilistic Roadmap)

构成自由位形空间中的Roadmap



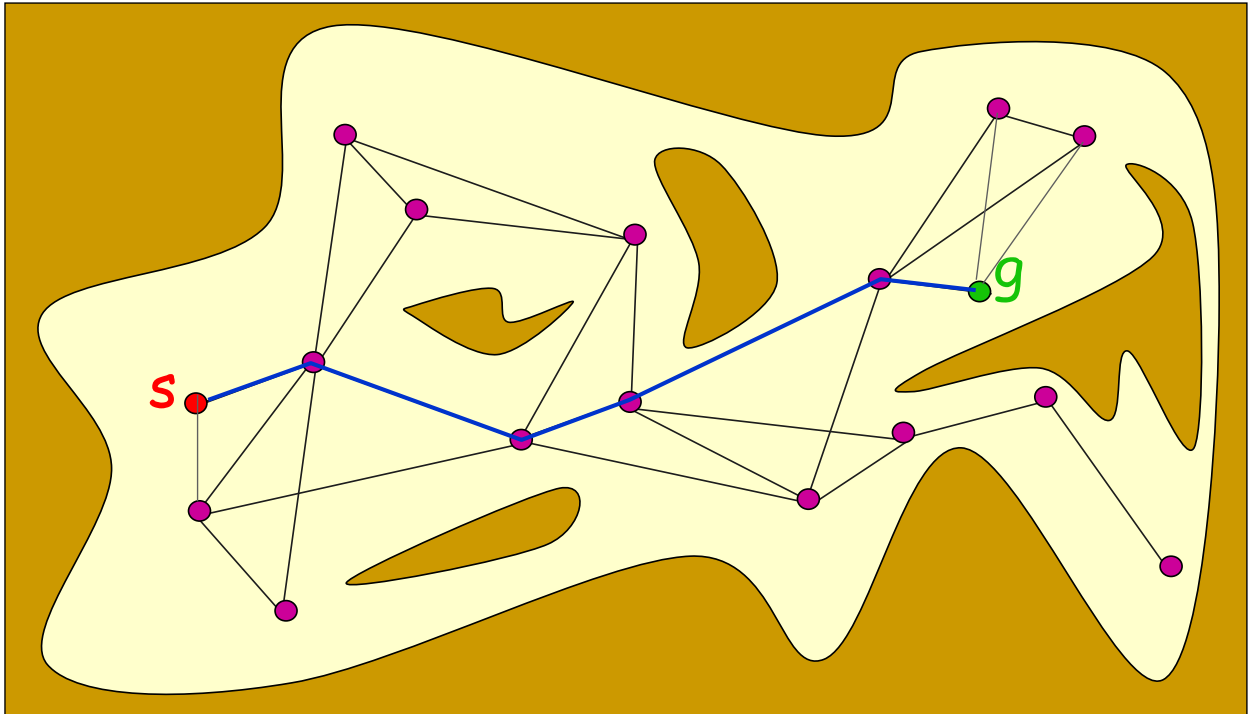
1. PRM(Probabilistic Roadmap)

加入起始点和终止点



1. PRM(Probabilistic Roadmap)

在PRM中搜索一条从起始点到终止点的路径



Algorithm 6 Roadmap Construction Algorithm

Input:

n : number of nodes to put in the roadmap

k : number of closest neighbors to examine for each configuration

Output:

A roadmap $G = (V, E)$

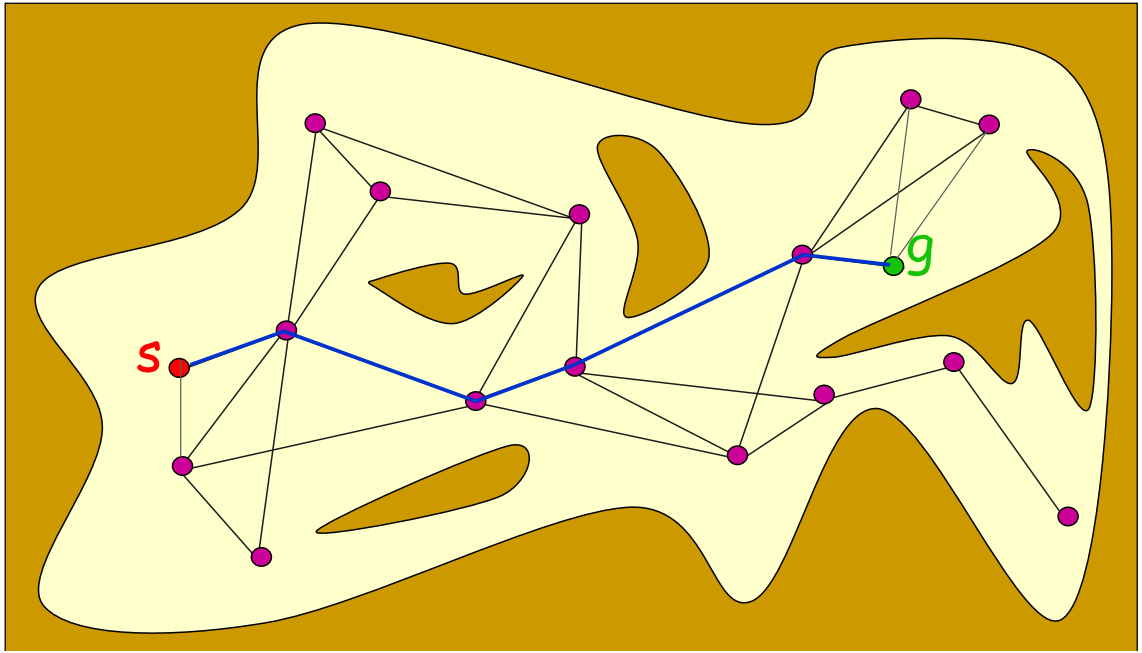
1: $V \leftarrow \emptyset$
2: $E \leftarrow \emptyset$
3: **while** $|V| < n$ **do**
4: **repeat**
5: $q \leftarrow$ a random configuration in $\mathcal{Q}$
6: **until** q is collision-free
7: $V \leftarrow V \cup \{q\}$
8: **end while**
9: **for all** $q \in V$ **do**
10: $N_q \leftarrow$ the k closest neighbors of q chosen from V according to *dist*
11: **for all** $q' \in N_q$ **do**
12: **if** $(q, q') \notin E$ **and** $\Delta(q, q') \neq$ `NIL` **then**
13: $E \leftarrow E \cup \{(q, q')\}$
14: **end if**
15: **end for**
16: **end for**

PRM需要考虑的几个问题

随机位形选择	通常采用均匀随机采样方式
寻找最近邻点	可以采用KD树方法加速
生成局部路径	在一定范围内直接连接图节点
检查路径无碰	可以增量式取点或者二分法取点，判断点是否在障碍物区域内

PRM需要考虑的几个问题

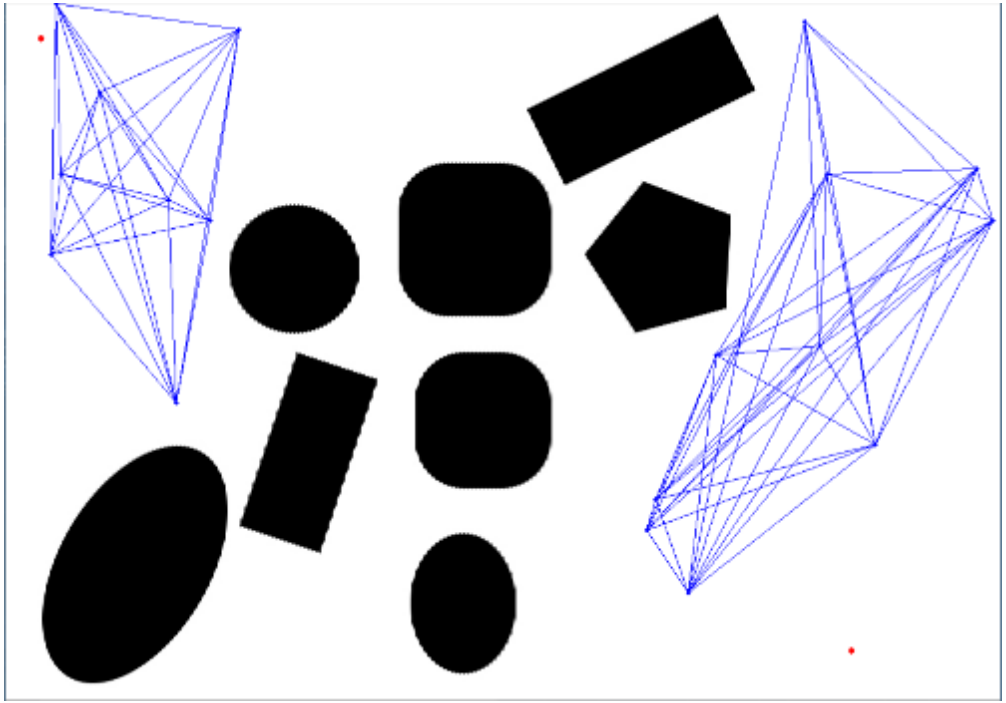
- 进行路径优化



PRM需要考虑的几个问题

- 非全联通情况

当环境复杂，存在狭窄通道等困难场景时，由于PRM对自由位形空间的近似覆盖，存在图非全联通的情况



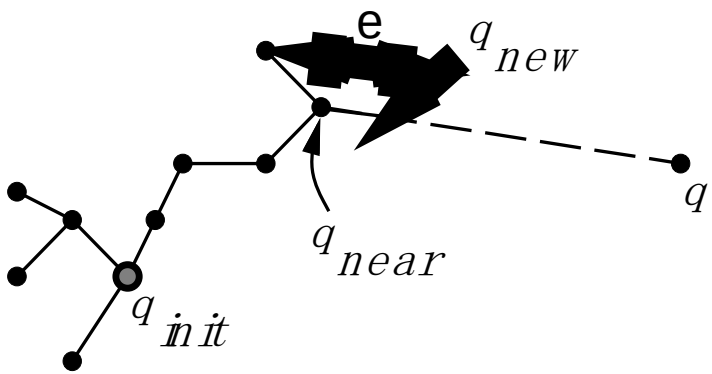
可查阅文献找出解决非全联通问题的解决方案

PRM

- 优点：
  - 简化了对环境的解析计算，可以快速构建得到行车图
  - 适用于高维度自由位形空间中的规划
  - 是一个近似完备的路径规划方法
- 缺点：
  - 对自由空间连通性表达的完整性依赖于采样次数
  - 从算法通用性上来讲难以评估需要多少时间做充分采样
  - 不考虑机器人执行的可行性

2. RRT(Rapid-Exploring Random Tree)

- 1998年由美国爱荷华州立大学Steven M.Lavalle教授提出
- 基本思想：
  - 连通图采用树的形式，以起始点作为树的根节点
  - 采用在空间中随机采样、连接树中最近节点的方式拓展树
  - 考虑机器人的运动执行能力
  - 通过树结构可以直接回溯得到路径



2. RRT(Rapid-Exploring Random Tree)

Algorithm 1: RRT Algorithm

Input: $\mathcal{M}$,x_{init},x_{goal}
Result: A path Γ from x_{init} to x_{goal}
$\mathcal{T}$.init();
for $i = 1$ *to* n **do**
  $x_{rand} \leftarrow Sample(\mathcal{M})$;
  $x_{near} \leftarrow Near(x_{rand}, \mathcal{T})$;
  $x_{new} \leftarrow Steer(x_{rand}, x_{near}, StepSize)$;
  $E_i \leftarrow Edge(x_{new}, x_{near})$;
  if *CollisionFree*$(\mathcal{M}, E_i)$ **then**
    $\mathcal{T}.addNode(x_{new})$;
    $\mathcal{T}.addEdge(E_i)$;
  if $x_{new} = x_{goal}$ **then**
    Success();

2. RRT(Rapid-Exploring Random Tree)

Algorithm 1: RRT Algorithm

Input: $\mathcal{M}$,x_{init},x_{goal}

Result: A path Γ from x_{init} to x_{goal}

$\mathcal{T}$.init();

以运动规划初始状态x_{init}为根节点，建立搜索树

for $i = 1$ *to* n **do**

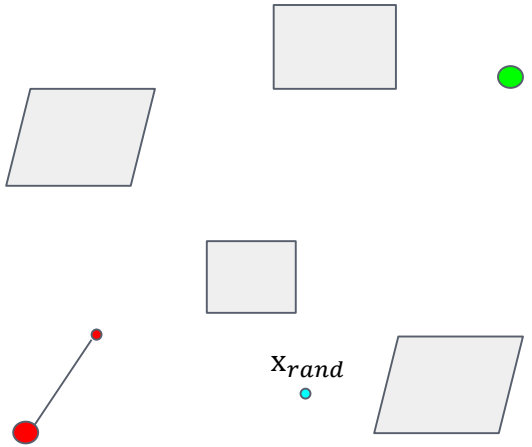
- $x_{rand} \leftarrow Sample(\mathcal{M})$;
- $x_{near} \leftarrow Near(x_{rand}, \mathcal{T})$;
- $x_{new} \leftarrow Steer(x_{rand}, x_{near}, StepSize)$;
- $E_i \leftarrow Edge(x_{new}, x_{near})$;
- **if** *CollisionFree*$(\mathcal{M}, E_i)$ **then**
  - $\mathcal{T}.addNode(x_{new})$;
  - $\mathcal{T}.addEdge(E_i)$;
- **if** $x_{new} = x_{goal}$ **then**
  - Success();

2. RRT(Rapid-Exploring Random Tree)

Algorithm 1: RRT Algorithm

Input: $\mathcal{M}$,x_{init},x_{goal}
Result: A path Γ from x_{init} to x_{goal}
$\mathcal{T}$.init();
for $i = 1$ *to* n **do**
  $x_{rand} \leftarrow Sample(\mathcal{M})$;
  $x_{near} \leftarrow Near(x_{rand}, \mathcal{T})$;
  $x_{new} \leftarrow Steer(x_{rand}, x_{near}, StepSize)$;
  $E_i \leftarrow Edge(x_{new}, x_{near})$;
  if *CollisionFree*$(\mathcal{M}, E_i)$ **then**
    $\mathcal{T}.addNode(x_{new})$;
    $\mathcal{T}.addEdge(E_i)$;
  if $x_{new} = x_{goal}$ **then**
    Success();

在可行空间中随机采样

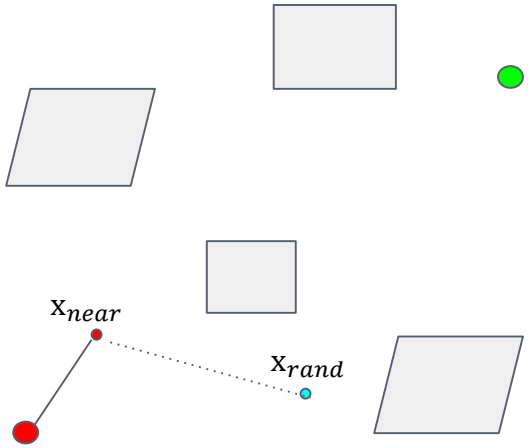


2. RRT(Rapid-Exploring Random Tree)

Algorithm 1: RRT Algorithm

Input: $\mathcal{M}$,x_{init},x_{goal}
Result: A path Γ from x_{init} to x_{goal}
$\mathcal{T}$.init();
for $i = 1$ *to* n **do**
  $x_{rand} \leftarrow Sample(\mathcal{M})$;
  $x_{near} \leftarrow Near(x_{rand}, \mathcal{T})$;
  $x_{new} \leftarrow Steer(x_{rand}, x_{near}, StepSize)$;
  $E_i \leftarrow Edge(x_{new}, x_{near})$;
  if *CollisionFree*$(\mathcal{M}, E_i)$ **then**
    $\mathcal{T}.addNode(x_{new})$;
    $\mathcal{T}.addEdge(E_i)$;
  if $x_{new} = x_{goal}$ **then**
    Success();

找到树中离采样点最近的树节点

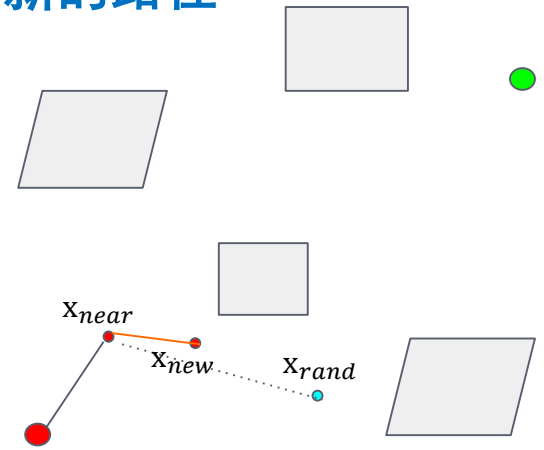


2. RRT(Rapid-Exploring Random Tree)

Algorithm 1: RRT Algorithm

Input: $\mathcal{M}$,x_{init},x_{goal}
Result: A path Γ from x_{init} to x_{goal}
$\mathcal{T}$.init();
for $i = 1$ *to* n **do**
  $x_{rand} \leftarrow Sample(\mathcal{M})$;
  $x_{near} \leftarrow Near(x_{rand}, \mathcal{T})$;
  $x_{new} \leftarrow Steer(x_{rand}, x_{near}, StepSize)$;
  $E_i \leftarrow Edge(x_{new}, x_{near})$;
  if *CollisionFree*$(\mathcal{M}, E_i)$ **then**
    $\mathcal{T}.addNode(x_{new})$;
    $\mathcal{T}.addEdge(E_i)$;
  if $x_{new} = x_{goal}$ **then**
    Success();

根据机器人执行能力生成新节点和新的路径

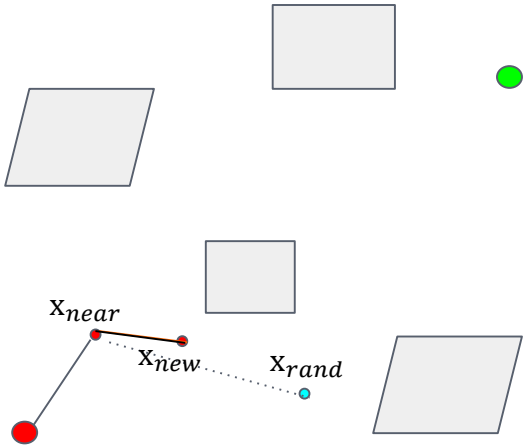


2. RRT(Rapid-Exploring Random Tree)

Algorithm 1: RRT Algorithm

Input: $\mathcal{M}$,x_{init},x_{goal}
Result: A path Γ from x_{init} to x_{goal}
$\mathcal{T}$.init();
for $i = 1$ *to* n **do**
  $x_{rand} \leftarrow Sample(\mathcal{M})$;
  $x_{near} \leftarrow Near(x_{rand}, \mathcal{T})$;
  $x_{new} \leftarrow Steer(x_{rand}, x_{near}, StepSize)$;
  $E_i \leftarrow Edge(x_{new}, x_{near})$;
  if *CollisionFree*$(\mathcal{M}, E_i)$ **then**
    $\mathcal{T}.addNode(x_{new})$;
    $\mathcal{T}.addEdge(E_i)$;
  if $x_{new} = x_{goal}$ **then**
    Success();

根据无碰检测加入树中

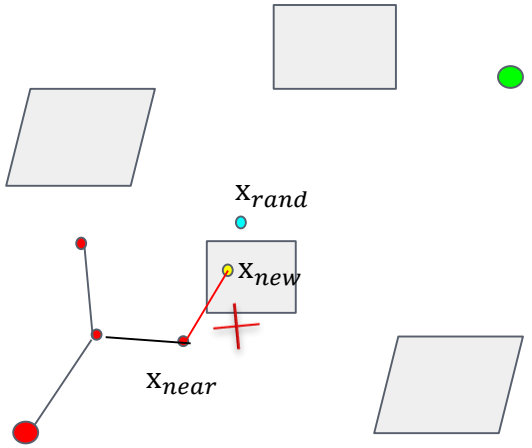


2. RRT(Rapid-Exploring Random Tree)

Algorithm 1: RRT Algorithm

Input: $\mathcal{M}$,x_{init},x_{goal}
Result: A path Γ from x_{init} to x_{goal}
$\mathcal{T}$.init();
for $i = 1$ *to* n **do**
  $x_{rand} \leftarrow Sample(\mathcal{M})$;
  $x_{near} \leftarrow Near(x_{rand}, \mathcal{T})$;
  $x_{new} \leftarrow Steer(x_{rand}, x_{near}, StepSize)$;
  $E_i \leftarrow Edge(x_{new}, x_{near})$;
  if *CollisionFree*$(\mathcal{M}, E_i)$ **then**
    $\mathcal{T}.addNode(x_{new})$;
    $\mathcal{T}.addEdge(E_i)$;
  if $x_{new} = x_{goal}$ **then**
    Success();

不断重复该过程

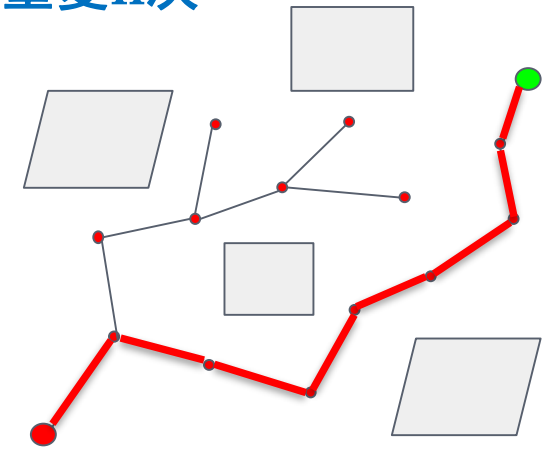


2. RRT(Rapid-Exploring Random Tree)

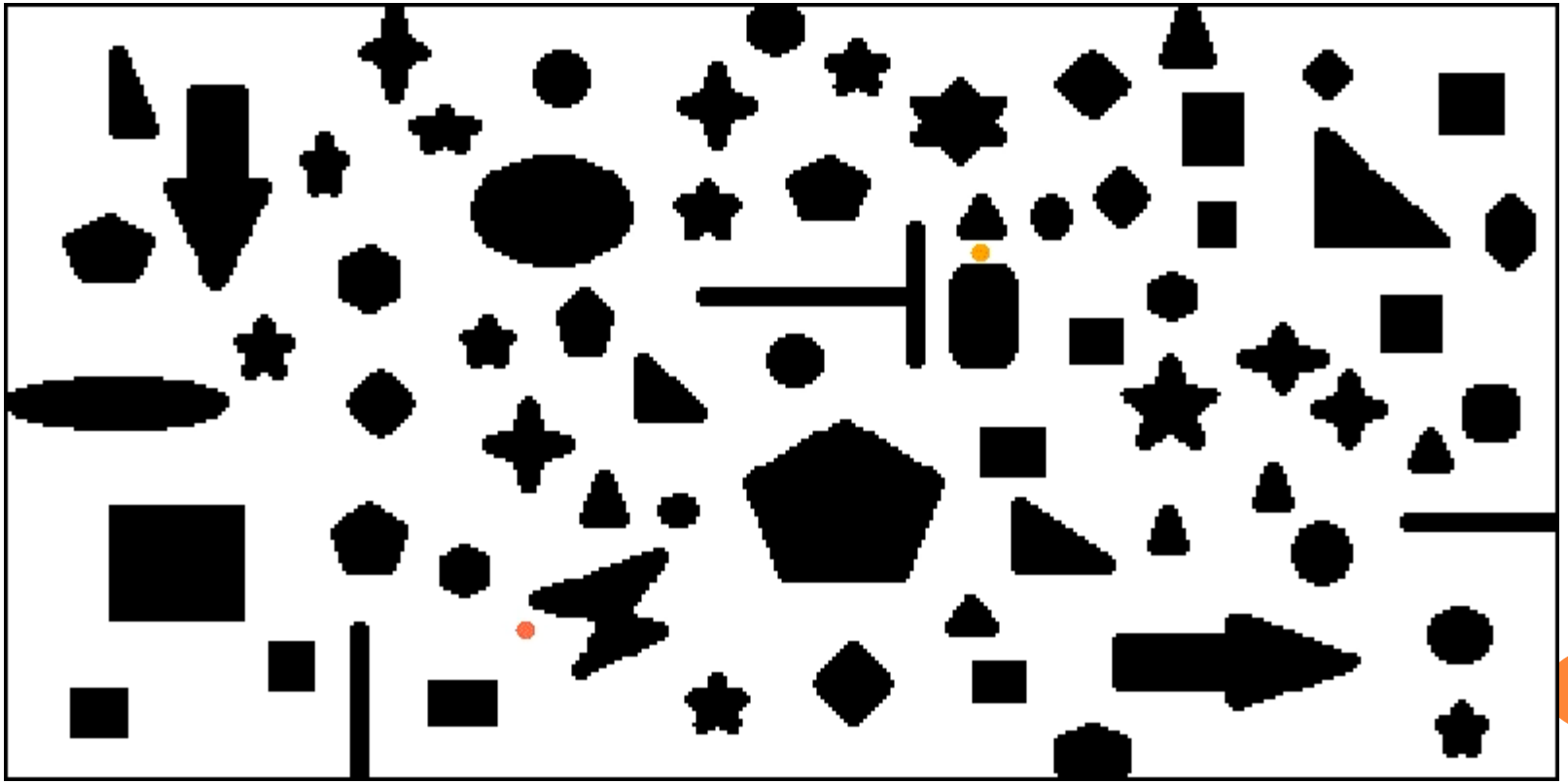
Algorithm 1: RRT Algorithm

Input: $\mathcal{M}$,x_{init},x_{goal}
Result: A path Γ from x_{init} to x_{goal}
$\mathcal{T}$.init();
for $i = 1$ *to* n **do**
  $x_{rand} \leftarrow Sample(\mathcal{M})$;
  $x_{near} \leftarrow Near(x_{rand}, \mathcal{T})$;
  $x_{new} \leftarrow Steer(x_{rand}, x_{near}, StepSize)$;
  $E_i \leftarrow Edge(x_{new}, x_{near})$;
  if *CollisionFree*$(\mathcal{M}, E_i)$ **then**
    $\mathcal{T}.addNode(x_{new})$;
    $\mathcal{T}.addEdge(E_i)$;
  if $x_{new} = x_{goal}$ **then**
    Success();

直到树节点生长到重点区域，或者重复n次

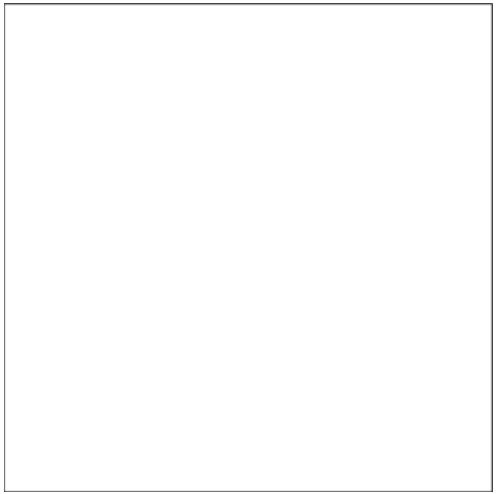


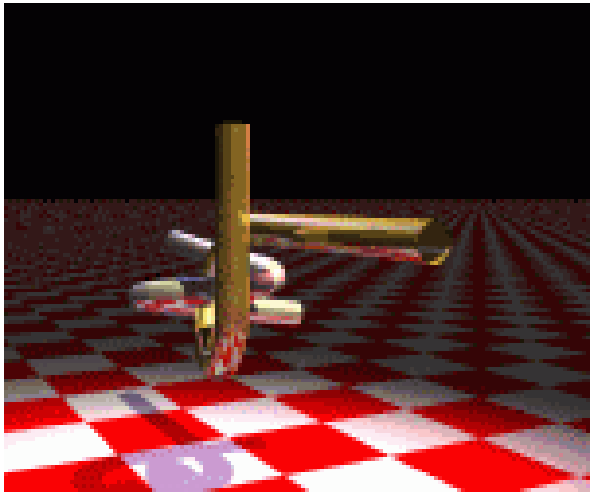
2. RRT(Rapid-Exploring Random Tree)



2. RRT(Rapid-Exploring Random Tree)

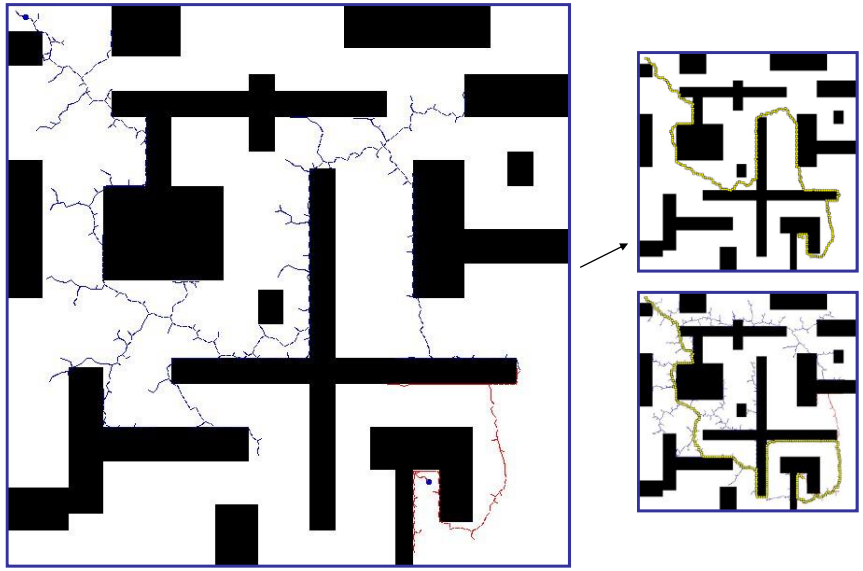
- 影响规划收敛速度的三个步骤
  - *随机状态的采样*
  - *在搜索树中查找与随机状态距离最近的节点*
  - *新生成节点的防碰检测*





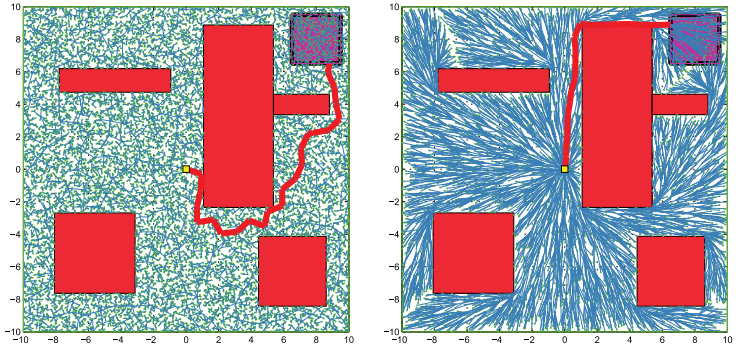
RRT改进1

- 针对问题：RRT扩张偏向状态空间未探测部分，但不是偏向目标点，当环境复杂时计算效率低
- 双向搜索Bidirectional-RRT



RRT改进2

- 针对问题：随机性小步扩展导致路径曲折，成本高
- RRT*：为实现渐近最优，考虑路径成本
  - 寻找树中新节点邻域内到新节点路径最短的节点，建立连接，加入树集合
  - 对树中新节点邻域内节点进行判断，如果从新节点到该节点形成的路径优于现有树中路径，则将该节点父节点修改为新节点



RRT改进2

- 针对问题：随机性小步扩展导致路径曲折，成本高

Algorithm 2: RRT*Algorithm

Input: $\mathcal{M}$,x_{init},x_{goal}
Result: A path Γ from x_{init} to x_{goal}
$\mathcal{T}$.init();
for $i = 1$ *to* n **do**
  $x_{rand} \leftarrow Sample(\mathcal{M})$;
  $x_{near} \leftarrow Near(x_{rand}, \mathcal{T})$;
  $x_{new} \leftarrow Steer(x_{rand}, x_{near}, StepSize)$;
  if *CollisionFree*(x_{new}) **then**
    $X_{near} \leftarrow NearC(\mathcal{T}, x_{new})$;
    $x_{min} \leftarrow ChooseParent(X_{near}, x_{near}, x_{new})$;
    $\mathcal{T}.addNodEdge(x_{min}, x_{new})$;
    $\mathcal{T}.rewire()$;

为实现渐近最优，新节点加入时根据路径成本进行树结构优化

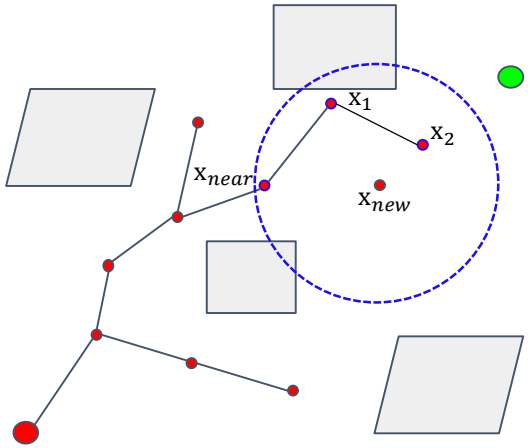
RRT改进2

- 针对问题：随机性小步扩展导致路径曲折，成本高

Algorithm 2: RRT*Algorithm

Input: $\mathcal{M}$,x_{init},x_{goal}
Result: A path Γ from x_{init} to x_{goal}
$\mathcal{T}$.init();
for $i = 1$ *to* n **do**
  $x_{rand} \leftarrow Sample(\mathcal{M})$;
  $x_{near} \leftarrow Near(x_{rand}, \mathcal{T})$;
  $x_{new} \leftarrow Steer(x_{rand}, x_{near}, StepSize)$;
  if *CollisionFree*(x_{new}) **then**
    $X_{near} \leftarrow NearC(\mathcal{T}, x_{new})$;
    $x_{min} \leftarrow ChooseParent(X_{near}, x_{near}, x_{new})$;
    $\mathcal{T}.addNodEdge(x_{min}, x_{new})$;
    $\mathcal{T}.rewire()$;

选择多个邻节点



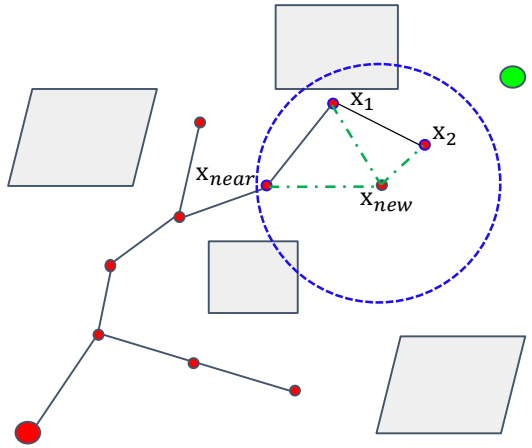
RRT改进2

- 针对问题：随机性小步扩展导致路径曲折，成本高

Algorithm 2: RRT*Algorithm

Input: $\mathcal{M}$,x_{init},x_{goal}
Result: A path Γ from x_{init} to x_{goal}
$\mathcal{T}$.init();
for $i = 1$ *to* n **do**
  $x_{rand} \leftarrow Sample(\mathcal{M})$;
  $x_{near} \leftarrow Near(x_{rand}, \mathcal{T})$;
  $x_{new} \leftarrow Steer(x_{rand}, x_{near}, StepSize)$;
  if *CollisionFree*(x_{new}) **then**
    $X_{near} \leftarrow NearC(\mathcal{T}, x_{new})$;
    $x_{min} \leftarrow ChooseParent(X_{near}, x_{near}, x_{new})$;
    $\mathcal{T}.addNodEdge(x_{min}, x_{new})$;
    $\mathcal{T}.rewire()$;

评估各邻节点作为父节点下的总路径长度



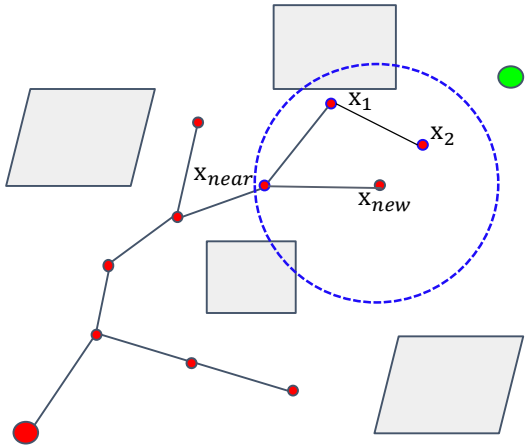
RRT改进2

- 针对问题：随机性小步扩展导致路径曲折，成本高

Algorithm 2: RRT*Algorithm

Input: $\mathcal{M}$,x_{init},x_{goal}
Result: A path Γ from x_{init} to x_{goal}
$\mathcal{T}$.init();
for $i = 1$ *to* n **do**
  $x_{rand} \leftarrow Sample(\mathcal{M})$;
  $x_{near} \leftarrow Near(x_{rand}, \mathcal{T})$;
  $x_{new} \leftarrow Steer(x_{rand}, x_{near}, StepSize)$;
  if *CollisionFree*(x_{new}) **then**
    $X_{near} \leftarrow NearC(\mathcal{T}, x_{new})$;
    $x_{min} \leftarrow ChooseParent(X_{near}, x_{near}, x_{new})$;
    $\mathcal{T}.addNodEdge(x_{min}, x_{new})$;
    $\mathcal{T}.rewire()$;

根据总最短路径选择父节点



RRT改进2

- 针对问题：随机性小步扩展导致路径曲折，成本高

Algorithm 2: RRT*Algorithm

Input: $\mathcal{M}$,x_{init},x_{goal}
Result: A path Γ from x_{init} to x_{goal}
$\mathcal{T}$.init();
for $i = 1$ *to* n **do**
  $x_{rand} \leftarrow Sample(\mathcal{M})$;
  $x_{near} \leftarrow Near(x_{rand}, \mathcal{T})$;
  $x_{new} \leftarrow Steer(x_{rand}, x_{near}, StepSize)$;
  if *CollisionFree*(x_{new}) **then**
    $X_{near} \leftarrow NearC(\mathcal{T}, x_{new})$;
    $x_{min} \leftarrow ChooseParent(X_{near}, x_{near}, x_{new})$;
    $\mathcal{T}.addNodEdge(x_{min}, x_{new})$;
    $\mathcal{T}.rewire()$;

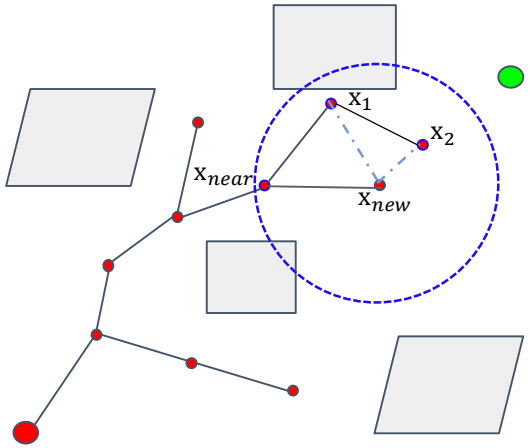
优化邻节点路径，如果从新节点到该节点形成的路径优于现有树中路径，则将该节点父节点修改为新节点

RRT改进2

- 针对问题：随机性小步扩展导致路径曲折，成本高

Algorithm 2: RRT*Algorithm

Input: $\mathcal{M}$,x_{init},x_{goal}
Result: A path Γ from x_{init} to x_{goal}
$\mathcal{T}$.init();
for $i = 1$ *to* n **do**
  $x_{rand} \leftarrow Sample(\mathcal{M})$;
  $x_{near} \leftarrow Near(x_{rand}, \mathcal{T})$;
  $x_{new} \leftarrow Steer(x_{rand}, x_{near}, StepSize)$;
  if *CollisionFree*(x_{new}) **then**
    $X_{near} \leftarrow NearC(\mathcal{T}, x_{new})$;
    $x_{min} \leftarrow ChooseParent(X_{near}, x_{near}, x_{new})$;
    $\mathcal{T}.addNodEdge(x_{min}, x_{new})$;
    $\mathcal{T}.rewire()$;

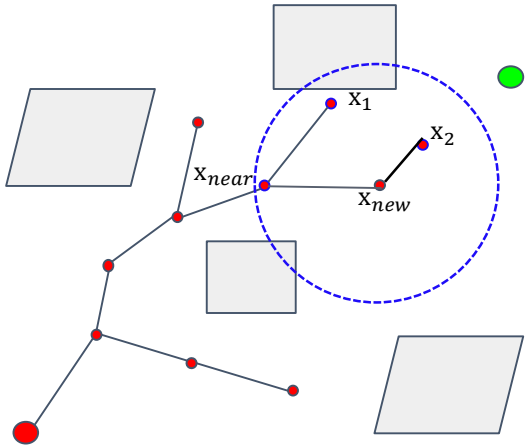


RRT改进2

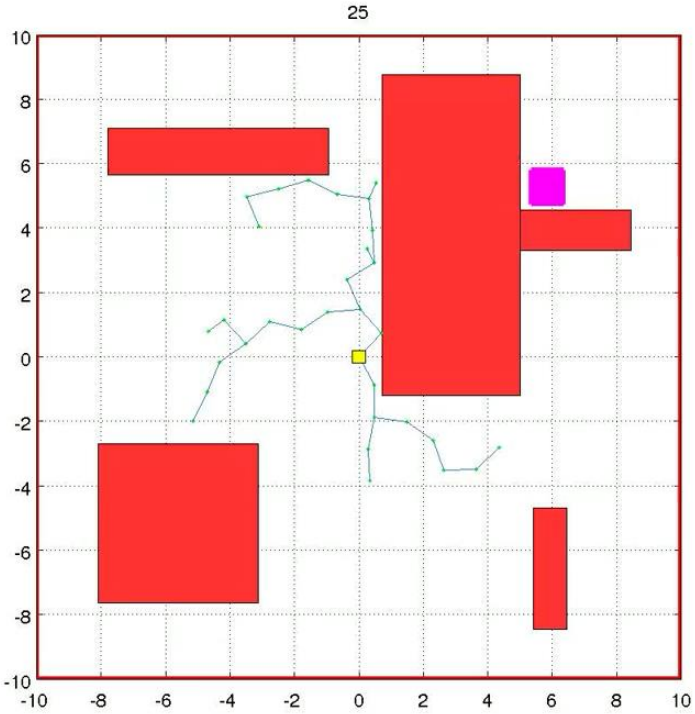
- 针对问题：随机性小步扩展导致路径曲折，成本高

Algorithm 2: RRT*Algorithm

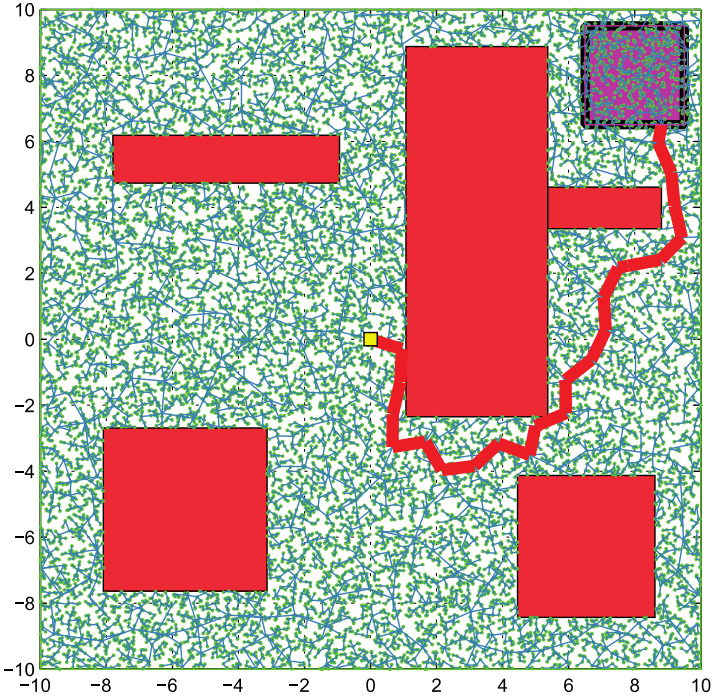
Input: $\mathcal{M}$,x_{init},x_{goal}
Result: A path Γ from x_{init} to x_{goal}
$\mathcal{T}$.init();
for $i = 1$ *to* n **do**
  $x_{rand} \leftarrow Sample(\mathcal{M})$;
  $x_{near} \leftarrow Near(x_{rand}, \mathcal{T})$;
  $x_{new} \leftarrow Steer(x_{rand}, x_{near}, StepSize)$;
  if *CollisionFree*(x_{new}) **then**
    $X_{near} \leftarrow NearC(\mathcal{T}, x_{new})$;
    $x_{min} \leftarrow ChooseParent(X_{near}, x_{near}, x_{new})$;
    $\mathcal{T}.addNodEdge(x_{min}, x_{new})$;
    $\mathcal{T}.rewire()$;

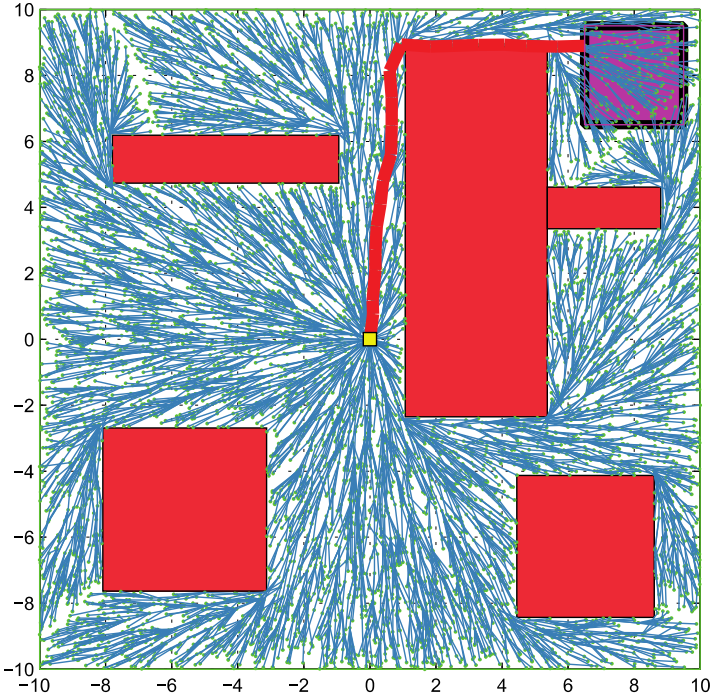


RRT*



RRT*


10
8
6
4
2
0
−2
−4
−6
−8
−10
−10 −8 −6 −4 −2 0 2 4 6 8 10


10
8
6
4
2
0
−2
−4
−6
−8
−10
−10 −8 −6 −4 −2 0 2 4 6 8 10

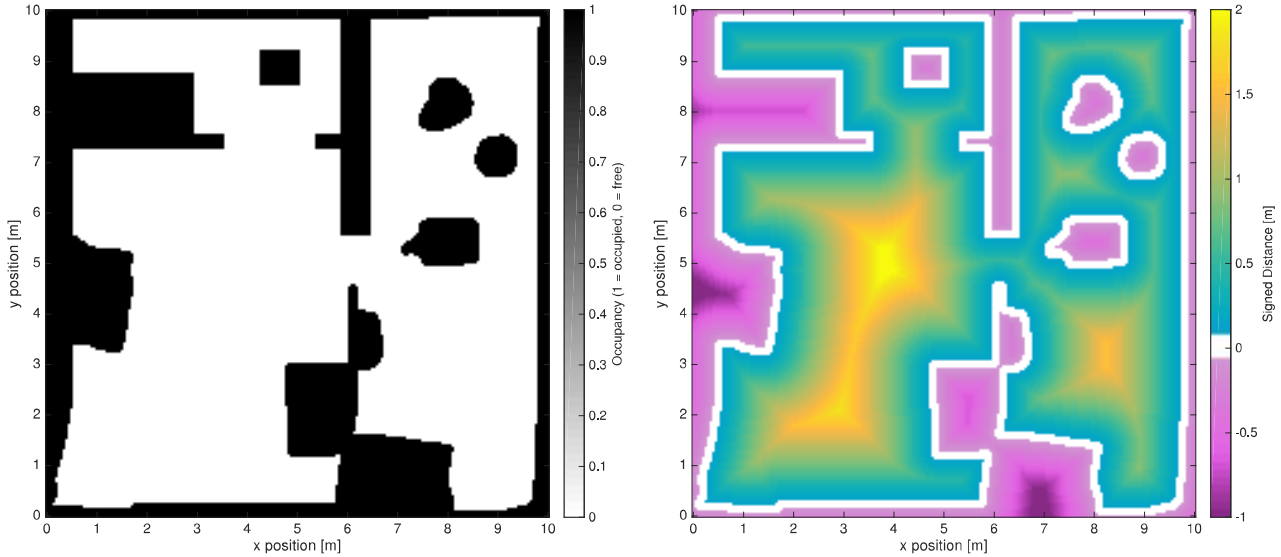
分辨率完备与概率完备方法比较

- 空间离散采样：
  - 分辨率完备是基于解析计算的姿态空间分解
  - 概率完备是基于随机采样生成连通图或者树
- 位形空间连通性表示：
  - 分辨率完备完全表达了自由空间的连通性，高维情况下计算负担重
  - 概率完备方法是近似表达了连通性，但计算快速，只需要计算单个机器人姿态是否存在碰撞，其效率与碰撞检测模块效率相关

2.5 路径规划近期研究

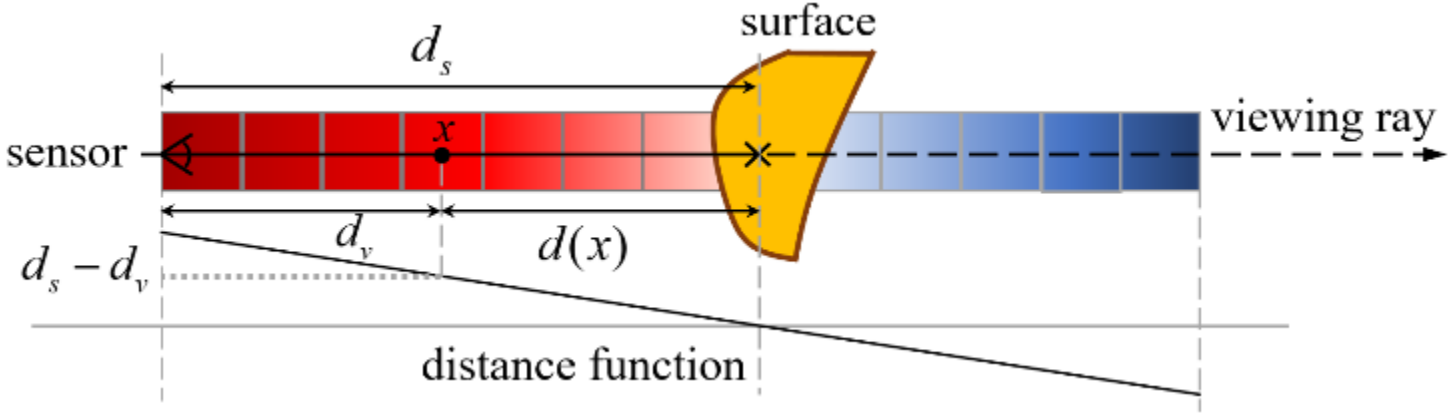
1. 地图形式优化

- **ESDF：有向欧氏距离场，Euclidean Signed Distance Fields**
  - **每个栅格存储距离最近的被占用栅格的欧氏距离，** $d_s - d_v$



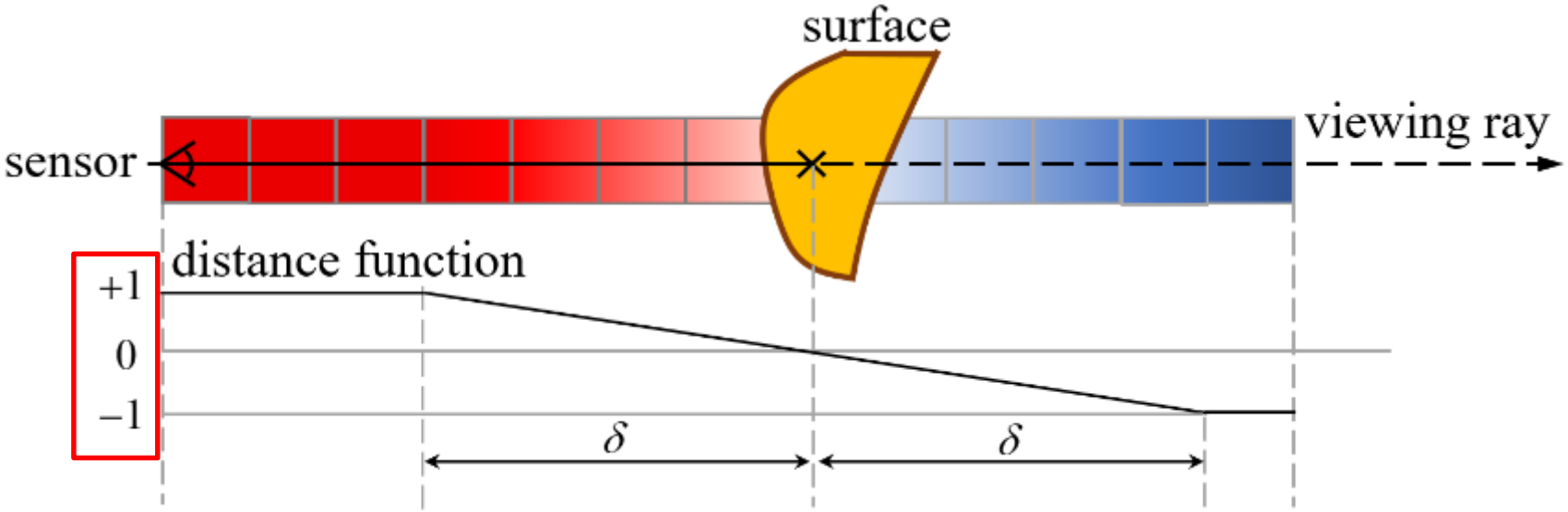
ESDF

- 可以支撑生成GVD (Generalized Voronoi Diagrams)
- 可以更快实现碰撞检测，并可通过正负符号快速判断栅格是位于传感器与障碍物之间、障碍物表面还是障碍物之后



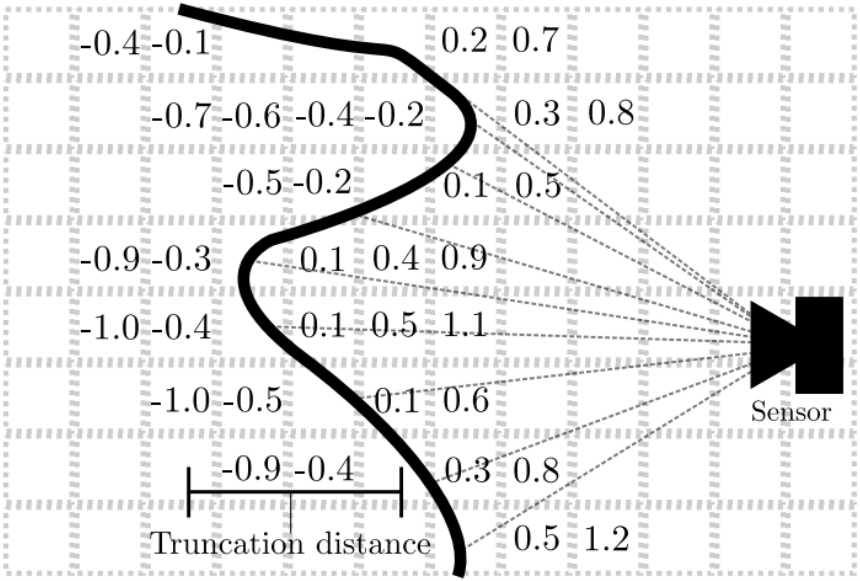
TSDF

- 截断距离场，Truncated Signed Distance Fields, TSDF
  - 只计算障碍物附近一定距离内栅格的ESDF数值，其他截断

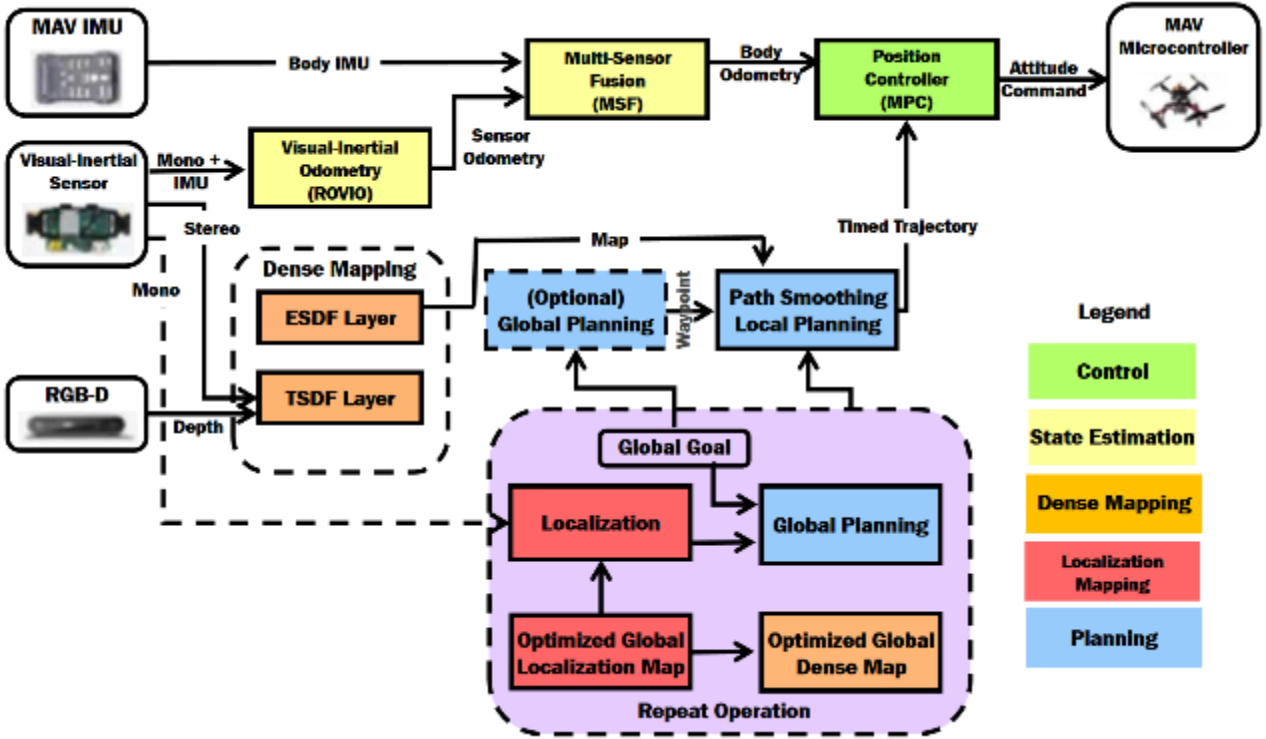


TSDF

- 隐式地表征了环境表面信息，可快速重构高分辨率三维曲面

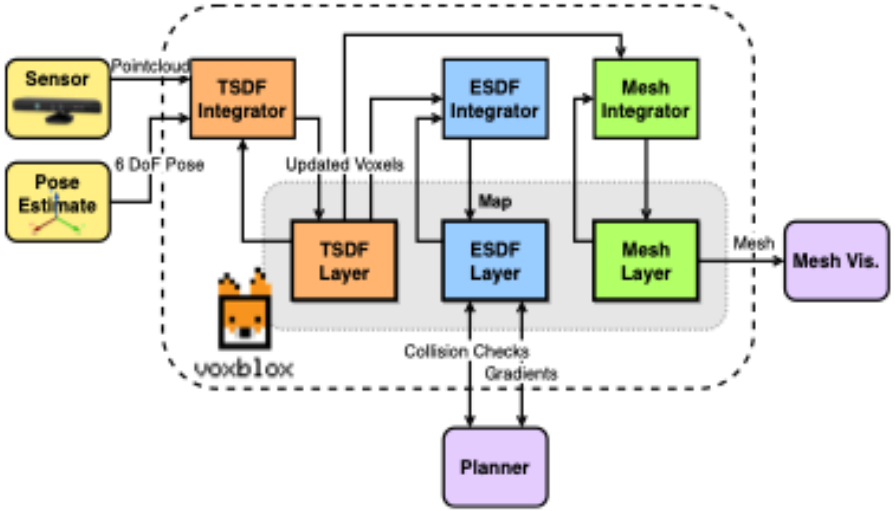


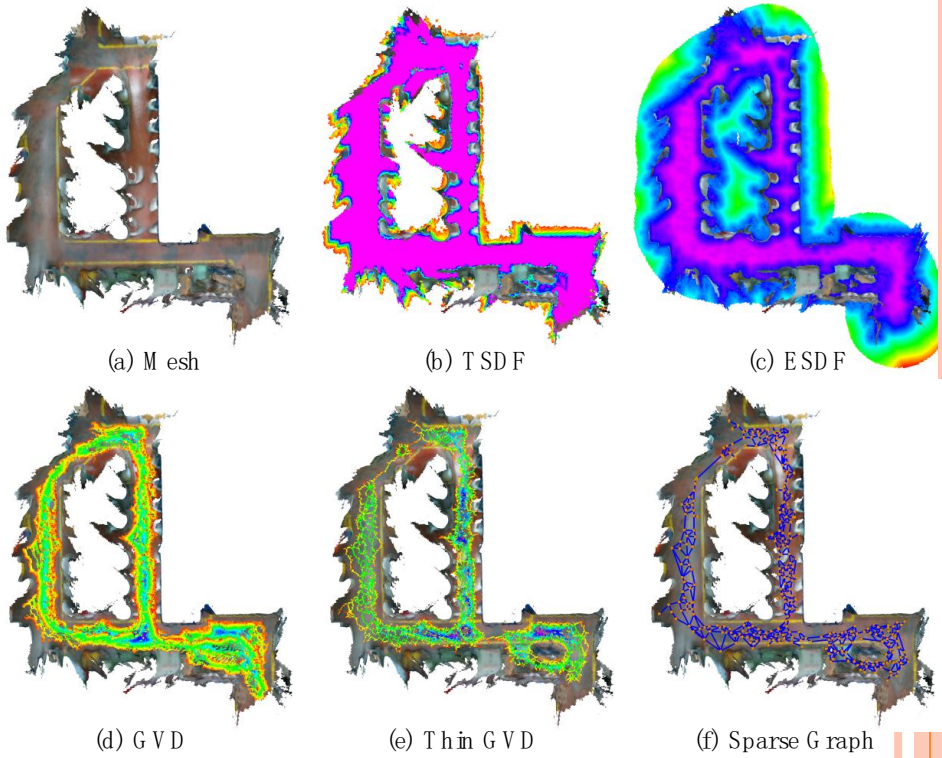
基于ESDF的路径规划



[Juan Nieto etal. (ETHZ) An open-source system for vision-based micro-aerial vehicle mapping, planning, and flight in cluttered environments, JFR, 2020]

基于ESDF的路径规划

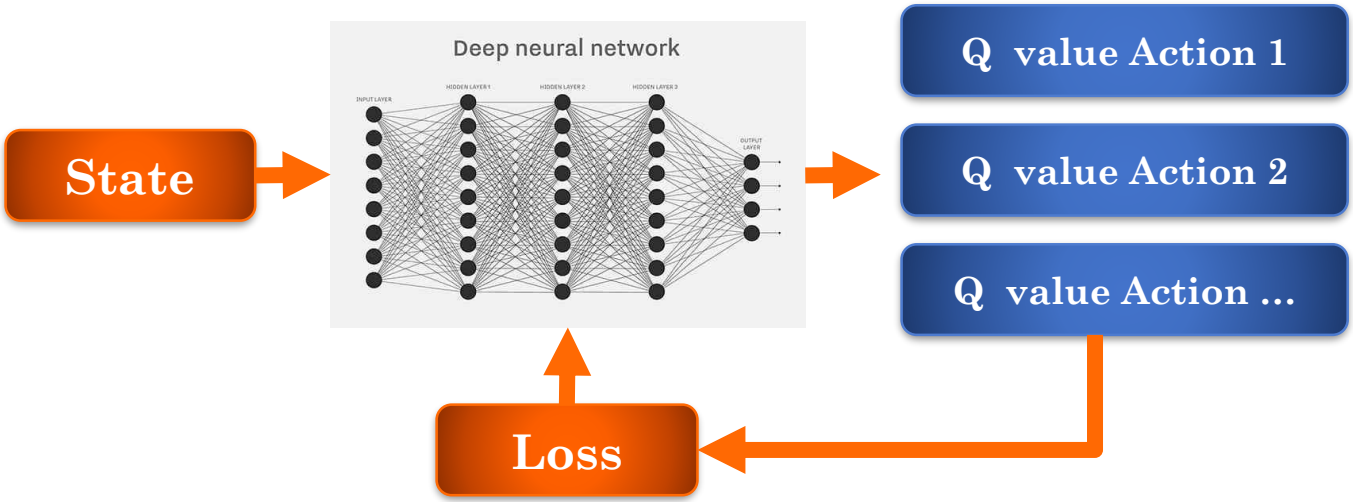




[Juan Nieto etal. (ETHZ) An open-source system for vision-based micro-aerial vehicle mapping, planning, and flight in cluttered environments, JFR, 2020]

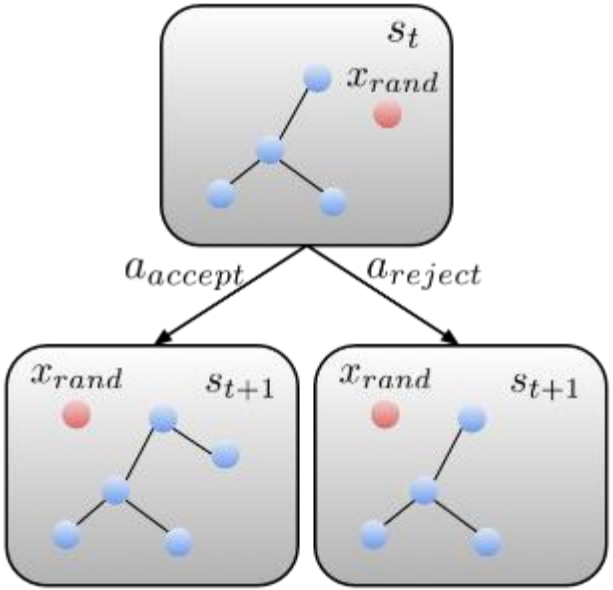
2. 采用学习的路径规划

- 将路径规划问题转化为马尔可夫决策问题（MDP, Markov Decision Process ），采用强化学习/深度强化学习进行求解



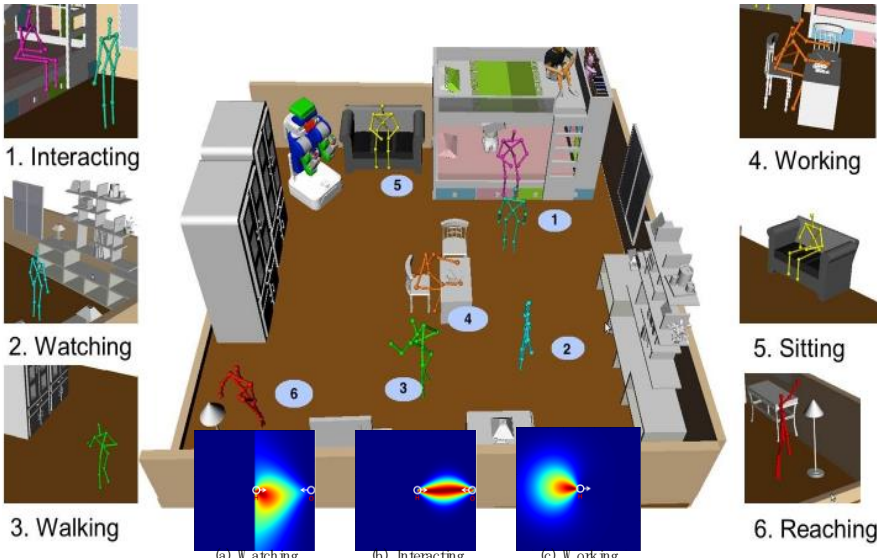
2. 采用学习的路径规划

- 其他研究案例：用深度学习学RRT的采样分布



3. SOCIAL行为路径规划

- 综合地考虑与环境中人的交互等
- e.g.学习地图成本：基于观察/指导反馈方式学习空间中人的行为，获得行为空间分布，构建成本地图



END!